Ministério da Educação
Secretaria de Educação Profissional e Tecnológica
Instituto Federal Catarinense
Câmpus Videira

PAULO SÉRGIO PIERDONÁ

COMPARAÇÃO DE ESTRUTURAS DE ACELERAÇÃO BVH EM SIMULAÇÕES DE TEMPO REAL.

Videira
2026

PAULO SÉRGIO PIERDONÁ

COMPARAÇÃO DE ESTRUTURAS DE ACELERAÇÃO BVH EM SIMULAÇÕES DE TEMPO REAL.

Trabalho de Conclusão de Curso apresentado ao Curso de graduação em Ciência da Computação do Instituto Federal Catarinense – Câmpus Videira para obtenção do título de bacharel em Ciência da Computação.
Orientador: Fábio Jose Rodrigues Pinheiro

Videira
2026

AGRADECIMENTOS

Agradeço, primeiramente, ao meu pai, Sérgio Pierdoná, e à minha mãe, Mariluci Dellani Pierdoná, pelo apoio, incentivo e dedicação ao longo de toda a minha trajetória. Agradeço também aos demais familiares, que de alguma forma contribuíram para minha formação e estiveram presentes durante essa caminhada.

Ao meu orientador, Prof. Fábio Jose Rodrigues Pinheiro, pela orientação, disponibilidade e contribuições durante o desenvolvimento deste trabalho, que foram fundamentais para sua realização.

Aos professores Diego Ricardo Krohl e Manassés Ribeiro, por aceitarem participar da banca examinadora e pelas contribuições dedicadas à avaliação deste trabalho.

Aos meus colegas e amigos de curso, pela companhia, pelas experiências compartilhadas e pelo apoio durante os anos de graduação. As trocas de conhecimento e os momentos vivenciados ao longo desse período fizeram parte importante dessa trajetória.

Por fim, agradeço ao curso de Ciência da Computação do Instituto Federal Catarinense Campus Videira, bem como aos professores e demais profissionais envolvidos, pelo apoio, pela disponibilidade e pelos ensinamentos proporcionados ao longo da graduação, que contribuíram para minha formação acadêmica e profissional.

RESUMO

A evolução da computação gráfica em tempo real tem impulsionado a busca por técnicas de renderização altamente realistas, como o *ray tracing*. No entanto, o elevado custo computacional associado a essa abordagem exige o uso eficiente de estruturas de aceleração geométrica, sendo as *Bounding Volume Hierarchies* (BVHs) amplamente empregadas para otimizar os testes de interseção entre raios e primitivas. Diante disso, este trabalho teve como objetivo analisar e comparar diferentes abordagens de construção e representação de BVHs — especificamente a BVH binária padrão, a *Linear Bounding Volume Hierarchy* (LBVH), a *Binned Surface Area Heuristic* (BSAH) e a *8-Wide Bounding Volume Hierarchy* (BVH8) —, avaliando seus impactos no desempenho de renderização em tempo real. Para isso, foi desenvolvida uma aplicação gráfica utilizando a *Application Programming Interface* (API) Vulkan que permite a configuração, execução e coleta sistemática de métricas de *hardware*, consumo de memória e duração de quadros. Os resultados experimentais indicam que abordagens baseadas em heurísticas de área, como a BSAH, proporcionam melhor desempenho durante a travessia e reduzem o tempo de renderização por quadro, embora apresentem maior custo computacional durante a fase de construção. Por outro lado, estruturas largas como a BVH8 demonstram vantagens no tempo de construção, mas elevam o consumo de recursos de memória. O estudo evidencia, portanto, que a escolha da estrutura de aceleração envolve um *trade-off* direto entre o custo de pré-processamento e a eficiência de travessia, fornecendo uma base sólida para futuras investigações em renderização de alto desempenho.

Palavras-chave: *Ray tracing*. Estruturas de aceleração. *Bounding Volume Hierarchies*. Vulkan. Computação gráfica em tempo real.

ABSTRACT

The evolution of real-time computer graphics has driven the search for highly realistic rendering techniques, such as ray tracing. However, the high computational cost associated with this approach requires the efficient use of geometric acceleration structures, with Bounding Volume Hierarchies (BVHs) being widely used to optimize intersection tests between rays and primitives. Therefore, this work aimed to analyze and compare different approaches for constructing and representing BVHs — specifically standard binary BVH, LBVH, BSAH, and BVH8 — evaluating their impacts on real-time rendering performance. For this purpose, a graphical application was developed using the Vulkan API, enabling the configuration, execution, and systematic collection of hardware metrics, memory consumption, and frame durations. The experimental results indicate that area heuristic-based approaches, such as BSAH, provide better traversal performance and reduce frame rendering time, despite having a higher computational cost during the construction phase. On the other hand, wide structures such as BVH8 demonstrate advantages in construction time, but increase memory resource consumption. The study demonstrates that selecting an acceleration structure involves a direct trade-off between preprocessing cost and traversal efficiency, providing a solid foundation for future investigations in high-performance rendering.

Key-words: Ray tracing. Acceleration structures. Bounding Volume Hierarchies. Vulkan. Real-time computer graphics.

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de *Index Buffer* 20
Figura 2 – Exemplo do funcionamento do *ray casting* 22
Figura 3 – Exemplo do funcionamento do *Whitted ray casting* 23
Figura 4 – Exemplo de triângulo sendo recortado 25
Figura 5 – Funções de aresta 3D para um triângulo 26
Figura 6 – Caso em que primitivas competem pelo mesmo pixel 27
Figura 7 – Imagem path-traced (esquerda). Imagem suavizada (centro). A qualidade de referência renderizada com 2048 amostras por pixel (direita) 29
Figura 8 – Exemplo de BVH 30
Figura 9 – Exemplo de travessia de BVH para encontrar a interseção, descartamos toda a subárvore do nó C, uma vez que o raio não intersecta a caixa delimitadora associada a ela 31
Figura 10 – Ordenação da curva de Morton e os códigos de Morton correspondentes . . 33
Figura 11 – Ilustração da técnica de *binning*. À esquerda: *binning* de um nó BVH com dois planos divididos em um eixo. Os valores *Axis-aligned Bounding boxes* (AABBs) são recalculados para cada parte das primitivas cortadas. À direita: as caixas delimitadoras dos nós filhos esquerdo e direito para o segundo plano dividido 35
Figura 12 – Organização das etapas da aplicação 41
Figura 13 – Funcionamento dos *batches* 44
Figura 14 – Quadro renderizado no instante de 20s de simulação, obtido via *Nsight Graphics*. 60
Figura 15 – Quadro renderizado no instante de 10s de simulação com instâncias sobrepostas. 61
Figura 16 – Visão geral da distribuição das instâncias geométricas no ambiente virtual. . 62
Figura 17 – Tamanho total das *Bottom-Level Acceleration Structure* (BLAS)s de cada implementação 63
Figura 18 – Tamanho total da *Top-Level Acceleration Structure* (TLAS) de cada implementação 63
Figura 19 – Uso da banda de escrita da *Random Access Memory* (RAM) 64
Figura 20 – Tamanho das alocações de memória realizadas pelo Vulkan 65

Figura 21 – Tempo de construção das BLASs 67
Figura 22 – Tempo de construção das TLASs 68
Figura 23 – *Nsight Systems*: tempo registrado para o *upload* da TLAS, com aproximadamente 3 ms selecionados 68
Figura 24 – Tempo médio de produção do quadro na *Graphics Processing Unit* (GPU) . 70
Figura 25 – Tempo médio de produção do quadro na *Central Processing Unit* (CPU) . . 71
Figura 26 – Quantidade média de *warps* de pixels simultaneamente ativos por ciclo . . . 72
Figura 27 – Quantidade média de *warps* de pixels simultaneamente ativos por ciclo, expressa em percentual 73
Figura 28 – Percentual médio de emissão de instruções dos *Streaming Multiprocessors* (SMs) utilizado durante a execução 74
Figura 29 – Percentual de ocupação efetiva dos SMs da GPU 74
Figura 30 – Consumo médio de potência da GPU 76
Figura 31 – Linhas representando operações durante o *frame* 77
Figura 32 – Comparação das principais métricas em relação à BVH padrão 81

LISTA DE TABELAS

Tabela 1 – Comparação de desempenho e ocupação da GPU entre as abordagens 79

LISTA DE ACRÔNIMOS

AABB	*Axis-aligned Bounding Box*
AMD	*Advanced Micro Devices*
API	*Application Programming Interface*
BLAS	*Bottom-Level Acceleration Structure*
BSAH	*Binned Surface Area Heuristic*
BV	*Bounding Volume*
BVH	*Bounding Volume Hierarchy*
BVH16	*16-Wide Bounding Volume Hierarchy*
BVH4	*4-Wide Bounding Volume Hierarchy*
BVH8	*8-Wide Bounding Volume Hierarchy*
CBVH	*Compressed Bounding Volume Hierarchies*
CLBVH	*Compressed-Leaf Bounding Volume Hierarchies*
CPU	*Central Processing Unit*
CUDA	*Compute Unified Device Architecture*
GGX	*Trowbridge-Reitz GGX Distribution*
GPU	*Graphics Processing Unit*
HIP	*Heterogeneous Interface for Portability*
HLBVH	*Hierarchical Linear Bounding Volume Hierarchy*
LBVH	*Linear Bounding Volume Hierarchy*
MBVH	*Multi Bounding Volume Hierarchy*
MVH	*Minimal Bounding Volume Hierarchy*
NVML	*NVIDIA Management Library*
NVTX	*NVIDIA Tools Extension Library*

QBVH	*Quad Bounding Volume Hierarchy*
RAM	*Random Access Memory*
SAH	*Surface Area Heuristic*
SBVH	*Split Bounding Volume Hierarchy*
SIMT	*Single Instruction, Multiple Threads*
SM	*Streaming Multiprocessor*
TLAS	*Top-Level Acceleration Structure*
VRAM	*Video Random Access Memory*
WBVH	*Wide Bounding Volume Hierarchy*

SUMÁRIO

1 INTRODUÇÃO 13
1.1 Problema de Pesquisa 13
1.2 Objetivos 14
1.2.1 Objetivo Geral 14
1.2.2 Objetivos Específicos 14
1.3 Justificativa 14
2 FUNDAMENTAÇÃO TEÓRICA 16
2.1 Arquitetura de Hardware e APIs 16
2.1.1 *Hardware* dedicado *Graphics Processing Unit*s (GPUs) 16
2.1.2 CUDA 16
2.1.3 Vulkan 17
2.1.4 Arquitetura das GPUs NVIDIA 17
2.1.5 Métricas de GPU 18
2.1.6 Métricas do *Pipeline* 19
2.1.7 Métricas do sistema 19
2.2 Conceitos e Técnicas de Computação Gráfica 20
2.2.1 Primitivas 20
2.2.2 Efeitos físicos da luz 21
2.2.3 *Ray tracing* 21
2.2.4 Rasterização 24
2.2.5 Abordagem híbrida 27
2.3 Estruturas de aceleração 29
2.3.1 Linear BVH (LBVH) 31
2.3.2 BVH comprimida 32
2.3.3 BVH larga 32
2.3.4 Binned Surface Area Heuristic BVH (BSAH) 34
2.3.5 Hierarchical LBVH (HLBVH) 34
2.3.6 BVH com Divisões Espaciais (SBVH) 34
2.3.7 Estruturas de Aceleração no Vulkan 35
2.4 Trabalhos Correlatos 36

3 DESENVOLVIMENTO 38
3.1 Decisões de Desenvolvimento 38
3.1.1 Estrutura da Aplicação 38
3.1.2 Estruturas de Aceleração 39
3.1.3 Organização da Renderização 40
3.1.4 Configuração das Estruturas *Bounding Volume Hierarchy* (BVH) 42
3.2 Aplicação Desenvolvida 43
3.2.1 Criação de instâncias 43
3.2.2 Criação de luz 44
3.2.3 Criação das BVHs 46
3.2.4 Criação da *Top-Level Acceleration Structure* (TLAS) 48
3.3 Construção e travessia das BVHs 49
3.3.1 BVH 49
3.3.1.1 Construção 50
3.3.1.2 Travessia 50
3.3.2 *Linear Bounding Volume Hierarchy* (LBVH) 51
3.3.3 *Binned Surface Area Heuristic* (BSAH) 52
3.3.4 *8-Wide Bounding Volume Hierarchy* (BVH8) 53
3.4 Configuração dos Experimentos 54
3.4.1 Configuração de Hardware e Software 54
3.4.2 Preparação para os Experimentos 54
3.4.3 Cenas Utilizadas 55
3.4.4 Parâmetros de Execução 55
4 Coleta e Tratamento dos Dados 57
4.1 Execução dos Experimentos 57
4.2 Coleta das Métricas 57
4.3 Tratamento dos Dados 58
5 RESULTADOS E DISCUSSÃO 60
5.1 Cena Desenvolvida e Amostras de Renderização 60
5.2 Apresentação das métricas 62
5.2.1 Consumo de memória 62
5.2.2 *Central Processing Unit* (CPU) 66

5.2.2.1 *Bottom-Level Acceleration Structure* (BLAS) 66
5.2.2.2 TLAS 67
5.2.2.3 Tempo de *upload* 68
5.2.3 Duração do quadro 69
5.2.4 Ocupação dos *Streaming Multiprocessor*s (SMs) e dos *warps* 71
5.2.4.1 *Warps* 71
5.2.4.2 SMs 73
5.2.5 Consumo de energia e temperatura 75
5.2.6 *Shaders* e ocupação dos *warps* 76
5.3 Conclusões 79
5.3.1 Desempenho das estruturas 80
5.3.2 Avaliação da aplicação desenvolvida 82
5.3.3 Trabalhos futuros 83
REFERÊNCIAS 85

1 INTRODUÇÃO

A computação gráfica tem evoluído significativamente nas últimas décadas, impulsionada pela necessidade de gerar imagens cada vez mais realistas em diferentes contextos. Nesse cenário, o uso de algoritmos de *ray tracing* tem se destacado pela sua capacidade de produzir imagens altamente realistas. No entanto, o custo computacional associado a essa técnica ainda representa um desafio, especialmente em aplicações de tempo real.

Para viabilizar o *ray tracing* em tempo real, torna-se fundamental o uso de estruturas de aceleração. As BVHs permitem reduzir significativamente o número de testes de interseção entre raios e primitivas geométricas, evitando que toda a geometria da cena precise ser examinada durante cada consulta. Com o avanço do hardware dedicado, como as GPUs modernas, diferentes estratégias de construção e representação dessas estruturas têm sido propostas, apresentando características distintas em termos de qualidade da hierarquia, custo de construção, utilização de memória e desempenho durante a travessia (Meister; Bittner, 2022).

Além da utilização em *ray tracing*, estruturas baseadas em BVHs também podem ser empregadas em outras aplicações que envolvem consultas espaciais, como processamento de nuvens de pontos, consultas de vizinhança e detecção de colisões. Essa diversidade de aplicações reforça a importância de avaliar o comportamento dessas estruturas sob diferentes condições de execução.

Diante desse cenário, torna-se relevante investigar como diferentes abordagens de construção e representação de BVHs se comportam em aplicações de tempo real, considerando tanto as características das cenas quanto os recursos computacionais disponíveis.

1.1 PROBLEMA DE PESQUISA

Com o avanço das aplicações de renderização em tempo real, torna-se essencial o uso de estruturas de aceleração eficientes, como as variantes de BVH. No entanto, diferentes estratégias de construção e representação dessas estruturas apresentam comportamentos distintos de acordo com as características das cenas e das cargas de trabalho utilizadas.

Além disso, o desempenho de uma BVH não depende exclusivamente do tempo necessário para sua construção. A qualidade da hierarquia e sua representação podem influenciar o desempenho durante a travessia, bem como o uso dos recursos computacionais disponíveis (Meister; Bittner, 2022).

Diante disso, surge o seguinte problema de pesquisa: qual das abordagens de BVH avaliadas apresenta melhor desempenho em aplicações de *ray tracing* em tempo real, considerando diferentes características das cenas?

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Analisar e comparar diferentes abordagens de estruturas de aceleração baseadas em BVH, avaliando seu desempenho em aplicações de *ray tracing* em tempo real, por meio do desenvolvimento de uma aplicação que permita a execução e comparação sistemática dos métodos avaliados.

1.2.2 Objetivos Específicos

- Analisar as características das diferentes abordagens de BVH utilizadas no trabalho;
- Implementar e comparar as estruturas BVH, LBVH, BSAH e BVH8;
- Desenvolver uma aplicação que permita configurar, executar e comparar diferentes abordagens de BVH;
- Avaliar o desempenho das diferentes estruturas em aplicações de *ray tracing* em tempo real;
- Disponibilizar uma aplicação que possa ser utilizada como base para a realização de novos experimentos e para a avaliação de outras abordagens de BVH.

1.3 JUSTIFICATIVA

A utilização de estruturas de aceleração geométrica, como BVH, é fundamental para viabilizar aplicações de alto desempenho em computação gráfica, especialmente em contextos de tempo real. A necessidade de reduzir o custo das consultas geométricas torna essas estruturas relevantes em diferentes aplicações que envolvem processamento espacial.

A relevância dessas estruturas também pode ser observada em dispositivos com recursos computacionais mais restritos. Chung et al. (Chung *et al.*, 2018) demonstraram a utilização de estruturas de aceleração para reduzir o tempo de execução de *path tracing* em um dispositivo móvel real. Além disso, estruturas baseadas em BVHs podem ser utilizadas em

aplicações envolvendo processamento de nuvens de pontos e consultas de vizinhança (Jakob; Guthe, 2021), bem como em detecção de colisões (Larsson; Akenine-Möller, 2006).

Diante disso, este trabalho se justifica pela necessidade de comparar diferentes abordagens de construção e representação de BVHs, considerando métricas de desempenho relevantes para aplicações de tempo real. Além da comparação entre as estruturas avaliadas, a aplicação desenvolvida estabelece um ambiente para execução dos experimentos e pode servir como base para que outros pesquisadores realizem novos testes e avaliem diferentes modelos de BVH.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 ARQUITETURA DE HARDWARE E APIS

2.1.1 *Hardware* dedicado GPUs

No início, a aparência dos gráficos baseados em GPU era um tanto monótona e padronizada devido à falta de programabilidade, mas agora a maioria dos estágios do *pipeline* gráfico é programável. Infelizmente, observou-se que o aumento na programabilidade atingiu um limite, e mais liberdade exigiria a capacidade de alterar a própria estrutura do *pipeline*. Isso, por sua vez, está firmemente enraizado no *hardware* gráfico específico da GPU e não é facilmente modificado (Laine; Karras, 2011).

Uma grande parte do trabalho no *pipeline* gráfico, principalmente a execução de *shaders* de *vertex* (vértice) e *fragment* (fragmento), mas também outros estágios programáveis, é realizada pelos núcleos de *shader* programáveis. Entre esses processos destacam-se *marshaling* (organização de dados) e *scheduling* (agendamento de operações) de dados. Esses processos incluem gerenciamento de filas FIFO, controle de *buffer* de quadro na memória da GPU, busca de dados de entrada do *shader* na memória local antes da execução do *shader*, acionamento de execuções de *shader*, execução de operações de rasterização. Além dessas tarefas, os núcleos também realizam cálculos mais triviais, como a construção de equações de borda e plano, rasterização e *clipping* (Laine; Karras, 2011).

2.1.2 CUDA

A NVIDIA introduziu o *Compute Unified Device Architecture* (CUDA) em 2007. Por meio do CUDA, os núcleos programáveis (*shader cores*) passaram a ser acessíveis para aplicações de propósito geral (Laine; Karras, 2011). Tal abordagem possibilita a implementação de algoritmos fora do *pipeline* gráfico tradicional, ampliando a flexibilidade no desenvolvimento.

A plataforma CUDA apresenta limitações de compatibilidade entre plataformas (Song *et al.*, 2024). A fim de ampliar a portabilidade do código, a *Advanced Micro Devices* (AMD) criou o *Heterogeneous Interface for Portability* (HIP), que permite aos desenvolvedores migrarem o código CUDA existente para plataformas AMD (Advanced Micro Devices, Inc., 2026). Para cenários que exigem maior independência de fornecedor e suporte a diferentes arquiteturas de hardware, o OpenCL surge como uma alternativa multiplataforma para computação paralela, abrangendo CPUs, GPUs e outros dispositivos (Song *et al.*, 2024).

2.1.3 Vulkan

As placas gráficas começaram a oferecer cada vez mais funcionalidades e algumas causam incompatibilidade no código. Como, por exemplo, sintaxe para *shaders*, que geram inconsistência entre fornecedores e GPUs móveis que possuem arquitetura baseada em seus requisitos de energia e espaço. O Vulkan[1] resolve esses problemas de compatibilidade por ser projetado para arquiteturas gráficas modernas. Vulkan reduz a sobrecarga de lidar com diversos *drivers*, permitindo que programadores especifiquem suas intenções por meio de uma API (Alexander Overvoorde, 2023). Diferentemente das outras plataformas, OpenCL, voltadas à computação de propósito geral, o Vulkan é mais direcionado ao desenvolvimento de aplicações gráficas, como renderização.

2.1.4 Arquitetura das GPUs NVIDIA

O CUDA permite definir funções especiais chamadas *kernels*. Diferente de funções comuns, um *kernel* é executado paralelamente por múltiplas *threads* da GPU. Essas funções são executadas por blocos de *thread*, que são grupos de *threads* que têm sua memória compartilhada e sincronizam sua execução para coordenar acessos à memória (NVIDIA Corporation, 2022).

Um *kernel* pode ser executado por múltiplos blocos de *thread*. Os blocos são organizados em um *grid* unidimensional, bidimensional ou tridimensional de blocos de *thread*. O número de blocos de *thread* em um *grid* é geralmente determinado pelo tamanho dos dados sendo processados, que normalmente excede o número de processadores (NVIDIA Corporation, 2022).

A arquitetura da GPU é construída em torno de um conjunto escalável de multiprocessadores de transmissão (SMs) *multithread*. Quando um programa CUDA na CPU hospedeira invoca um *kernel*, os blocos do *grid* são enumerados e distribuídos para os SMs com capacidade de execução disponível. Múltiplos blocos de *thread* (*thread blocks*) podem ser executados simultaneamente em um mesmo SM (NVIDIA Corporation, 2022).

Para gerenciar um número tão grande de *threads*, é empregada uma arquitetura exclusiva para multiprocessamento chamada *Single Instruction, Multiple Threads* (SIMT), que trabalha com o princípio de que uma única instrução controla múltiplas *threads* (NVIDIA Corporation, 2022).

[1] Vulkan é uma *Application Programming Interface* (API) gráfica e de computação multiplataforma mantida pelo *Khronos Group*, projetada para oferecer acesso de baixo nível ao hardware gráfico moderno.

O SM cria, gerencia, agenda e executa *threads* em grupos de 32 *threads* paralelas chamadas *warps*. As *threads* individuais que compõem um *warp* começam juntas no mesmo endereço de programa, mas possuem seu próprio contador de endereço de instrução e estado de registro, sendo, portanto, livres para desviar e executar independentemente (NVIDIA Corporation, 2022).

Quando um SM recebe um ou mais blocos de *threads* para executar, ele os divide em *warps* e cada *warp* é agendado por um agendador de *warps* para execução. A forma como um bloco é particionado em *warps* é sempre a mesma; cada *warp* contém *threads* com IDs de *thread* consecutivos e crescentes, sendo que o primeiro *warp* contém a *thread* 0 até a *thread* 31 (NVIDIA Corporation, 2022).

Um *warp* executa uma instrução por vez; essa instrução é comum para todas as *threads* no *warp*. Portanto, a eficiência máxima é alcançada quando todas as 32 *threads* de um *warp* seguem um caminho de execução. Uma divergência de *warp* (*warp divergence*) ocorre se as *threads* de um *warp* divergirem por meio de um desvio condicional dependente de dados. O *warp* executa cada caminho de divergência, desabilitando e habilitando as *threads*. Essa divergência ocorre apenas dentro de um *warp*; *warps* diferentes executam independentemente (NVIDIA Corporation, 2022).

2.1.5 Métricas de GPU

Para saber o desempenho de cada tipo de BVH, é preciso coletar métricas capazes de representar o comportamento dos recursos computacionais da GPU. Entre essas métricas estão a utilização dos SMs, a execução de *shaders*, o uso da memória da GPU e outros comportamentos da GPU. Essas informações permitem identificar como diferentes estruturas BVH influenciam a utilização dos recursos disponíveis durante a renderização.

O *NVIDIA Nsight Systems* é uma ferramenta de análise de desempenho que permite observar a execução da aplicação ao longo do tempo, relacionando atividades da CPU, GPU, APIs e outros componentes do sistema. A ferramenta também permite coletar métricas de hardware da GPU por meio de amostragem periódica, incluindo informações relacionadas à atividade dos SMs, ocupação de *warps* e largura de banda de memória (NVIDIA, 2026b). Dessa forma, o *Nsight Systems* pode ser utilizado para relacionar o comportamento dos recursos da GPU com as diferentes etapas da aplicação.

2.1.6 Métricas do *Pipeline*

Apesar do *Nsight Systems* conseguir coletar uma grande variedade de métricas, a ferramenta possui limitações. Por exemplo, a ferramenta não apresenta o mesmo nível de detalhamento para as etapas de execução dos *shaders* que ferramentas específicas para análise da GPU. Dessa forma, outras ferramentas podem ser utilizadas para complementar as informações obtidas.

O desempenho de uma aplicação de *ray tracing* não depende apenas da utilização individual dos recursos da GPU, mas também da forma como as diferentes etapas do *pipeline* são executadas e sincronizadas. Por esse motivo, são consideradas métricas relacionadas à duração dos *frames*, construção das estruturas de aceleração, chamadas da API Vulkan, atividade das filas e operações de sincronização.

O *NVIDIA Nsight Graphics*, por meio do *GPU Trace*, permite observar a execução dos *frames* em uma linha temporal, relacionando eventos, filas de execução, marcadores da aplicação e métricas das unidades de *hardware*. A ferramenta também permite observar atividades submetidas às diferentes filas da GPU, bem como operações de sincronização entre elas (NVIDIA, 2026a). O *Nsight Systems* complementa essa análise ao registrar a execução das chamadas da API Vulkan, os trabalhos correspondentes na GPU e a duração dos *frames* (NVIDIA, 2026b).

2.1.7 Métricas do sistema

Além das métricas diretamente relacionadas à execução do *pipeline* gráfico, o desempenho da aplicação pode ser influenciado pelos recursos gerais do sistema. Dessa forma, são consideradas métricas relacionadas à utilização da CPU, memória *Random Access Memory* (RAM), ao consumo energético e à temperatura da GPU. A capacidade de processamento da CPU e o desempenho do subsistema de memória também podem influenciar o tempo de execução das aplicações, uma vez que o processamento depende tanto da execução das instruções quanto da disponibilidade dos dados necessários (Franchitti, 2024).

O conjunto dessas métricas permite complementar a análise de desempenho das estruturas BVH. Enquanto as métricas da GPU caracterizam a utilização dos recursos computacionais do dispositivo, as métricas do *pipeline* permitem observar a duração e a organização das diferentes etapas da execução. Por sua vez, as métricas do sistema permitem identificar condições externas à execução direta do *pipeline* que podem influenciar os resultados.

2.2 CONCEITOS E TÉCNICAS DE COMPUTAÇÃO GRÁFICA

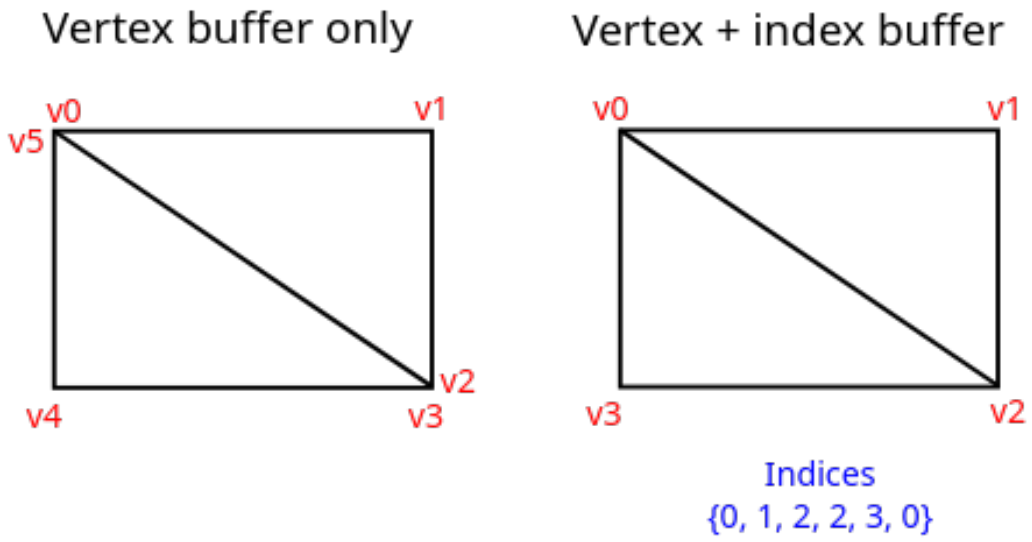
2.2.1 Primitivas

Em um sistema de modelagem hierárquica, objetos complexos são construídos a partir da junção de elementos mais básicos organizados em diferentes níveis. Para isso, são necessários elementos que não dependam de outros elementos para serem descritos. Em sistemas hierárquicos, chamamos esses elementos de nível mais baixo de primitivas (Wyvill, 1995).

No *ray tracing*, as primitivas podem ser: polígonos, objetos geométricos como a esfera ou objetos mais complexos como *patches* paramétricos (Wyvill, 1995). Na rasterização, é possível usar figuras mais complexas, porém o Vulkan usa apenas pontos, linhas e triângulos, pois são as mais otimizadas. Em especial, os triângulos são amplamente utilizados uma vez que permitem a aproximação de superfícies arbitrárias e complexas (Khronos Group, 2026).

Uma otimização importante de ser citada é o *Index Buffer* (Vulkan.org, 2026b). Em uma malha 3D, é comum que diferentes triângulos compartilhem os mesmos vértices. Sem um *Index Buffer*, cada triângulo precisaria armazenar novamente os vértices que utiliza, gerando dados duplicados e aumentando desnecessariamente o consumo de memória. O *Index Buffer* resolve esse problema armazenando índices que apontam para os vértices já existentes no *vertex buffer*, permitindo que um mesmo vértice seja reutilizado por vários triângulos.

Figura 1 – Exemplo de *Index Buffer*



Fonte: (Wang, 2022)

2.2.2 Efeitos físicos da luz

O objetivo de um algoritmo de *ray tracing* é simular diversos efeitos ópticos. Entre os principais efeitos estão a reflexão, refração, transparência, translucidez e oclusão.

A reflexão ocorre quando um raio de luz atinge uma superfície e retorna ao meio original. Em superfícies especulares, como espelhos metálicos, o ângulo de reflexão é igual ao ângulo de incidência, permitindo a formação de reflexos nítidos (Marschner; Shirley, 2021).

A refração ocorre quando a luz atravessa materiais transparentes, sofrendo alteração em sua direção devido à diferença entre os índices de refração dos meios envolvidos. Materiais dielétricos, como vidro, água e diamante, apresentam esse comportamento. A mudança de direção do raio refratado é descrita pela Lei de Snell, sendo um dos principais efeitos utilizados para representar materiais transparentes em renderização física (Marschner; Shirley, 2021).

A transparência corresponde à capacidade de um material permitir a passagem da luz através de sua superfície. Em materiais transparentes, parte da energia luminosa é transmitida para o interior do objeto, possibilitando visualizar objetos posicionados atrás dele. Esse efeito é geralmente combinado com refração para produzir representações fisicamente plausíveis de vidro, líquidos e outros materiais translúcidos (Akenine-Möller *et al.*, 2018).

A translucidez é semelhante à transparência, porém a luz transmitida sofre espalhamento interno dentro do material. Como consequência, a luz emerge difusa, impedindo a formação de imagens nítidas através do objeto. Materiais como cera, pele humana e mármore apresentam comportamento translúcido (Akenine-Möller *et al.*, 2018).

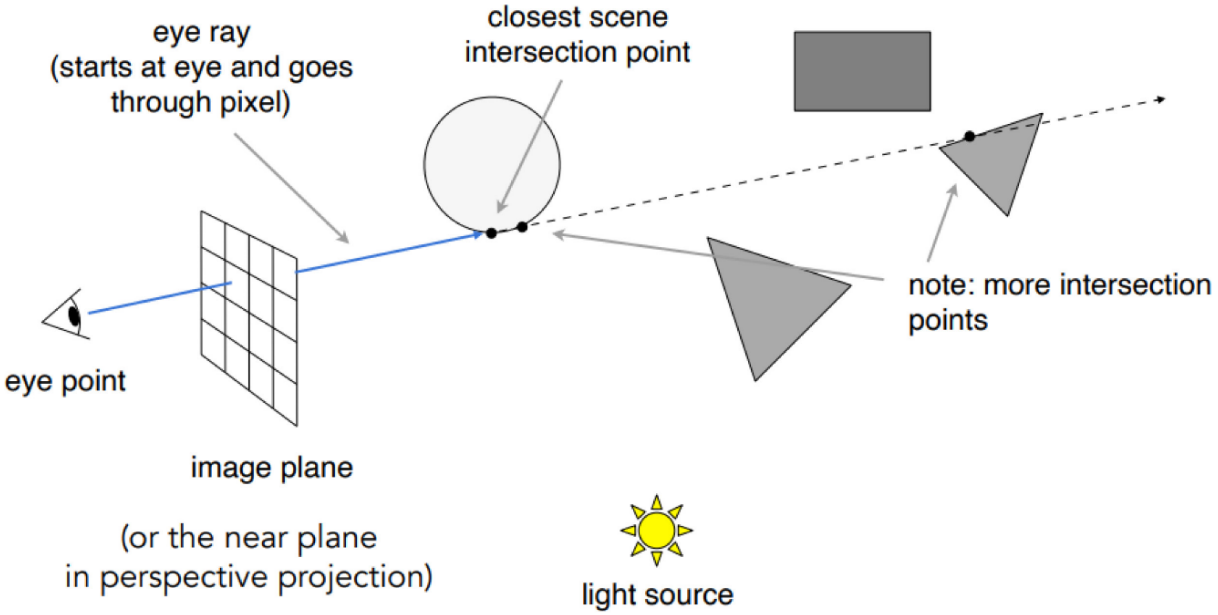
A oclusão ocorre quando um objeto bloqueia parcial ou totalmente a propagação da luz em uma cena. Esse fenômeno é responsável pela formação de sombras e regiões menos iluminadas (Marschner; Shirley, 2021).

2.2.3 *Ray tracing*

Ray tracing é uma técnica de renderização gráfica 3D baseada na simulação da propagação da luz real em um cenário virtual tridimensional. O termo pode ser utilizado tanto para se referir a um algoritmo específico, quanto para uma classe de algoritmos que lançam raios através de uma cena. Comparado com a maioria dos outros algoritmos de renderização, o *ray tracing* proporciona um efeito de luz e sombra mais realista. Um dos trabalhos pioneiros nessa área foi o algoritmo de *ray casting*, proposto por Appel em 1968, que serviu como base para o desenvolvimento das técnicas modernas de *ray tracing* (Wang, 2022).

O *ray casting* renderiza a cena emitindo um raio primário do ponto de observação ou câmera para cada pixel, encontrando o objeto mais próximo que bloqueia o caminho do raio. Raios adicionais, como raios de sombra, podem ser utilizados para verificar a visibilidade da luz a partir do ponto de interseção (Wang, 2022).

Figura 2 – Exemplo do funcionamento do *ray casting*



Fonte: (Wang, 2022)

As faces são orientadas utilizando a regra da mão direita: os vértices são listados em sentido anti-horário quando se olha o objeto do exterior. Para uma face ABC, o vetor normal n=AB*AC aponta para fora. (Freud *et al.*, 2006).

A equação do raio que parte da câmera é um dos elementos fundamentais do *ray casting*, pois permite determinar o ponto de interseção com os objetos da cena. Essa equação pode ser expressa como (Freud *et al.*, 2006):

$$\mathbf{SI} = \beta_p \, \mathbf{SP}, \quad \beta_p \in \mathbb{R}, \; 0 < \beta_p \leq 1$$

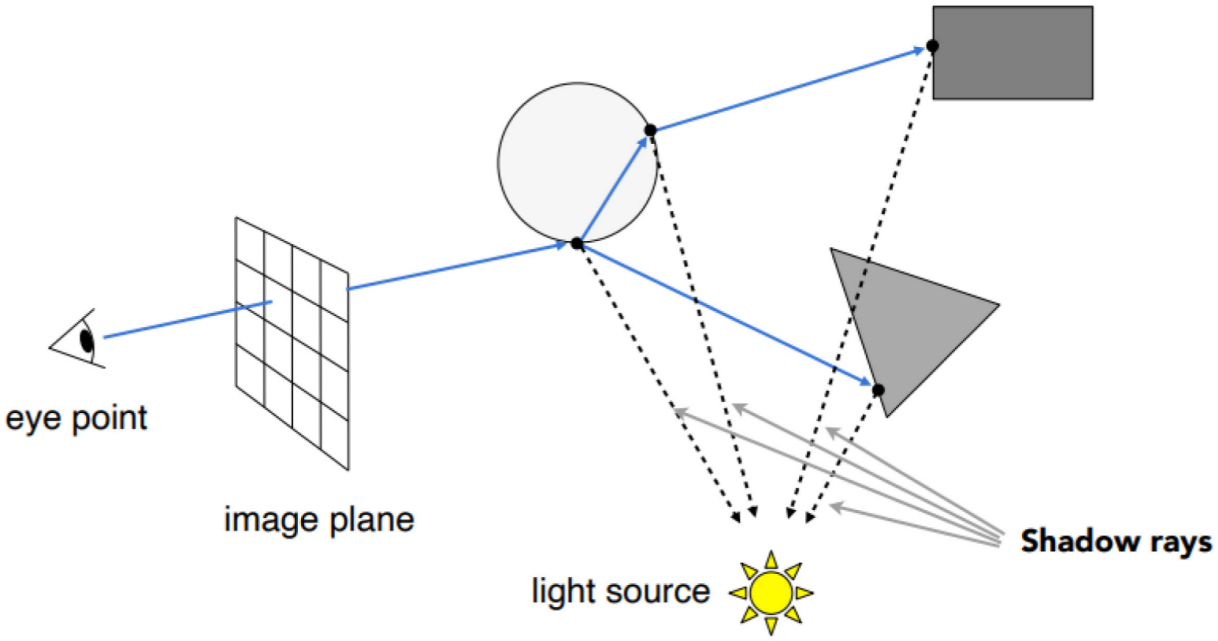
Em que:

- S representa a posição de origem do raio;
- P corresponde ao centro de um pixel no detector;
- $\mathbf{SP}$ define a direção do raio;
- I é o ponto de interseção do raio e a superfície do objeto;

- β_p é um parâmetro escalar que determina a posição do ponto de interseção ao longo do raio.
- SI é vetor da origem até o ponto de interseção.

O *Whitted ray casting*, uma evolução do *ray casting*, foi desenvolvido por Turner Whitted em 1979. Esse algoritmo usa quatro tipos de raios diferentes. O raio da câmera é similar ao *ray casting*. O segundo raio de reflexão simula superfícies refletoras e semirrefletoras. O terceiro raio de refração pode incidir no objeto a partir de um ângulo. O quarto raio de sombra projeta sombra gerada pela fonte de luz e pelo objeto. A superfície nem sempre é perfeitamente refletora, seria necessário modelar um componente difuso, mas modelos de iluminação difusa mais realistas implicam maior custo computacional (Wang, 2022).

Figura 3 – Exemplo do funcionamento do *Whitted ray casting*



Fonte: (Wang, 2022)

Apesar de os dois algoritmos citados gerarem sombras, as sombras produzidas são rígidas, possuem bordas bem definidas e utilizam luzes pontuais. Entretanto, em diversas aplicações, é necessário utilizar fontes de luz com área. As sombras suaves (*soft shadows*) geradas por luzes com área produzem regiões de umbra e penumbra, fornecendo maior percepção espacial e profundidade entre os objetos da cena (Schwärzler *et al.*, 2012).

Uma forma de produzir sombras suaves em *ray tracing* consiste em realizar múltiplas amostragens da fonte de luz. Nesse método, vários raios de sombra são lançados em direção a diferentes pontos da luz com área (*area light*). Para cada raio, verifica-se se existe algum ob-

jeto bloqueando o caminho entre o ponto da superfície e a fonte luminosa. A intensidade final da sombra é calculada pela proporção entre raios bloqueados e não bloqueados (Schwärzler *et al.*, 2012).

O *Whitted ray casting* consegue simular reflexões em superfícies refletoras e semirrefletoras. Entretanto, quando poucas amostras são utilizadas, a imagem pode apresentar ruídos e perda de qualidade visual. Para reduzir esse problema, pode-se utilizar a técnica de *Importance Sampling*, que concentra as amostras nas direções que mais contribuem para a iluminação da cena (Dupuy; Benyoub, 2023).

Uma das distribuições mais utilizadas para esse processo é a *Trowbridge-Reitz GGX Distribution* (GGX), um modelo de *microfacets* empregado para representar reflexos especulares de maneira mais realista. Nesse modelo, a superfície é composta por pequenos *microfacets* que refletem a luz em diferentes direções. O *Importance Sampling* baseado em GGX direciona mais raios para as direções de maior contribuição especular, reduzindo ruídos e melhorando a qualidade dos reflexos com menor custo computacional (Dupuy; Benyoub, 2023).

2.2.4 Rasterização

A rasterização é uma técnica básica de renderização. Uma tarefa de renderização complexa pode ser dividida em diferentes subtarefas associadas a diferentes objetos. Em seguida, cada objeto é dividido em polígonos projetados na tela. Finalmente, cada pixel coberto pelos polígonos é renderizado (Wang, 2022). A rasterização era inicialmente realizada por *software* em CPUs. Entretanto, o avanço do hardware gráfico dedicado levou à ampla adoção das GPUs para essa tarefa, devido ao seu desempenho significativamente superior, tornando-se a abordagem predominante em aplicações de renderização em tempo real (Laine; Karras, 2011).

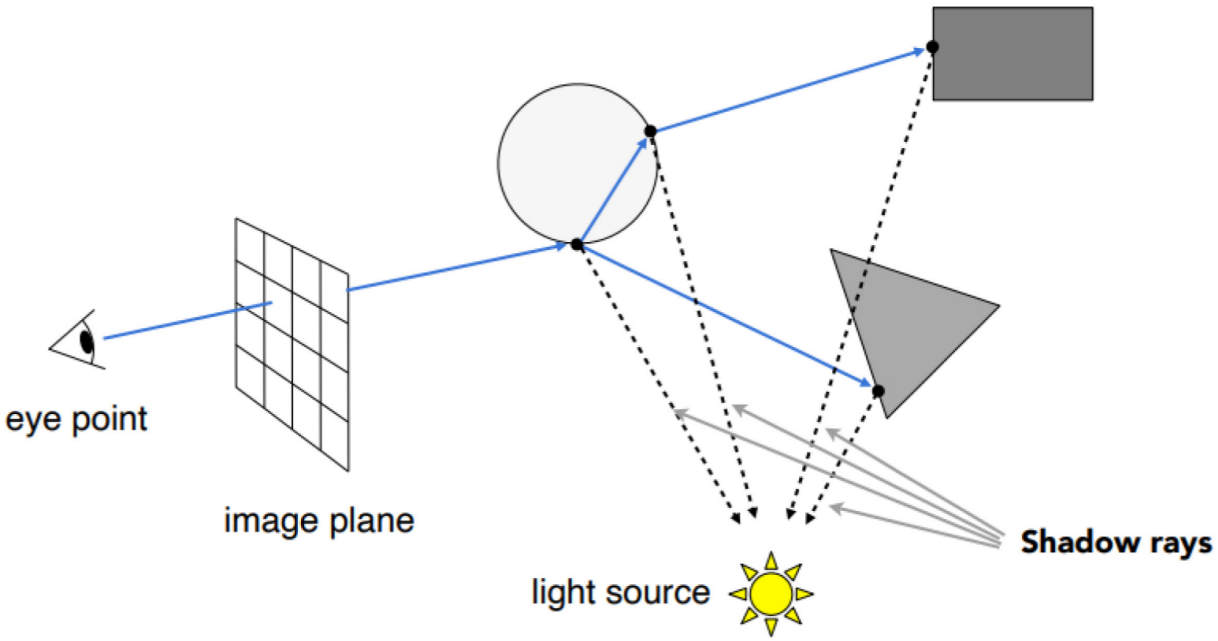
O *pipeline* de rasterização opera em triângulos projetados e recortados em 2D. O coração do algoritmo de rasterização, *coverage computation* (cálculo de cobertura), determina para todos os pixels (e possivelmente várias subamostras em cada pixel), se eles estão cobertos por um determinado triângulo (Davidovič *et al.*, 2012).

O teste hierárquico de regiões de pixels para cobertura de triângulos, chamado *binning*, é um fator importante para o desempenho de renderização, pois permite identificar rapidamente áreas da imagem potencialmente cobertas. As regiões podem ser avaliadas em paralelo e hierarquicamente para rapidamente identificar amostras relevantes na tela, reduzindo o número de testes individuais (Davidovič *et al.*, 2012).

O *coverage test* (teste de cobertura) normalmente usa funções de aresta 2D lineares no espaço da imagem, uma para cada aresta do triângulo, cujo sinal determina qual lado da aresta está dentro do triângulo. Um pixel está dentro do triângulo se todas as três funções de aresta forem positivas em sua localização e, portanto, o cálculo da cobertura se torna uma simples avaliação das três funções de aresta, o que é adequado para arquiteturas paralelas (Davidovič *et al.*, 2012).

O *clipping* (recorte) à esquerda e à direita pode ser tratado como arestas extras do polígono, representadas por El e Er. Ao considerar esses limites como arestas, o algoritmo de rasterização evita processar partes fora da área visível, parando quando atinge esses limites. O limite superior define onde a renderização começa, e o inferior indica onde ela termina (Pineda, 1988).

Figura 4 – Exemplo de triângulo sendo recortado



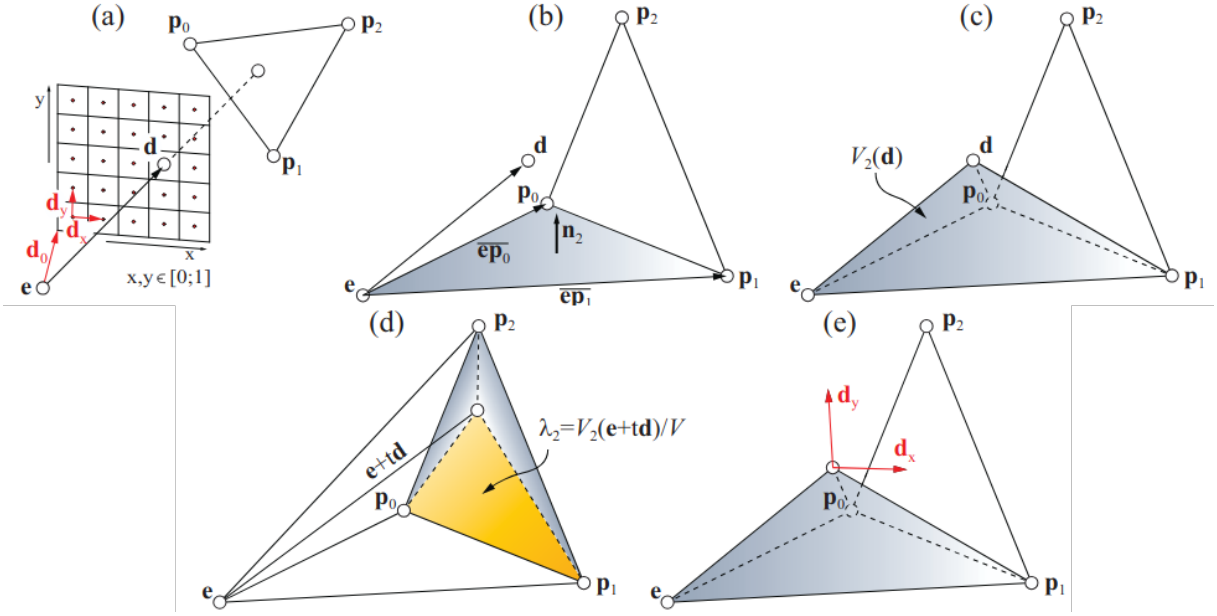
Fonte: (Pineda, 1988)

Na interpolação de atributos sobre um triângulo em 3D, qualquer função linear (u), como cores ou coordenadas de textura, pode ser descrita pela equação (u = aX + bY + cZ + d), em que ((X, Y, Z)T) representa um ponto no espaço tridimensional. Os parâmetros (a), (b) e (c) são determinados a partir dos valores associados aos vértices do triângulo. Assumindo o espaço canônico do olho, onde o centro de projeção está na origem, a direção de visualização coincide com o eixo (z) e o campo de visão é de 90 graus, a divisão dessa equação por (Z) resulta no conhecido esquema de interpolação correta em perspectiva 2D (Davidovič *et al.*, 2012).

Testar se uma amostra de pixel é coberta por um triângulo 2D é equivalente a testar

se um raio. Um raio partindo do ponto de vista e passando por esse pixel, intersecta o triângulo. Como exemplo: (e) é a localização do ponto de vista e o raio passa por (d = d0 + xdx + ydy), para coordenadas de pixel (x, y), enquanto (p0, p1, p2) são os vértices do triângulo. Podemos formular funções de aresta lineares 3D usando o método de volume com sinal, conhecido como teste de interseção raio-triângulo de Plücker. Na prática, usamos (n2 = (p1 - e) x (p0 - p1)), que é numericamente mais estável para triângulos pequenos. Definindo a função de aresta 3D correspondente como (V2(d) = n2 · d), que é equivalente a calcular o sinal escalonado (Davidovič *et al.*, 2012).

Figura 5 – Funções de aresta 3D para um triângulo



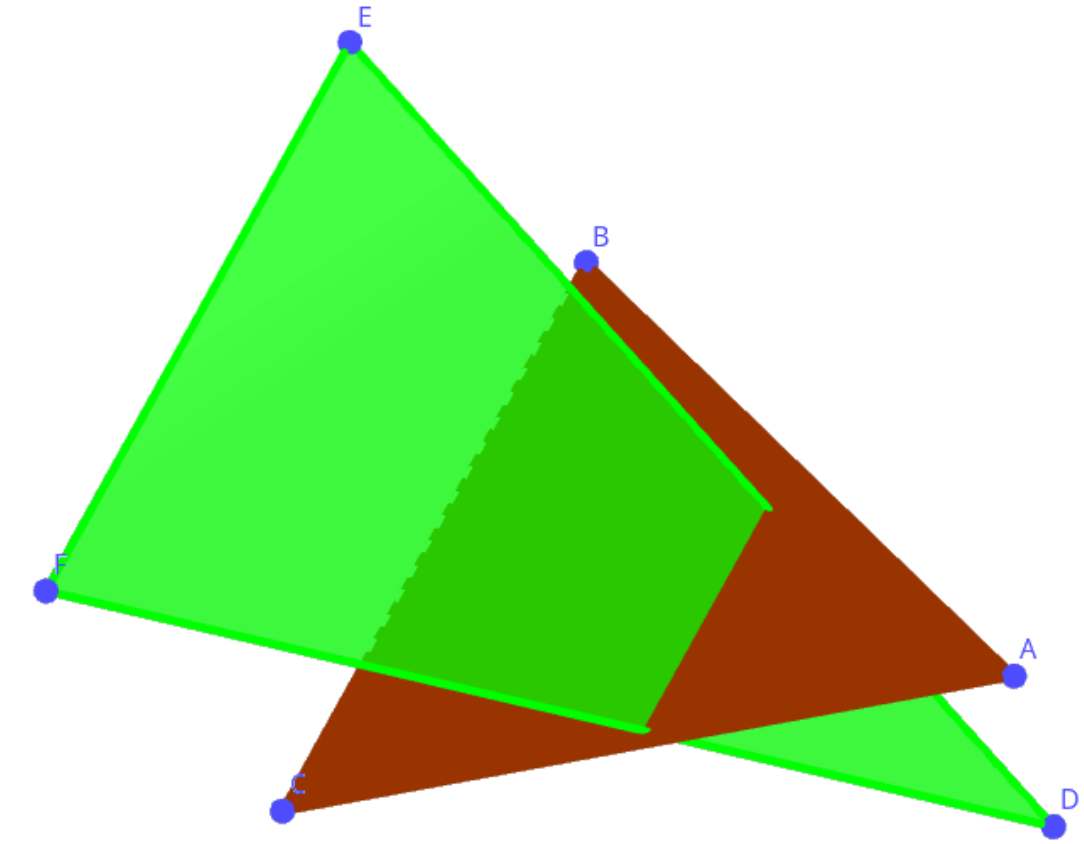
Fonte: (Davidovič *et al.*, 2012)

Grande parte do trabalho no *pipeline* gráfico, principalmente a execução de *shaders* de *vertex* (vértice) e *fragment* (fragmento), bem como outros estágios programáveis, é realizada pelos núcleos de *shader* programáveis (Laine; Karras, 2011).

O *vertex shader* é responsável por decodificar os dados geométricos. Já o *fragment shader* reconstrói o conteúdo da textura ao nível de pixel de saída, por meio da reconstrução do conteúdo da textura via combinação linear (Schuster *et al.*, 2022).

Durante a rasterização, quando uma primitiva intersecta ou passa à frente de outra, torna-se ambíguo qual delas deve ser visível em cada pixel. O Z-buffer é um algoritmo que resolve a visibilidade ao nível de pixel. O Z-buffer processa os polígonos, atualiza e armazena os valores de profundidade de cada pixel durante a conversão da varredura. Porém, o Z-buffer não lida bem com superfícies parcialmente transparentes (Greene; Kass; Miller, 1993).

Figura 6 – Caso em que primitivas competem pelo mesmo pixel



Fonte: Elaborado pelo autor, usando GeoGebra Classic

A rasterização se tornou a abordagem dominante para aplicações interativas devido aos seus baixos requisitos computacionais iniciais (não requer cálculo de ponto flutuante no espaço de imagem 2D), à sua capacidade de ser implementada em *hardware* e ao desempenho cada vez maior de placas gráficas dedicadas. No entanto, o tratamento de efeitos globais não é nativo e depende de aproximações, como reflexos e sombras (Davidovič *et al.*, 2012).

2.2.5 Abordagem híbrida

Os dois principais algoritmos usados em computação gráfica para gerar imagens 2D a partir de cenas 3D são a rasterização e o *ray tracing*. A rasterização projeta cada triângulo no plano da imagem e enumera todos os pixels cobertos em 2D, enquanto o *ray tracing* opera em 3D, gerando raios através de cada pixel e encontrando a interseção mais próxima com um triângulo (Davidovič *et al.*, 2012).

Com a rasterização 3D, as principais diferenças entre as abordagens passam a se

concentrar em dois pontos. A travessia da cena, como identificar os objetos relevantes para a cena, usando estruturas e organizações das primitivas. A enumeração de amostras potencialmente cobertas no plano da imagem (*binning*), que determina quais pixels ou amostras podem ser influenciados por cada primitiva (Davidovič *et al.*, 2012).

Rasterização e *ray tracing* são abordagens distintas. Porém, não são técnicas distintas, são pontos extremos de um mesmo espectro contínuo. Sendo assim, diferentes comportamentos podem ser obtidos por meio da ativação ou desativação de determinados componentes, como geração de amostras, *binning* e testes de oclusão. Permitindo aplicar tanto estratégias de rasterização quanto de *ray tracing* (Davidovič *et al.*, 2012).

O *pipeline* de renderização híbrida (*hybrid rendering pipeline*) combina técnicas de rasterização, computação paralela e *ray tracing* para produzir imagens em tempo real com qualidade visual próxima à obtida por métodos de *ray tracing offline*. A interação de múltiplas etapas gráficas, a utilização das capacidades únicas de cada etapa e a modularização do processo de renderização permitem alcançar vários aspectos visuais de forma otimizada (Barré-Brisebois *et al.*, 2019).

O PICA PICA é um experimento de traçado de raios em tempo real com agentes de autoaprendizagem. O sistema adota um *pipeline* de renderização híbrida que combina rasterização, *shaders* de computação e técnicas de *ray tracing* para compor a imagem final. A arquitetura do *pipeline* é organizada em múltiplas etapas, cada uma responsável por um aspecto específico da renderização e utiliza uma abordagem modular para maximizar eficiência e qualidade visual (Barré-Brisebois *et al.*, 2019).

Para gerar imagens de alta qualidade, seria necessário gerar milhares de amostras por pixel. Mesmo com uma BVH otimizada, algoritmos eficientes de busca e GPUs rápidas, isso não é possível em tempo real atualmente, exceto para as cenas mais simples. As imagens que podem ser geradas com as restrições de desempenho atuais possuem muito ruído. Porém, elas podem ser tratadas com algoritmos de redução de ruído (*denoising*), podendo gerar imagens com apenas um raio por pixel com qualidade próxima a um *ray tracing offline* (Akenine-Möller *et al.*, 2018).

O *pipeline* do PICA PICA se inicia com o *object-space rendering* (renderização no espaço de objetos), trata as estruturas geométricas e coordenadas de textura, faz o cálculo de transparência e translucidez para raios. Depois, com *global illumination*, calcula oclusão (esse processo é limitado a um número máximo de testes por quadro) (Barré-Brisebois *et al.*, 2019).

Figura 7 – Imagem path-traced (esquerda). Imagem suavizada (centro). A qualidade de referência renderizada com 2048 amostras por pixel (direita)



Fonte: (Akenine-Möller *et al.*, 2018) cortesia da NVIDIA Corporation

G-buffer, uma abreviação de *geometric buffers* (*buffers* geométricos), refere-se a *buffers* usados para armazenar propriedades de superfície. Cada G-buffer é um alvo de renderização separado. Esse alvo pode ser mapa de profundidade, *buffer* normal, *buffer* de rugosidade, oclusão, cor da textura (albedo), intensidade da luz, intensidade especular e imagem quase final (Akenine-Möller *et al.*, 2018).

A partir das informações armazenadas nos G-buffers, o *pipeline* calcula sombras geradas pelas fontes de luz e remove possíveis ruídos. O *pipeline* usa os dados geométricos e de material para determinar os pontos de origem dos raios de reflexão. Em seguida, é computada a iluminação direta proveniente das fontes de luz e sua interação com os materiais dos objetos. Por fim, a imagem resultante passa por etapas de pós-processamento, nas quais são aplicados efeitos como *anti-aliasing* (suavização de serrilhado) e desfoque de movimento (*motion blur*) (Barré-Brisebois *et al.*, 2019).

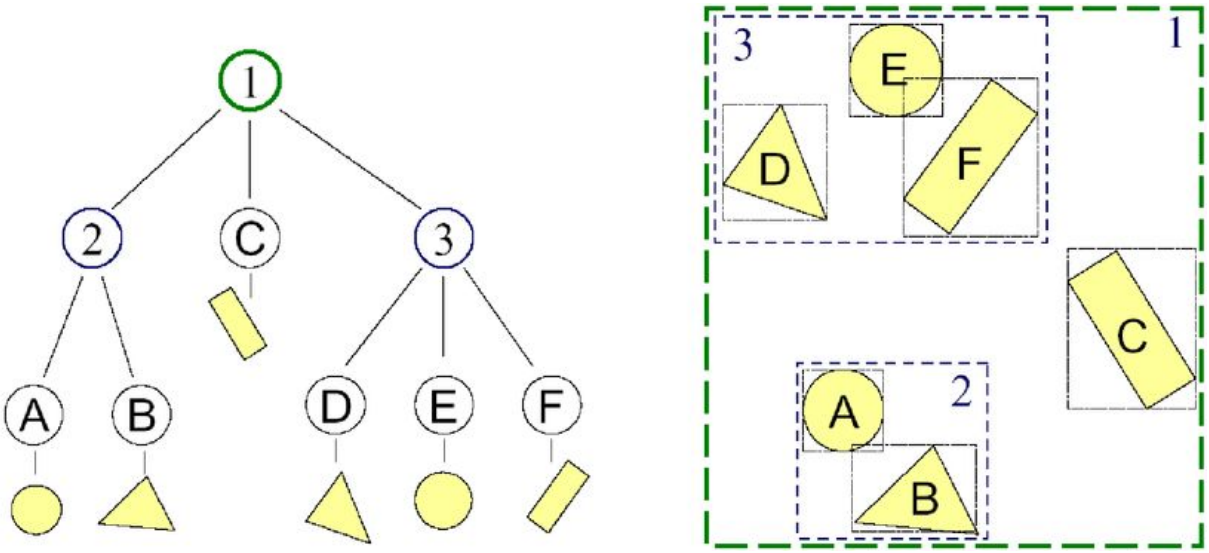
2.3 ESTRUTURAS DE ACELERAÇÃO

O *ray tracing* vem se tornando cada vez mais importante para aplicações em tempo real. No entanto, exige alto poder computacional e depende de estruturas de dados pré-processadas para alcançar um desempenho em tempo real. Dentre os métodos para melhorar a eficiência do *ray tracing*, as estruturas de dados hierárquicas são extremamente importantes e populares (Stich; Friedrich; Dietrich, 2009).

Representações hierárquicas de objetos tridimensionais são eficientes tanto em tempo

quanto em uso de memória. Essas representações tipicamente consistem em árvores cujos ramos representam volumes delimitadores (Rubin; Whitted, 1980). A ideia por trás de uma BVH é subdividir as primitivas de uma cena em conjuntos possivelmente sobrepostos. Para cada um desses conjuntos, um *Bounding Volume* (BV) (volume delimitador) é calculado e estes são organizados em uma estrutura de árvore. Os limites de cada nó nesta árvore são escolhidos de forma que ele delimite exatamente todos os nós na subárvore correspondente e cada nó folha delimite exatamente as primitivas contidas (Pantaleoni; Luebke, 2010).

Figura 8 – Exemplo de BVH

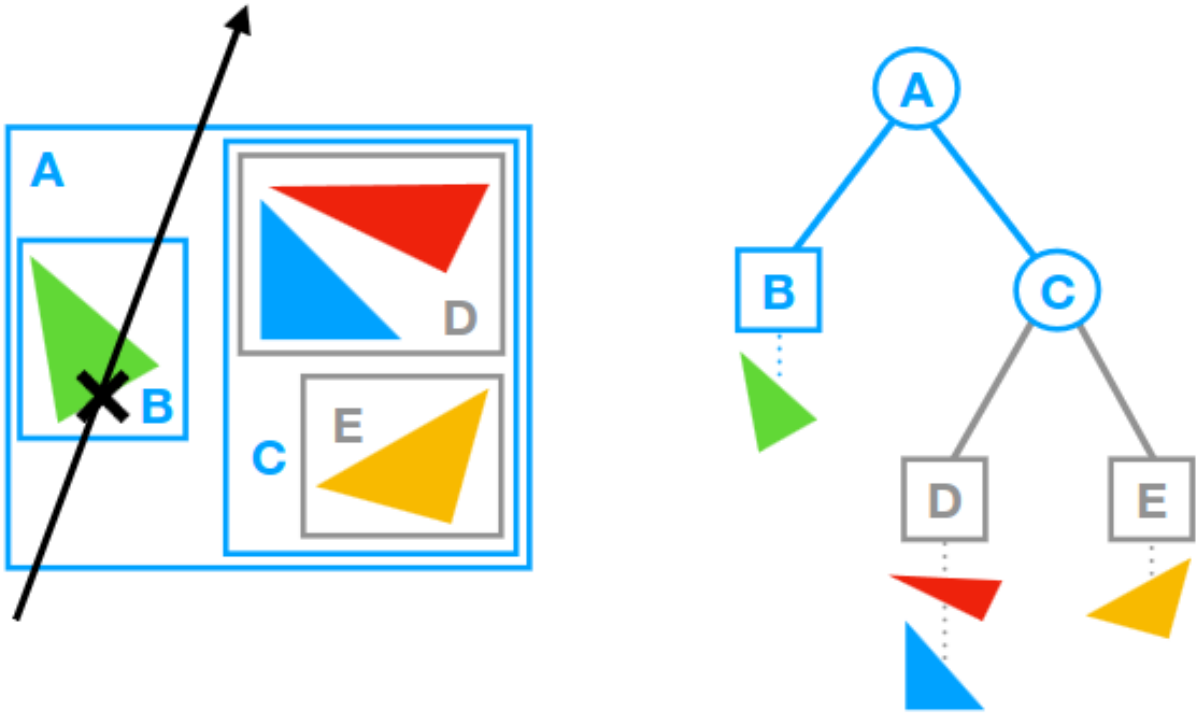


Fonte: (Gonçalves, 2009)

Para encontrar a interseção mais próxima, a travessia de um raio inicia-se no nó raiz e é realizada de forma *top-down* (de cima para baixo). Geralmente, utiliza-se uma pilha para armazenar nós internos que podem conter a interseção mais próxima. A cada iteração, o nó do topo é removido e o raio é testado contra seu BV. Caso haja interseção, seus nós filhos são adicionados à pilha, no caso de nós internos, ou as primitivas são testadas diretamente, no caso de nós folha. Testando as primitivas, a distância até a interseção mais próxima encontrada até o momento é armazenada. Essa distância permite ignorar os nós que estão mais distantes do que a interseção encontrada anteriormente. A travessia continua até que a pilha esteja vazia. Para um teste de oclusão, quando apenas precisa testar se o ponto está visível ou ocluído, é possível usar um *early exit* (saída antecipada) após encontrar a primeira interseção (Meister *et al.*, 2021).

Originalmente, BVH utilizava pelo menos seis *bounding slabs* (planos limitantes) para formar um casco fechado ao redor de um objeto. Cada slab corresponde à região compreendida entre dois planos paralelos, que restringem o espaço em uma determinada direção; a

Figura 9 – Exemplo de travessia de BVH para encontrar a interseção, descartamos toda a subárvore do nó C, uma vez que o raio não intersecta a caixa delimitadora associada a ela



Fonte: (Meister *et al.*, 2021)

interseção desses *slabs* define o BV (Pantaleoni; Luebke, 2010).

Se esses *slabs* são perpendiculares aos eixos do sistema de coordenadas do mundo, o volume delimitador resultante é denominado *Axis-aligned Bounding Box* (AABB). Em BVHs convencionais, os limites de um AABB são representados por seis valores de ponto flutuante, correspondentes às coordenadas mínimas e máximas ao longo de cada eixo cartesiano, chamados de *bounding box* (Pantaleoni; Luebke, 2010).

Comparadas a outras abordagens, as BVHs têm geralmente os menores requisitos de memória e um *memory footprint* (uso de memória) pré-computável (Pantaleoni; Luebke, 2010). Técnicas híbridas têm sido investigadas minuciosamente nos últimos anos, buscando combinar os benefícios das árvores BVHs. Alguns modelos serão apresentados.

2.3.1 Linear BVH (LBVH)

O primeiro método paralelo de construção de LBVH foi introduzido por Lauterbach. O método inicia atribuindo um código de Morton ao centroide de cada primitiva geométrica. Em seguida, as primitivas são ordenadas de acordo com esses códigos e a hierarquia é construída dividindo os nós nos pontos em que ocorre a maior diferença de bits entre dois

códigos de Morton consecutivos. O código de Morton é calculado mapeando as coordenadas espaciais para um cubo unitário tridimensional e intercalando os bits binários de cada coordenada (Apetrei, 2014).

O código de Morton é gerado a partir das coordenadas espaciais das primitivas. Esse processo converte posições tridimensionais em valores lineares que preservam a proximidade espacial entre os objetos. O algoritmo envolve múltiplas etapas de processamento dependentes entre si, e requer grandes *buffers* temporários e é uma ordem de magnitude mais lento do que, por exemplo, ordenar os códigos. Posteriormente, Pantaleoni e Luebke aprimoraram o método que gera a hierarquia sequencialmente a partir da raiz, processando cada nível em paralelo (Apetrei, 2014).

2.3.2 BVH comprimida

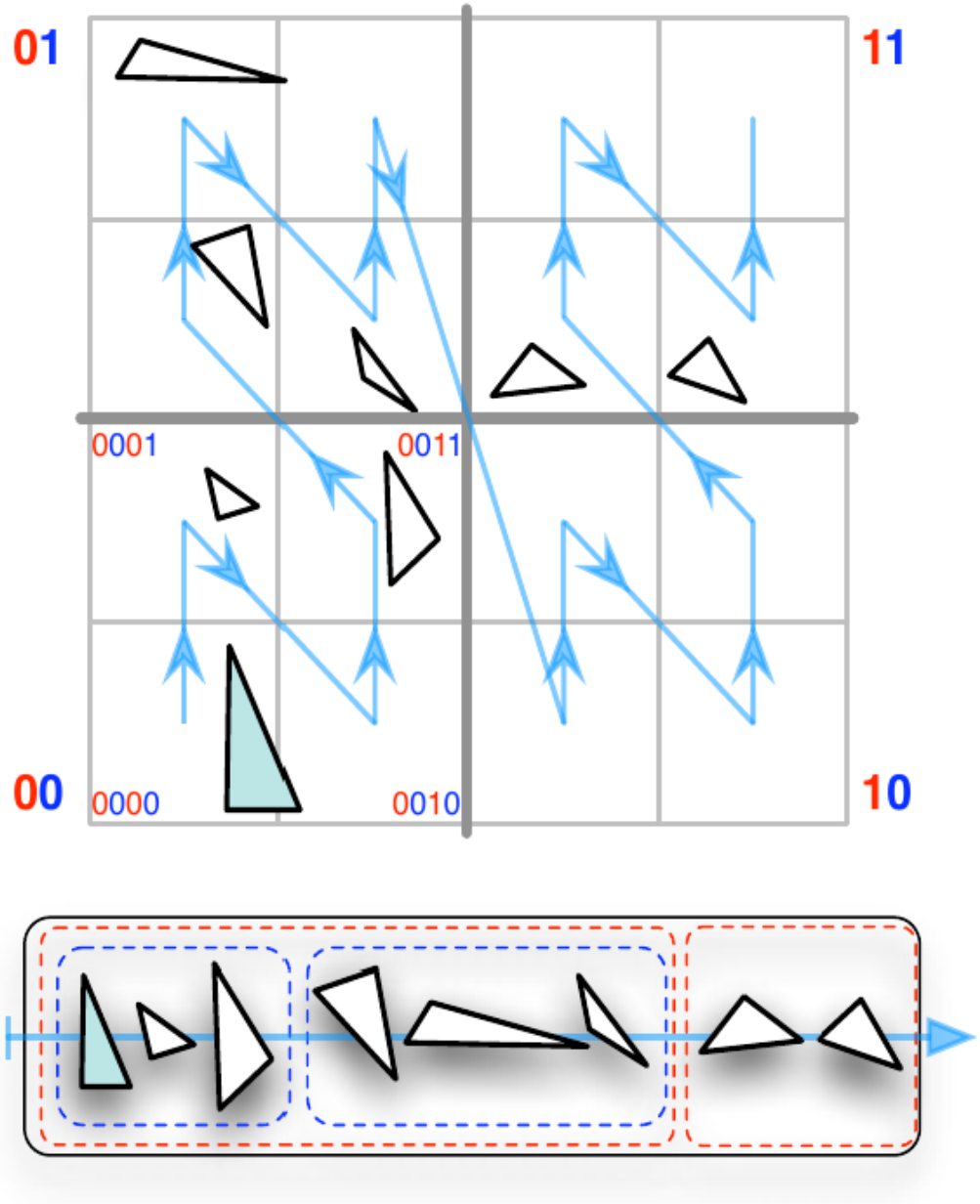
Uma maneira de reduzir a sobrecarga de memória é comprimir a estrutura de dados de aceleração. A compressão funciona particularmente bem para BVHs, reduzindo *memory footprint* e o uso de banda. Porém, essa compressão adiciona um custo de descompressão durante a travessia, o que deve resultar em uma diminuição de desempenho. Em cenas grandes, quando a memória dedicada da GPU não é suficiente, essa perda de desempenho é compensada pela possibilidade de manter toda a BVH na memória principal e por um melhor uso do *cache* (Benthin *et al.*, 2018).

Existem várias versões de compressão: *Compressed Bounding Volume Hierarchies* (CBVH) comprime a árvore por completo; *Compressed-Leaf Bounding Volume Hierarchies* (CLBVH) comprime apenas as folhas da árvore, tendo uma travessia mais rápida; *Minimal Bounding Volume Hierarchy* (MVH) usa a *bounding box* do pai para construir os filhos, conseguindo trabalhar com apenas 2 bytes, guardando nos filhos e folhas em uma pilha de dados (Pantaleoni; Luebke, 2010) (Benthin *et al.*, 2018).

2.3.3 BVH larga

Além do consumo de memória, o tempo de travessia também se torna um problema em cenas com muitas primitivas. O *Wide Bounding Volume Hierarchy* (WBVH), também chamado de *Multi Bounding Volume Hierarchy* (MBVH), *Quad Bounding Volume Hierarchy* (QBVH), *4-Wide Bounding Volume Hierarchy* (BVH4), BVH8 ou *16-Wide Bounding Volume Hierarchy* (BVH16). Diferente das BVH comuns que são binárias, o WBVH utiliza nós com

Figura 10 – Ordenação da curva de Morton e os códigos de Morton correspondentes



Fonte: (Pantaleoni; Luebke, 2010)

múltiplos filhos, reduzindo o tempo de travessia. Embora a sobrecarga da travessia de raios seja reduzida com a redução da profundidade da árvore, a taxa de transferência de raios acaba sendo limitada por outras restrições, como armazenamento e largura de banda da memória (Vaidyanathan; Woop; Benthin, 2022) (Tsakok, 2009).

Por esse motivo, as estruturas WBVH comprimidas são uma escolha interessante para a travessia, com nós internos comprimidos resultando em taxas de acerto de *cache* aprimoradas e maior taxa de transferência de raios (Vaidyanathan; Woop; Benthin, 2022) (Tsakok, 2009).

2.3.4 Binned Surface Area Heuristic BVH (BSAH)

Nenhuma dessas técnicas anteriores melhora a qualidade da árvore. *Surface Area Heuristic* (SAH) BVH é um algoritmo aplicado na separação dos objetos geométricos, que considera a densidade da geometria no espaço 3D, elimina eficientemente regiões vazias e reduz interações. Infelizmente, construir uma árvore de alta qualidade é custoso. Esse custo computacional torna a técnica inviável para cenas dinâmicas e animadas, onde são reconstruídas a cada quadro.

O BSAH é uma aproximação do SAH que reduz o custo de construção. Em vez de avaliar todos os planos de divisão entre primitivas, o espaço do nó é dividido em um número fixo de regiões chamadas *bins*. As primitivas são distribuídas entre essas regiões e o algoritmo avalia planos de corte entre os *bins*, calculando o custo SAH de cada divisão com base na quantidade de primitivas e na área das AABBs resultantes. O plano com menor custo é selecionado e o processo é aplicado recursivamente até completar a árvore. Existem híbridos como *Hierarchical Linear Bounding Volume Hierarchy* (HLBVH) que têm a construção mais rápida, mas perdem mais qualidade (Garanzha *et al.*, 2011) ou *Split Bounding Volume Hierarchy* (SBVH) com menor custo de travessia, mas maior uso de memória.

2.3.5 Hierarchical LBVH (HLBVH)

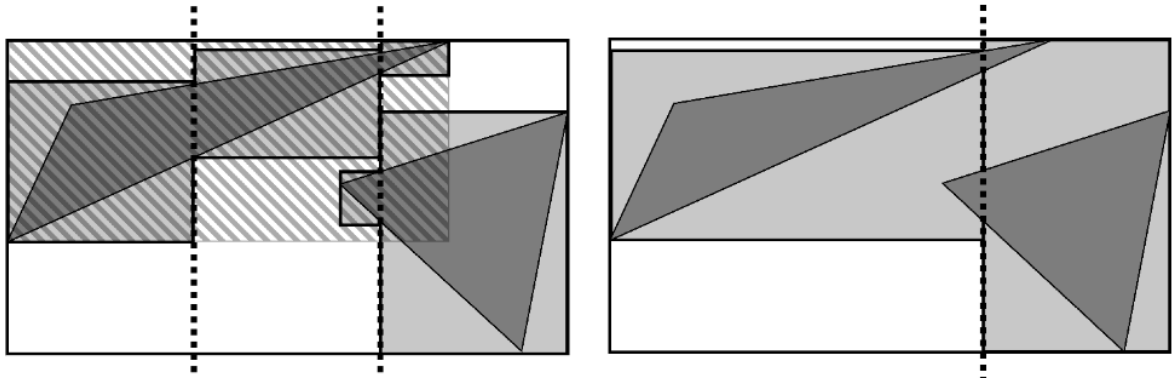
A HLBVH foi proposta por Pantaleoni como uma extensão da LBVH tradicional, buscando melhorar a eficiência da construção paralela da hierarquia, mas que reduz substancialmente tanto a quantidade de cálculos quanto o tráfego de memória necessários para construir a BVH. O algoritmo introduz três modificações principais. Primeiramente, uma reformulação na etapa de ordenação baseada na Morton *curve-based*, exigida pelo procedimento original, na qual foi empregada uma decomposição hierárquica em grade para explorar a coerência espacial presente na malha de entrada. Em segundo lugar, um novo procedimento de emissão de hierarquia de nós que aprimora a técnica de Lauterbach. Terceiro, um novo algoritmo híbrido que usa a SAH para construir os níveis BVH superiores e usa particionamento rápido baseado na curva de Morton dentro das subárvores do conjunto (Pantaleoni; Luebke, 2010).

2.3.6 BVH com Divisões Espaciais (SBVH)

Há ainda uma outra forma de gerar uma árvore com qualidade SBVH que usa BSAH para dividir o espaço. O grande diferencial ocorre no plano de divisão, caso a primitiva esteja

atravessando o plano escolhido, a primitiva é dividida em duas AABB, uma para cada lado do plano. As AABB geradas dessa divisão são menores, consequentemente isso gera um custo menor, porém gerar mais AABBs aumenta o consumo de memória (Stibora, 2016).

Figura 11 – Ilustração da técnica de *binning*. À esquerda: *binning* de um nó BVH com dois planos divididos em um eixo. Os valores AABBs são recalculados para cada parte das primitivas cortadas. À direita: as caixas delimitadoras dos nós filhos esquerdo e direito para o segundo plano dividido



Fonte: (Stibora, 2016)

2.3.7 Estruturas de Aceleração no Vulkan

No *Vulkan*, as estruturas de aceleração são divididas em dois níveis: BLAS e TLAS. A BLAS armazena a geometria utilizada para representar uma *mesh*, sendo formada principalmente pelas primitivas geométricas, como triângulos. Dessa forma, diferentes *meshes* ou partes de uma *mesh* podem possuir suas próprias estruturas BLAS (Vulkan Documentation Project, 2025a).

A TLAS, por outro lado, não armazena diretamente os triângulos da geometria. Ela armazena instâncias que referenciam estruturas BLAS, juntamente com informações como a transformação de cada instância. Dessa forma, uma mesma BLAS pode ser utilizada por diferentes instâncias em posições, rotações ou escalas diferentes, permitindo representar a cena completa a partir das geometrias armazenadas nas BLASs (Vulkan Documentation Project, 2025a).

Durante a criação dessas estruturas, o *Vulkan* fornece uma interface para descrever as geometrias, consultar os requisitos de memória, criar as estruturas e executar os comandos de construção. A implementação utilizada durante a travessia dos raios é dependente da GPU e não é exposta diretamente ao usuário. Dessa forma, a aplicação fornece ao *Vulkan* as informações necessárias para a construção das estruturas, enquanto os detalhes de sua implementação

permanecem ocultos (Vulkan Documentation Project, 2025a).

Essa divisão também permite atualizar a cena sem necessariamente reconstruir toda a geometria. Quando uma instância é movimentada, por exemplo, sua transformação pode ser atualizada na TLAS, mantendo a geometria armazenada na BLAS. O *Vulkan* também permite que uma TLAS seja atualizada quando ela foi criada com suporte a atualizações, possibilitando modificar as transformações das instâncias durante a execução (Vulkan Documentation Project, 2025b).

2.4 TRABALHOS CORRELATOS

Chung et al. (Chung *et al.*, 2018) realizaram uma comparação entre estruturas de aceleração do tipo BVH e *KD-tree* para aplicações de *ray tracing* executadas em dispositivos móveis reais. O protocolo experimental considerou uma mesma cena e resolução de renderização para as diferentes estruturas, permitindo comparar o tempo de execução obtido por cada abordagem. Os resultados indicaram um melhor desempenho da *KD-tree* em relação à BVH. Os autores também destacam que o desempenho das estruturas pode variar de acordo com o dispositivo e com as características dos modelos utilizados, apontando a necessidade de avaliações em diferentes cenários.

Posteriormente, Meister e Bittner (Meister; Bittner, 2022) apresentaram uma análise comparativa de diferentes métodos de construção de BVHs para *ray tracing* em GPUs. O estudo estabeleceu um protocolo de comparação envolvendo diferentes algoritmos de construção, cenas e configurações de execução, avaliando aspectos como o tempo de construção das estruturas e o desempenho durante a travessia dos raios. Além das BVHs binárias tradicionais, os autores investigaram técnicas como *spatial splits*, *ray reordering* e WBVHs. Os resultados mostraram que diferentes estratégias apresentam comportamentos distintos conforme a cena e a configuração utilizada, destacando a importância de uma avaliação sistemática para a comparação das estruturas.

Embora ambos os trabalhos investiguem o desempenho de estruturas de aceleração para *ray tracing*, seus experimentos apresentam diferentes abordagens de comparação, considerando características específicas das estruturas e dos cenários avaliados. O presente trabalho diferencia-se por analisar diferentes estruturas BVH em cenários de renderização em tempo real, utilizando uma mesma base de comparação para avaliar as abordagens LBVH, BSAH e BVH8. A partir das características observadas nos trabalhos correlatos, é definido um protocolo experi-

mental próprio, buscando estabelecer condições de comparação consistentes entre as diferentes estruturas. A comparação considera métricas relacionadas ao desempenho da renderização em tempo real, buscando identificar as características e limitações de cada abordagem.

3 DESENVOLVIMENTO

Neste capítulo é apresentada a implementação da aplicação desenvolvida para a realização dos experimentos. São descritas as principais decisões de desenvolvimento, a organização da aplicação, as estruturas BVH utilizadas e os métodos empregados para a coleta dos dados. Antes da apresentação da aplicação, são apresentadas algumas decisões tomadas durante o desenvolvimento, principalmente em relação à utilização das estruturas de aceleração e à forma de execução da renderização.

3.1 DECISÕES DE DESENVOLVIMENTO

Antes de apresentar a aplicação e os resultados obtidos, é necessário esclarecer algumas decisões tomadas durante o desenvolvimento. Essas decisões foram importantes para definir como as diferentes estruturas BVH seriam utilizadas e comparadas, além de estabelecer uma forma de execução que permitisse controlar as principais etapas do processo de renderização.

3.1.1 Estrutura da Aplicação

A aplicação foi construída a partir de uma aplicação já existente, chamada atualmente de *Apotheosis*. De forma resumida, o *Apotheosis* é um motor gráfico desenvolvido para um jogo *open source*, de autoria do autor deste trabalho. Esse motor gráfico foi escolhido por utilizar *Vulkan*, possibilitando maior controle sobre as etapas de renderização e sobre os recursos disponibilizados pelo *hardware*, além de possuir código aberto, ser facilmente compilável por meio de uma *Open Container Initiative Image Format* e já ser conhecido pelo autor deste trabalho (SrPatsu21, 2026).

O *Apotheosis* já possuía grande parte dos componentes utilizados neste trabalho. O código era baseado principalmente no tutorial de *Vulkan* (Alexander Overvoorde, 2023) e na documentação do *GLFW* (GLFW, 2026). A partir das alterações realizadas para suportar as diferentes estruturas BVHs e do aprimoramento das interfaces da aplicação, o código passou a possuir algumas diferenças em relação à versão original.

Uma das principais mudanças foi a redução da compatibilidade com dispositivos que não possuem suporte a alguns recursos utilizados pela aplicação, como *Indexing* e *Buffer Device Address*. Além disso, para a coleta de algumas métricas, também é necessário o suporte

a *Host Query Reset*. Dessa forma, a aplicação passou a exigir um conjunto maior de recursos do dispositivo utilizado nos experimentos.

Apesar dessas alterações, a estrutura geral do *Apotheosis* foi mantida. Entre os elementos reaproveitados estão principalmente a organização do projeto, as bibliotecas utilizadas e os arquivos relacionados à compilação. Foram realizadas pequenas adições e alterações nesses componentes para permitir a implementação das estruturas BVH, a configuração dos experimentos e a coleta dos dados.

A utilização de uma aplicação já existente também permitiu concentrar o desenvolvimento nas partes relacionadas à comparação das estruturas de aceleração, evitando a necessidade de desenvolver novamente componentes que não fazem parte do objetivo principal deste trabalho.

3.1.2 Estruturas de Aceleração

Apesar de a aplicação utilizar *Vulkan*, as estruturas de aceleração disponibilizadas pela API apresentam algumas limitações para os objetivos deste trabalho. Como apresentado anteriormente, as estruturas BLAS e TLAS utilizadas pelo *Vulkan* possuem sua implementação interna abstraída da aplicação. Dessa forma, não é possível definir diretamente como a estrutura da árvore será organizada ou implementar diferentes estratégias de construção utilizando a estrutura fornecida pela API (Vulkan Documentation Project, 2025a).

Essa abstração é adequada para aplicações que utilizam o sistema de *ray tracing* fornecido pelo *Vulkan*, porém não atende diretamente ao objetivo de comparar diferentes formas de construção e organização de BVHs. Para este trabalho, é necessário possuir controle sobre a estrutura de dados utilizada, permitindo implementar e testar diferentes abordagens, como BVH, LBVH, BSAH e BVH8.

Outro fator considerado foi a necessidade de utilizar recursos específicos da API para realizar a travessia das estruturas fornecidas pelo *Vulkan*. Dessa forma, a utilização dessas estruturas também restringiria a execução da aplicação aos dispositivos que possuem suporte aos recursos necessários para essa abordagem. Neste trabalho, optou-se por manter a utilização do *Vulkan* para as demais etapas da renderização, mas implementar as estruturas BVHs utilizadas nos experimentos de forma própria.

Por não serem utilizadas as estruturas de aceleração fornecidas diretamente pelo *Vulkan*, é necessário implementar nos *shaders* o algoritmo responsável pela travessia da BVH.

Esse algoritmo precisa conhecer a organização dos nós da estrutura, a forma como os dados são armazenados e como acessar os diferentes elementos durante a execução. Essa decisão permite que a mesma aplicação utilize diferentes estruturas e que o processo de travessia seja controlado pelo próprio código desenvolvido para os experimentos.

Essa abordagem também permite separar a estrutura utilizada para armazenar e percorrer a BVH da implementação específica das estruturas de aceleração do *Vulkan*. Com isso, as estruturas avaliadas neste trabalho podem ser construídas e armazenadas de acordo com suas próprias características, enquanto a aplicação mantém uma forma comum de execução para realizar a comparação entre elas.

3.1.3 Organização da Renderização

O processo de renderização foi organizado em três etapas devido à dependência entre os dados produzidos em cada uma delas. A primeira etapa realiza a rasterização da cena e produz os *buffers* que contêm as informações necessárias para o processamento dos efeitos. A segunda etapa utiliza esses dados para realizar os cálculos que dependem da geometria da cena, incluindo o cálculo das sombras por meio das estruturas BVHs. Por fim, a terceira etapa reúne os resultados produzidos pelas etapas anteriores e realiza a composição da imagem final. Dessa forma, a divisão adotada representa o fluxo de dados necessário para a renderização.

A primeira etapa consiste na criação de um *G-buffer* utilizando a técnica de rasterização. Diferentemente de uma utilização convencional da rasterização, nessa etapa são armazenadas informações que serão utilizadas posteriormente durante o processamento dos raios. Entre essas informações estão *position*, *normal*, *albedo*, *material* e *depth*.

O *G-buffer* funciona como uma representação das informações da cena visíveis pela câmera. Dessa forma, a etapa posterior não precisa realizar novamente a busca da geometria correspondente a cada pixel para obter essas informações. A posição, a normal e os demais dados necessários já foram obtidos durante a rasterização e podem ser utilizados pelos *shaders* responsáveis pelo processamento dos efeitos.

Além do *G-buffer* principal, é utilizado um segundo *buffer* com informações semelhantes, porém sem o *depth*. Nesse *buffer* são armazenados materiais considerados transparentes ou que utilizam *blend*, como vidro, água e alguns cristais. Essa separação permite tratar esses materiais de forma diferente durante as etapas posteriores da renderização.

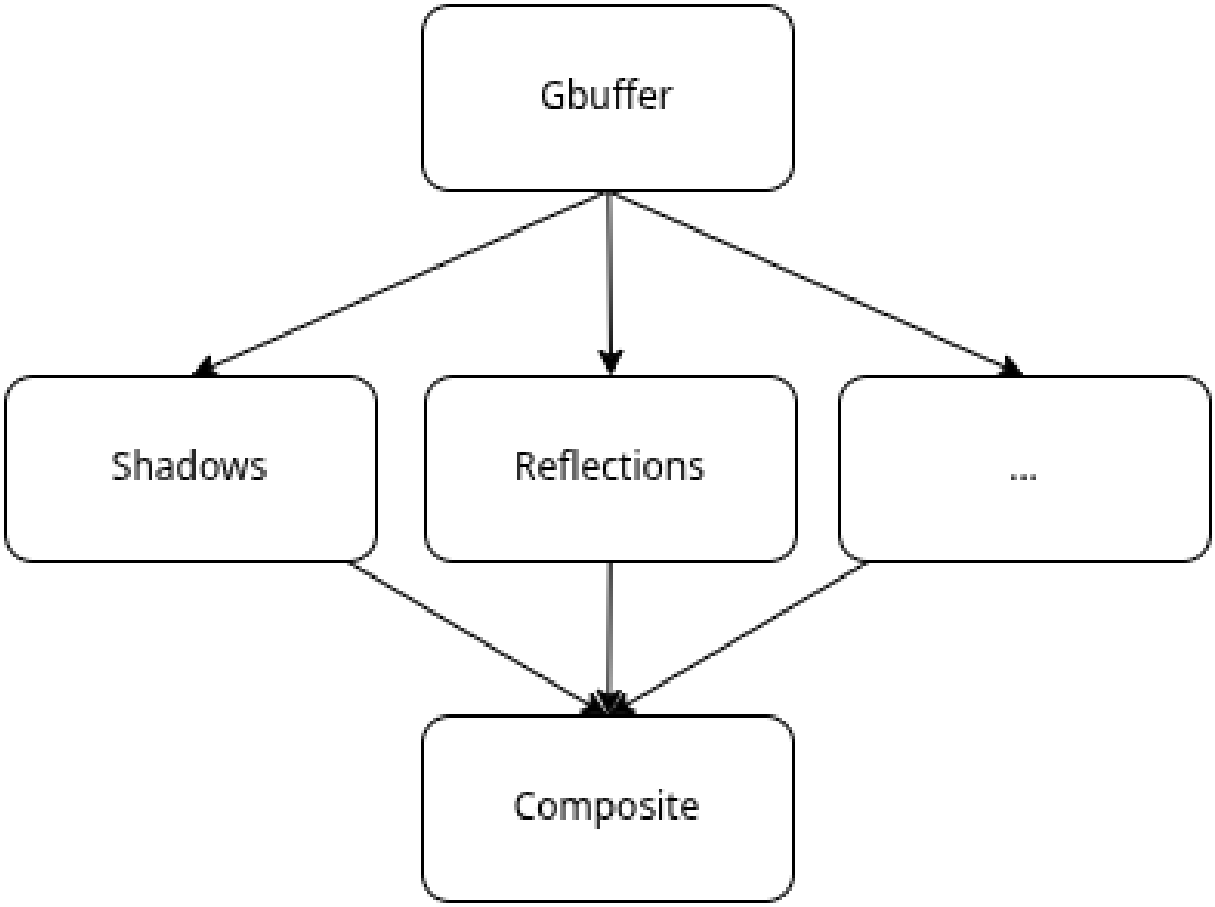
A segunda etapa utiliza as informações armazenadas nos *buffers* para calcular os

efeitos de iluminação que dependem da geometria da cena. Entre esses efeitos estão as sombras e os reflexos. É nessa etapa que as estruturas BVHs desenvolvidas neste trabalho são utilizadas para realizar a busca das interseções entre os raios e a geometria da cena.

Como a travessia da BVH é realizada por meio de uma implementação própria nos *shaders*, o custo dessa etapa pode ser elevado, principalmente quando comparado à utilização das estruturas de aceleração específicas fornecidas pelo *Vulkan*. Por esse motivo, o trabalho concentra a utilização do *ray tracing* em sombras, evitando a inclusão de algoritmos mais complexos de iluminação que poderiam aumentar significativamente o custo da renderização e dificultar a análise do comportamento das estruturas avaliadas.

A terceira etapa é responsável por combinar as imagens geradas nas etapas anteriores e produzir a imagem final apresentada ao sistema. Nessa etapa, os diferentes *buffers* são combinados e podem ser aplicados algoritmos de correção de luminosidade, filtros e outros processos necessários para transformar os resultados intermediários em uma imagem final adequada para apresentação.

Figura 12 – Organização das etapas da aplicação



Fonte: Elaborado pelo autor, utilizando draw.io

Neste trabalho, a segunda etapa implementa apenas o cálculo de sombras (*shadows*). Outros efeitos podem ser implementados posteriormente sem a necessidade de alterar as demais etapas da renderização, sendo necessário apenas incluí-los no processo de geração da imagem final, realizado na terceira etapa, também denominada *Composite*. Dessa forma, a organização adotada permite adicionar diferentes efeitos que utilizem as estruturas BVHs sem modificar a etapa responsável pela geração do *G-buffer*.

3.1.4 Configuração das Estruturas BVH

Para evitar alguns custos associados ao uso de polimorfismo, optou-se pela utilização de *templates* da linguagem C++. *Templates* permitem definir famílias de classes, funções ou tipos que podem ser especializadas a partir dos parâmetros fornecidos. Quando os argumentos são fornecidos, esses parâmetros são substituídos para obter uma especialização específica do *template*, que pode ser instanciada durante a compilação (cppreference, 2026).

Neste trabalho, essa característica é utilizada pelos construtores das estruturas BVHs. Os diferentes construtores possuem uma implementação baseada em *templates*, permitindo utilizar diferentes tipos de nós sem a necessidade de criar uma implementação independente para cada combinação. Dessa forma, o tipo de nó e o construtor utilizados podem ser definidos durante a compilação da aplicação.

A seleção da estrutura utilizada é realizada por meio de definições fornecidas ao compilador. Por exemplo, o trecho utilizado para configurar a BLAS verifica se a definição *USE_BLAS_BVH8*, *USE_BLAS_BSAH* ou *USE_BLAS_LBVH* foi fornecida. Caso nenhuma dessas definições esteja presente, é utilizada a implementação padrão da BVH. O mesmo processo é aplicado à TLAS, permitindo selecionar sua estrutura de forma independente da BLAS.

Essa configuração permite realizar diferentes combinações entre as estruturas sem modificar diretamente o código-fonte da aplicação. A escolha pode ser realizada no momento da compilação por meio de parâmetros fornecidos ao *CMake*. Por exemplo, o comando a seguir seleciona a BVH8 tanto para a BLAS quanto para a TLAS:

```
cmake .. -DCMAKE_CXX_FLAGS="-DUSE_BLAS_BVH8 -DUSE_TLAS_BVH8" \
    -DSHADER_DEFINITIONS="-DUSE_BLAS_BVH8 -DUSE_TLAS_BVH8"
```

Além de definir essas opções para o código C++, as mesmas definições são fornecidas aos *shaders* por meio do parâmetro *SHADER_DEFINITIONS*. Isso é necessário porque a

construção da estrutura e sua travessia precisam utilizar a mesma organização de dados. Dessa forma, quando uma estrutura é selecionada para a BLAS ou para a TLAS, a implementação responsável pela travessia nos *shaders* também pode ser configurada para utilizar a estrutura correspondente.

3.2 APLICAÇÃO DESENVOLVIDA

Nesta seção são apresentadas as principais partes do desenvolvimento relacionadas à construção e à utilização das estruturas BVHs. São descritos o gerenciamento das instâncias, a organização das geometrias e o processo de construção das estruturas de aceleração utilizadas nos experimentos.

3.2.1 Criação de instâncias

As instâncias dos objetos presentes na cena são armazenadas em um vetor de ponteiros. Cada instância pode possuir uma ou mais *submeshes*, que representam partes de uma mesma *mesh* associadas a materiais diferentes. Por exemplo, uma faca pode ser representada por uma única *mesh*, mas possuir uma *submesh* para o cabo de madeira e outra para a lâmina de aço.

Durante a criação do *G-buffer*, essas *submeshes* são organizadas em uma estrutura de dados baseada em *hashmap*, denominada *batch_map*, que armazena diferentes *batches*. Cada *batch* representa uma *submesh* e contém um vetor com as posições das instâncias que utilizam a mesma geometria, mas podem estar localizadas em posições diferentes da cena. Essa organização permite agrupar chamadas de desenho e utilizar a função *vkCmdDrawIndexed* para renderizar múltiplas instâncias da mesma geometria. Dessa forma, é possível reduzir a quantidade de chamadas de desenho necessárias e a sobrecarga associada ao envio de comandos para a GPU (Vulkan.org, 2026d).

Uma abordagem possível seria criar uma BLAS para cada *submesh*. Isso permitiria associar a geometria e seu material de forma mais direta, facilitando a identificação do material correspondente durante a travessia da estrutura. Entretanto, essa não foi a abordagem adotada neste trabalho. A BLAS é construída por *mesh*, abrangendo suas diferentes *submeshes*. Essa decisão exige que as informações de material sejam mantidas e identificadas durante a travessia da BLAS, pois uma mesma estrutura pode conter primitivas associadas a materiais diferentes.

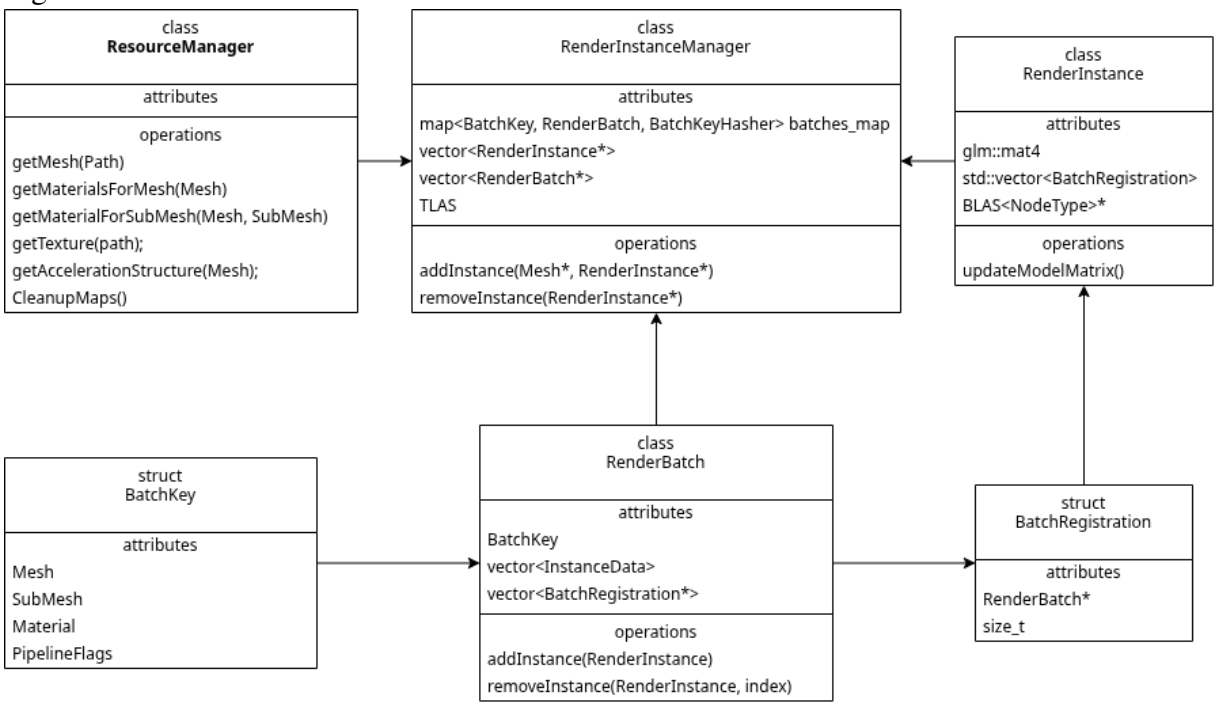
Durante a criação das instâncias, também é criado um registro denominado *Batch-*

Registration. Esse registro mantém a associação entre a instância, sua posição no vetor de instâncias do *batch* e um ponteiro para o próprio *batch*. Dessa forma, é possível atualizar ou remover instâncias sem precisar reconstruir todos os *batches*. Esse mecanismo já existia no *Apotheosis* e foi mantido na aplicação desenvolvida.

Para criar um *batch*, são necessárias as informações presentes na *mesh*, a partir das quais podem ser obtidos os dados utilizados na renderização. Essas informações são gerenciadas pelo *ResourceManager*, responsável por organizar e disponibilizar os recursos utilizados pela aplicação, incluindo aqueles armazenados na CPU e na memória da GPU. Dessa forma, a aplicação pode consultar os recursos existentes e reutilizá-los quando necessário.

O *ResourceManager* também participa do processo de criação das *meshes*. Como as informações necessárias para a construção da BLAS já estão disponíveis nesse processo, a estrutura de aceleração pode ser construída utilizando os mesmos dados da geometria, evitando a necessidade de realizar uma nova busca pelos recursos correspondentes.

Figura 13 – Funcionamento dos *batches*



Fonte: Elaborado pelo autor, utilizando draw.io

3.2.2 Criação de luz

A etapa de criação de luzes apresenta menor complexidade estrutural. Para representar uma fonte de luz na cena, foi criada uma estrutura (*struct*) contendo as principais propriedades necessárias para realizar os cálculos de iluminação nos *shaders*.

Código 1 – *Struct* com informações da luz

```
struct LightData {
    glm::vec3 position;
    float intensity;

    glm::vec3 color;
    float radius;

    Config::LightType type;
    float range;

    float _pad0;
    float _pad1;
};
```

Fonte: Elaborado pelo autor (2026).

A estrutura apresentada no Código 1 armazena a posição da luz, sua intensidade e sua cor. Também são armazenados o raio, o tipo da luz e o alcance utilizado durante os cálculos de iluminação. Essas informações permitem que o *shader* determine como a luz deve influenciar os objetos presentes na cena.

A posição é utilizada para determinar a direção e a distância entre a fonte de luz e o ponto que está sendo iluminado. A intensidade define a quantidade de iluminação produzida pela fonte, enquanto a cor permite alterar as componentes de cor da luz. O raio e o alcance são utilizados para controlar a região de influência da luz, evitando que sua contribuição seja aplicada de forma uniforme em toda a cena.

Após a criação e configuração da estrutura, os dados são armazenados em um *buffer* da GPU. Esse *buffer* é disponibilizado aos *shaders*, permitindo que as propriedades da luz sejam acessadas durante o processo de iluminação. Assim, a aplicação pode alterar as características da luz no lado da CPU, enquanto os *shaders* utilizam essas informações durante a renderização.

Durante a etapa de iluminação, as propriedades da luz são combinadas com as informações armazenadas no *G-buffer*, como posição, normal, material e *albedo*. A partir desses dados, o *shader* calcula a contribuição da luz sobre cada ponto da cena.

No caso das sombras, a posição da luz também é utilizada para determinar a direção do raio que deve ser testado. O raio é então percorrido pela estrutura BVH para verificar se

existe alguma geometria entre o ponto iluminado e a fonte de luz. Caso uma interseção seja encontrada, o ponto é considerado em sombra e a contribuição direta da luz pode ser descartada.

3.2.3 Criação das BVHs

Na aplicação desenvolvida, a BVH é construída por *mesh*, e não individualmente para cada *submesh*. Dessa forma, a estrutura de aceleração pode ser associada à instância correspondente. Quando uma nova *mesh* é adicionada e ainda não possui os recursos necessários registrados, a construção da BLAS pode aproveitar o momento em que os dados das primitivas estão disponíveis na CPU.

Para permitir a utilização de diferentes implementações de BVH, foi necessário definir uma representação comum para os dados utilizados durante a construção. Entre os parâmetros necessários estão os *vertices*, os índices que definem os triângulos e as informações das *submeshes*. Embora a BVH seja construída por *mesh*, as informações das *submeshes* ainda precisam ser consideradas para preservar a associação entre as primitivas e seus respectivos materiais. Isso é importante quando um objeto possui mais de um material ou quando algum deles apresenta transparência, utiliza *blend* ou *mask*.

Inicialmente, é criada uma estrutura denominada *PrimitiveRef*, que representa uma primitiva durante o processo de construção da BVH. Essa estrutura funciona como uma representação intermediária, permitindo organizar as primitivas antes da criação dos nós da árvore. A partir dessas referências, os algoritmos de construção podem organizar as primitivas e gerar a hierarquia correspondente.

A *PrimitiveRef* também contém uma estrutura *AABB* utilizada para representar os limites espaciais da primitiva. Na estrutura final dos nós, entretanto, os limites são armazenados de outra forma: cada nó mantém as AABBs dos seus filhos. Assim, durante a travessia, é possível testar os limites dos filhos antes de acessar os nós correspondentes. Essa organização busca reduzir a quantidade de acessos necessários aos dados dos nós durante a execução do algoritmo.

Outro parâmetro definido durante a construção é a quantidade máxima de triângulos associados a uma instância da BLAS. Nesta implementação, foi estabelecido um limite de quatro triângulos por instância. Essa decisão foi tomada após observar que a árvore apresentava um tamanho elevado, buscando reduzir a quantidade de nós e o espaço necessário para armazenar a estrutura.

Entretanto, esse limite também apresenta desvantagens. Em geometrias pequenas,

como cubos e outros objetos com poucos triângulos, diferentes instâncias podem acabar apresentando AABBs iguais ou muito semelhantes. Nesses casos, o teste de interseção com as caixas delimitadoras consegue descartar menos primitivas, transferindo uma parcela maior do trabalho para os testes de interseção com os triângulos.

Uma alternativa seria implementar um algoritmo capaz de determinar uma divisão mais adequada para cada geometria, considerando suas características espaciais e a quantidade de triângulos. No entanto, essa estratégia não foi implementada neste trabalho e fica como uma possibilidade para trabalhos futuros.

A estrutura utilizada para representar uma instância da BLAS é apresentada no Código 2.

Código 2 – *Struct* da instância da BLAS

```
struct BLASInstance {
    uint32_t firstTriangle; //4
    uint32_t triangleCount; //4

    uint32_t materialOffset; //4
    uint32_t pad0; //4
}; //16
```

Fonte: Elaborado pelo autor (2026).

A estrutura armazena o índice do primeiro triângulo associado à instância, a quantidade de triângulos e o deslocamento utilizado para acessar as informações de material. O campo *pad0* é utilizado como preenchimento, contribuindo para manter o tamanho e o alinhamento esperados para a estrutura.

A representação dos nós da BVH é apresentada no Código 3.

Código 3 – *Struct* do nó da BLAS

```
struct BVHNode {
    AABB childBounds[2]; //128

    uint32_t children[2]; //8

    uint32_t leaf; //4
    uint32_t pad0;//4
}; // 128 + 16 = 144
```

Fonte: Elaborado pelo autor (2026).

Nesse formato, cada nó armazena as caixas delimitadoras dos dois filhos, os índices utilizados para acessá-los e um campo que identifica a condição de folha. O campo *pad0* é utilizado como preenchimento. Essa representação corresponde à estrutura binária utilizada pelas implementações que adotam o tipo *BVHNode*; a BVH8, por utilizar nós com maior quantidade de filhos, emprega uma estrutura específica.

Durante a construção da árvore, as referências *PrimitiveRef* são utilizadas para organizar as primitivas e gerar os nós da BVH. Após a conclusão desse processo, os dados intermediários são convertidos para o formato utilizado durante a execução, incluindo as instâncias da BLAS e os nós da árvore.

Por fim, os nós e as instâncias são transferidos para dois *buffers* distintos na memória da GPU: um destinado às instâncias da BLAS e outro destinado aos nós da estrutura. Embora fosse possível armazenar os dois tipos de dados em um único *buffer*, optou-se por separá-los para simplificar a organização e o acesso aos dados durante a travessia.

O uso de campos de preenchimento, denominados *padding*, em algumas estruturas está relacionado aos requisitos de alinhamento de memória para o acesso aos dados pelos *shaders*. O *Vulkan* define regras de alinhamento para os dados armazenados em *buffers*, que devem ser consideradas na organização das estruturas compartilhadas entre a aplicação e os *shaders* (Vulkan.org, 2026c). Além disso, quando é utilizado *Buffer Device Address*, os endereços de memória e os dados acessados precisam respeitar os requisitos de alinhamento aplicáveis (Vulkan.org, 2026a).

3.2.4 Criação da TLAS

A TLAS possui um processo de criação semelhante ao da BLAS em alguns aspectos. Primeiramente, são criadas as referências para a BLAS de cada instância que será renderizada. Da mesma forma que na criação da BLAS, também é utilizada a estrutura *PrimitiveRef*. Entretanto, nesse caso, o campo *materialOffset* não é utilizado, pois essa informação é necessária apenas para as instâncias da BLAS.

Utilizando as *PrimitiveRef*, o processo de construção dos nós segue o mesmo princípio utilizado na BLAS. Após a criação dos nós, as *PrimitiveRef* são convertidas em instâncias da TLAS. Por fim, a TLAS é transferida para a GPU da mesma forma que a BLAS, porém utilizando um *buffer* separado.

Código 4 – *Struct* da instância da TLAS

```
struct TLASInstance {
    glm::mat4 inverseTransform; // 4*4*32 512

    uint64_t vertexAddress; //8
    uint64_t indexAddress; //8

    uint32_t blasIndex; // 4
    uint32_t nodeOffset; // 4
    uint32_t nodeCount; // 4
    uint32_t instanceOffset; // 4
};
```

Fonte: Elaborado pelo autor (2026).

A estrutura apresentada armazena a transformação inversa da instância, os endereços dos vértices e dos índices da geometria, além das informações necessárias para localizar a BLAS correspondente e os nós associados à instância. O campo *blasIndex* identifica a estrutura utilizada pela instância, enquanto *nodeOffset* e *nodeCount* permitem localizar os nós correspondentes. O campo *instanceOffset* é utilizado para identificar o deslocamento da instância dentro dos dados associados à estrutura.

3.3 CONSTRUÇÃO E TRAVESSIA DAS BVHs

Esta seção apresenta o processo utilizado para construir e percorrer os nós das árvores BVH implementadas neste trabalho. Inicialmente, é apresentada a implementação da BVH binária, utilizada como referência para compreender as demais estratégias. Em seguida, são descritas as alterações realizadas nas abordagens LBVH, BSAH e BVH8.

3.3.1 BVH

Exceto pela BVH8, todas as demais árvores utilizam o mesmo tipo de nó apresentado no Código 3 e seguem essencialmente a mesma forma de travessia. As principais diferenças entre essas estruturas estão relacionadas ao processo de construção. Por esse motivo, inicialmente é apresentada a implementação mais simples, permitindo compreender a organização utilizada e as alterações realizadas nas demais estratégias.

3.3.1.1 Construção

Dentro das funções *BLASInstanceBuilder::build* ou *TLASInstanceBuilder::build*, é chamada a função *BuilderType::build*. No nível responsável pela construção dos nós, a implementação é generalizada, permitindo que a mesma lógica seja utilizada tanto para a BLAS quanto para a TLAS. O tipo *BuilderType* é definido como um *template* de C++, permitindo utilizar diferentes classes de construção desde que forneçam uma função *build* compatível com os parâmetros esperados. Essa verificação ocorre durante a compilação.

No caso da BVH, a função *build* constrói a árvore de cima para baixo (*top-down*), ou seja, partindo da raiz, que contém todas as primitivas, e subdividindo o espaço sucessivamente até alcançar as folhas.

O algoritmo inicia considerando todo o conjunto de primitivas como um único bloco. Para evitar chamadas recursivas, ele utiliza uma pilha para controlar o processo de divisão de forma iterativa. A cada etapa, retira-se um intervalo da pilha e cria-se um nó. Caso esse nó possua um pai, ele é vinculado ao *slot* correspondente do nó superior e sua caixa delimitadora (AABB) é calculada.

Quando o nó contém mais de duas primitivas, ele atua como um nó interno: o algoritmo analisa a posição dos centroides das geometrias para escolher o melhor eixo de corte, reordena as primitivas dividindo-as ao meio com o *std::nth_element* e empilha as duas metades resultantes, esquerda e direita, para serem processadas em seguida. Esse processo de divisão continua até que um grupo passe a ter apenas uma ou duas primitivas. Nessa condição de parada, o nó é finalizado como uma folha (*leaf = 1*) e passa a armazenar diretamente os índices dessas primitivas, encerrando a ramificação daquele ramo.

3.3.1.2 Travessia

A travessia ocorre nos *shaders*. Da mesma forma que as estruturas foram criadas no C++, também é necessário criar suas representações correspondentes nos *shaders*. Foram implementadas algumas funções auxiliares para realizar a travessia, como *intersectTriangle*, utilizada para verificar interseções com os triângulos, e *intersectAABB*, utilizada para verificar interseções com as AABBs.

Atualmente, a travessia pode processar mais de um raio de forma iterativa utilizando uma pilha local na memória do *shader*. O fluxo de execução inicia na raiz da árvore, testando a interseção do raio com a caixa delimitadora do nó atual por meio da função *intersectAABB*.

Caso o raio não atinja a AABB, o ramo inteiro é descartado.

Se houver interseção e o nó for interno, os filhos esquerdo e direito são avaliados e empilhados, priorizando a visita ao filho mais próximo para aumentar a possibilidade de encontrar uma interseção mais próxima antecipadamente. No entanto, essa implementação possui uma limitação: em determinados cenários, o código não consegue realizar a parada antecipada (*early exit*) de forma completa, o que pode obrigar a travessia a explorar nós que poderiam ser descartados.

Por outro lado, caso o nó seja uma folha, o raio é testado diretamente contra as primitivas armazenadas utilizando a função *intersectTriangle*, atualizando o ponto de colisão mais próximo caso haja sucesso. Vale destacar também que, por ser implementado inteiramente por código próprio no *shader*, esse algoritmo não consegue aproveitar as estruturas de aceleração de *hardware* nativas disponibilizadas pelo *driver* da GPU, como os *RT cores*.

3.3.2 LBVH

A LBVH utiliza os códigos de Morton para estabelecer uma ordenação espacial das primitivas antes da construção da hierarquia. Esses códigos representam a posição espacial de cada primitiva por meio de uma sequência de bits obtida a partir das coordenadas de seu centroide. Após a ordenação, as primitivas são organizadas em uma sequência linear, a partir da qual são determinadas as divisões da hierarquia binária (Pantaleoni; Luebke, 2010).

O processo de construção começa com o cálculo dos limites dos centroides das primitivas. A partir desses limites, o centroide de cada primitiva é normalizado para o intervalo entre zero e um em cada eixo. Essa normalização permite representar as posições independentemente da escala da geometria utilizada.

Após a normalização, cada centroide é convertido em um código de Morton por meio da função *morton3D*. Na implementação desenvolvida, cada coordenada normalizada é quantizada para um valor inteiro entre zero e 1023, utilizando 10 bits por eixo. Em seguida, a função *expandBits* separa os bits de cada coordenada para permitir a intercalação das representações binárias dos eixos X, Y e Z. O resultado é um código de 30 bits que representa a posição tridimensional da primitiva em uma ordem linear. Esse procedimento segue o princípio geral de geração de códigos de Morton descrito na literatura (Pantaleoni; Luebke, 2010).

O código de Morton é armazenado juntamente com a *PrimitiveRef* correspondente em uma estrutura denominada *MortonPrimitive*. Após a geração desses valores para todas

as primitivas, os elementos são ordenados pelo código de Morton utilizando a função *oneapi::tbb::parallel_sort*. Dessa forma, primitivas com posições espacialmente próximas tendem a ocupar posições próximas na sequência ordenada, embora essa proximidade não seja garantida em todos os casos.

Com as primitivas ordenadas, a construção da árvore é realizada utilizando intervalos dessa sequência. Para determinar a divisão de cada intervalo, a função *determineSplit* utiliza a quantidade de bits iniciais em comum entre os códigos de Morton das primitivas localizadas nas extremidades do intervalo. Essa informação é obtida pela função *commonPrefix*, que compara os códigos por meio da operação *XOR* e determina a quantidade de bits iniciais iguais.

A partir desse prefixo comum, a função *determineSplit* procura uma posição adequada para separar o intervalo em dois grupos. A busca avalia posições candidatas dentro do intervalo e verifica o prefixo comum entre o código da primeira primitiva e o código de cada candidata. Quando o prefixo comum da candidata é maior que o prefixo do intervalo completo, a posição é considerada uma possível divisão. O processo continua até determinar a posição de separação utilizada para criar os dois filhos do nó.

A construção dos nós é realizada de forma paralela por níveis da árvore. Para cada nível, os intervalos que ainda precisam ser divididos são armazenados em estruturas *BuildTask*. Esses intervalos são processados utilizando *oneapi::tbb::parallel_for*, permitindo determinar as divisões e inicializar os nós de forma paralela. Os novos intervalos gerados são então utilizados como entrada para o próximo nível da construção.

Dessa forma, a implementação utiliza os códigos de Morton para ordenar espacialmente as primitivas e determinar as divisões da hierarquia. Após essa etapa, as AABBs são calculadas para os intervalos correspondentes e os nós são organizados em uma estrutura binária. A utilização de operações paralelas durante a geração dos códigos, a ordenação e a construção dos níveis permite explorar o paralelismo disponível durante a construção da LBVH.

3.3.3 BSAH

Enquanto a BVH tradicional divide as primitivas simplesmente ao meio usando a mediana, a construção baseada em SAH (*Surface Area Heuristic*) busca otimizar o custo de interseção do raio, escolhendo o ponto de corte que minimiza a probabilidade de o raio atingir os nós filhos. Para evitar o alto custo computacional de avaliar todas as posições possíveis de corte, esta implementação utiliza uma abordagem compartimentada (*Binned SAH*).

O processo inicia para cada intervalo gerado iterativamente por uma pilha, avaliando os eixos espaciais disponíveis. Em vez de ordenar todas as primitivas individualmente para cada teste, o algoritmo distribui as geometrias em um número fixo de compartimentos (*bins*, por padrão 16) com base nas coordenadas de seus centroides. Para cada compartimento, são acumuladas as caixas delimitadoras (AABB) e a contagem de elementos.

Em seguida, o algoritmo calcula prefixos e sufixos cumulativos dos custos de superfície utilizando a heurística de área, avaliando rapidamente o custo combinado de dividir o conjunto em cada ponto de fronteira entre os compartimentos. O ponto de corte que apresenta o menor custo global é selecionado.

Com o eixo e o ponto de divisão definidos, a função *std::partition* reagrupa as primitivas em duas metades com base no compartimento correspondente. Caso ocorra algum desbalanceamento extremo que impeça a partição por compartimentos, o algoritmo recorre a um método de divisão alternativo por mediana (*std::nth_element*). Por fim, os subintervalos esquerdo e direito são empilhados, calculando-se as AABBs finais para cada filho criado.

3.3.4 BVH8

A estrutura do nó muda em relação à BVH binária tradicional; o nó em si fica maior, mas a ideia principal de hierarquia espacial se mantém.

Código 5 – *Struct* do nó da BVH8

```
1  struct BVH8Node {
2  AABB childBounds[8]; //256
3  uint32_t children[8]; //32
4
5  uint32_t childCount; //4
6
7  uint32_t leaf; //4
8
9  uint32_t pad0; //4
10 uint32_t pad1; //4
11 }; //256 + 32 + 16
```

Fonte: Elaborado pelo autor (2026).

No que diz respeito à construção (*BVH8Builder*), a principal diferença em relação à BVH binária é o fator de ramificação. Enquanto a árvore binária divide o intervalo de primitivas

estritamente em duas metades, a BVH8 particiona o conjunto em até oito partes a cada nível. O algoritmo utiliza uma função recursiva que ordena as primitivas ao longo do eixo de maior dispersão e as distribui uniformemente entre os filhos disponíveis, calculando a AABB correspondente para cada um deles. Além disso, as folhas da BVH8 suportam armazenar até oito primitivas diretamente (*MAX_LEAF_PRIMS = 8*), reduzindo a profundidade total da árvore.

No caso específico da travessia, a BVH8 aproveita essa alta capacidade de ramificação: em vez de testar dois filhos, o algoritmo avalia múltiplas AABBs, podendo aproveitar o paralelismo disponível na GPU. Como resultado, a árvore fica muito mais rasa, reduzindo a quantidade de níveis que precisam ser percorridos durante o rastreamento dos raios e, consequentemente, a quantidade de acessos aos nós da estrutura.

3.4 CONFIGURAÇÃO DOS EXPERIMENTOS

Nesta seção são apresentados os procedimentos utilizados para a execução dos testes e o planejamento dos experimentos.

3.4.1 Configuração de Hardware e Software

Os testes foram executados em um ambiente pessoal operando sob o sistema operacional *Arch Linux*, utilizando o *kernel Linux* 7.1.11-arch1-1 e o sistema de arquivos *Btrfs* na partição raiz (*/*). O ambiente gráfico utilizado foi o gerenciador de janelas *KDE Plasma* 6.7.4, rodando sobre o protocolo *Wayland*.

Em termos de *hardware*, a máquina é equipada com um processador *Intel Core i5-10600K* (6 núcleos, 12 *threads* operando a uma frequência máxima de 4,10 GHz), acompanhado de 16 GiB de memória RAM. O processamento gráfico e as execuções de computação paralela via API *Vulkan* ficaram a cargo de uma placa de vídeo dedicada *NVIDIA GeForce RTX 5060*, com 8 GiB de memória *Video Random Access Memory* (VRAM), controlada pelo *driver* proprietário da *NVIDIA* na versão 610.57.04 e com suporte à instância *Vulkan* 1.4.357.

3.4.2 Preparação para os Experimentos

O primeiro ponto, e mais simples, é que o *Nsight Systems* não monitora diretamente o uso de memória RAM do processo. Para obter essa informação, a forma mais prática seria executar um *script* em paralelo durante a execução do teste. Contudo, uma métrica ainda mais útil é o consumo específico de memória das estruturas BVH; por isso, em um teste isolado, a

aplicação foi configurada para retornar no terminal o tamanho exato ocupado pelas estruturas BLAS e TLAS.

O *Nsight Systems* consegue capturar a grande maioria dos eventos de forma nativa, mas alguns eventos específicos exigem marcações manuais. Para isso, podem ser utilizados marcadores tanto na CPU quanto na GPU por meio do *NVIDIA Tools Extension Library* (NVTX). Tratando-se de uma API para anotação de código que oferece diversas funcionalidades, foi empregado especificamente o recurso *nvtx3::scoped_range* (NVIDIA, 2026c). Esse componente funciona como um marcador que armazena um título descritivo e finaliza automaticamente a marcação de tempo assim que o bloco de código sai de escopo.

Esse marcador foi utilizado para isolar e medir precisamente o tempo de construção e de transferência das BVHs para a GPU.

3.4.3 Cenas Utilizadas

A cena é a mesma para todos os experimentos. Ela foi construída de forma rápida, o que significa que não se trata de um cenário realista ou representativo de uma aplicação de produção. A cena consiste em uma junção de várias instâncias dispostas de forma um tanto desordenada, utilizando alguns modelos 3D que não são de autoria própria. A licença de uso estará disponível junto ao código-fonte. Esse modelos vieram de dos usuarios (Eveston - Elite Big Speakerman - Hugo) e (bean(alwayshasbean)), da plataforma *Sketchfab*, utilizada para publicação e compartilhamento de modelos 3D.

Além das múltiplas instâncias, o sistema também conta com animação. A animação compreende o movimento, a rotação e a alteração de escala do modelo completo. A câmera também se movimenta, rotacionando e se afastando do centro da cena em um trajeto circular.

Todas essas alterações garantem que a TLAS sofra modificações constantes, ainda que sutis, a cada *frame*. Além disso, a movimentação da câmera modifica os dados enviados ao *buffer* e assegura que novas regiões da TLAS sejam continuamente exploradas.

3.4.4 Parâmetros de Execução

Antes de iniciar os experimentos, realizou-se um levantamento minucioso para identificar quais métricas descritas na literatura poderiam ser efetivamente coletadas e de que forma essa extração seria realizada. Embora algumas métricas específicas tenham permanecido pendentes por limitações de contadores nas ferramentas de análise, o conjunto de dados obtidos

ainda supera a variedade encontrada em trabalhos correlatos.

As métricas foram divididas e estruturadas considerando as capacidades do *NVIDIA Nsight Systems* e do *NVIDIA Nsight Graphics*. Métricas de tempo de construção das estruturas de aceleração (BLAS e TLAS), tempo de quadro na GPU (*GPU Frame Time*), utilização dos multiprocessadores de transmissão (SM), execuções de *shaders* e transferência de dados via *PCIe* foram extraídas primariamente com o auxílio do *Nsight Graphics*. Em contrapartida, informações de uso de memória RAM do sistema, consumo energético monitorado via *NVIDIA Management Library* (NVML), temperatura da GPU, trocas de contexto (*context switches*) e atividades nas filas do *Vulkan* foram coletadas por meio do *Nsight Systems*.

Duas métricas merecem destaque metodológico devido ao seu tratamento específico:

1. ***Ray Tracing Throughput*:** Como a travessia da BVH é realizada diretamente via código próprio em *shaders* (e não pelos núcleos de *ray tracing* dedicados do *hardware*), o rendimento foi calculado relacionando o tempo de travessia ao número total de raios processados por quadro, por exemplo, avaliando resoluções fixas em *Mrays/s*.

2. ***Warp Divergence*:** Métricas associadas estritamente à divergência de caminhos condicionais dentro de um mesmo *warp* permaneceram pendentes de validação por contadores diretos, evitando o uso de métricas genéricas de ocupação de SM como equivalentes.

Para garantir a justiça na comparação de desempenho entre as diferentes estruturas avaliadas (BVH binária, BVH8, LBVH e BSAH), todas as execuções foram submetidas estritamente às mesmas condições de contorno: a mesma cena animada, idêntico número de instâncias geométricas, mesma resolução de tela, mesma disposição de câmera e a mesma carga de trabalho computacional (*workload*).

4 COLETA E TRATAMENTO DOS DADOS

Nesta seção, descreve-se a metodologia adotada para a extração, o monitoramento e o processamento das métricas de desempenho das diferentes estruturas de aceleração avaliadas.

4.1 EXECUÇÃO DOS EXPERIMENTOS

A execução dos testes foi estruturada em duas etapas complementares de perfilamento (*profiling*) para contemplar tanto o comportamento dinâmico e temporal da aplicação quanto os detalhes da renderização gráfica:

1. **Coletas de Longa Duração (*Nsight Systems*):** Os testes voltados a capturar o consumo de recursos globais do sistema, uso de memória RAM (*Peak RAM Usage*), temperatura, energia e atividades de filas do *Vulkan* foram executados com uma janela de duração de 30 segundos contínuos para cada tipo de BVH. Isso permitiu estabilizar as médias e registrar o comportamento da aplicação sob a animação contínua da cena e da câmera.

2. **Coletas de Quadros Detalhados (*Nsight Graphics*):** Para a análise minuciosa do *pipeline* gráfico, dos tempos de construção das estruturas de aceleração (*BLAS/TLAS Build Time*) e da execução dos *shaders*, as capturas foram realizadas em momentos específicos. O *profiling* detalhado coletou amostras exatas no segundo quadro e em instantes distribuídos nos segundos 10 e 20 de execução.

Dessa forma, o isolamento dos dados de sistema por meio do *Nsight Systems*, aliado à precisão temporal do *Nsight Graphics*, permitiu mapear com alta fidelidade o impacto de cada variação estrutural de BVH sobre a GPU e a CPU.

4.2 COLETA DAS MÉTRICAS

As execuções dos testes para cada tipo de BVH foram realizadas em uma mesma sessão experimental, garantindo consistência ambiental. Os artefatos gerados foram organizados em pastas nomeadas de acordo com a respectiva estrutura de aceleração, cujos códigos e arquivos estarão disponíveis no repositório oficial do projeto. Cada diretório contém o executável correspondente, os *shaders* associados e os modelos 3D necessários para a replicação integral dos testes.

Para cada configuração, foram coletados arquivos de registro e relatórios de perfilamento (*profiling*). Especificamente, encontram-se três arquivos com a extensão *.ngfx-capture*, referentes às capturas detalhadas do *Nsight Graphics*, e um arquivo *.nsys-rep*, gerado pelo *Nsight Systems* para a análise global do sistema. Além disso, conforme mencionado anteriormente, o arquivo *bvh_size_log.log* armazena os dados textuais referentes ao tamanho em memória de cada estrutura (BLAS e TLAS).

4.3 TRATAMENTO DOS DADOS

O *Nsight Systems* permite coletar uma grande quantidade de métricas durante a execução da aplicação. Entretanto, a ferramenta é voltada principalmente para a análise da execução ao longo do tempo e para a identificação de problemas de desempenho. Embora forneça os dados necessários para a análise, a apresentação e o tratamento estatístico dessas informações não são suficientes para a análise geral realizada neste trabalho. Dessa forma, foi necessário desenvolver uma etapa adicional para extrair, organizar e analisar os dados obtidos durante os experimentos.

O *Nsight Graphics* apresenta uma abordagem diferente, pois seu objetivo está mais relacionado à análise de um *frame* específico e à visualização das operações executadas durante sua renderização. Dessa forma, os dados obtidos nessa ferramenta são analisados de acordo com as características de cada métrica e com a forma como ela é disponibilizada pela própria ferramenta. Para as métricas obtidas pelo *Nsight Systems*, foi desenvolvido um processo próprio de extração e tratamento dos dados.

A primeira etapa desse processo é realizada pelo *script Python extrair_nsys.py*. O *script* utiliza o executável *nsys* do *Nsight Systems* para processar o arquivo *.nsys-rep* gerado durante a coleta. Como parte desse processo, os dados da execução são disponibilizados em um banco de dados *SQLite*, permitindo consultar as tabelas e informações registradas pelo *Nsight Systems*. A partir dessas informações, o *script* identifica as métricas utilizadas neste trabalho, realiza a extração dos dados correspondentes e gera um arquivo *Excel* denominado **_report_dados.xlsx*. Essa etapa permite separar os dados relevantes das demais informações registradas durante a execução (NVIDIA, 2026d).

A localização da instalação do executável *nsys* pode variar de acordo com o ambiente utilizado. Por esse motivo, o *script* possui uma variável denominada *NSYS_DEFAULT*, que pode precisar ser ajustada para apontar para a instalação correspondente do *Nsight Sys-*

tems. Essa configuração permite que o mesmo processo de extração seja utilizado em diferentes ambientes de execução.

Após a extração, os dados passam por uma segunda etapa de processamento realizada pelo *script Python analisar_nsys.py*. Esse *script* utiliza o arquivo gerado na etapa anterior e organiza os dados de acordo com cada métrica, calculando informações estatísticas utilizadas posteriormente na análise dos resultados. O resultado é armazenado em outro arquivo *Excel*, denominado **_report_analise.xlsx*.

Para cada métrica, são calculadas informações como unidade de medida, quantidade de amostras, soma, média, mediana, moda, tipo da moda, desvio padrão, valor mínimo e valor máximo. Também são calculadas as mesmas informações após a remoção dos valores considerados *outliers*, permitindo comparar os resultados antes e depois do tratamento dos dados.

A identificação dos *outliers* é complementada pelo cálculo dos limites inferior e superior do intervalo interquartil. Esses limites são utilizados para identificar valores que se encontram fora da faixa considerada para as amostras da métrica. Dessa forma, além dos valores estatísticos originais, o arquivo de análise registra a quantidade e o percentual de *outliers*, bem como os limites utilizados para sua identificação.

O arquivo final contém, portanto, os dados necessários para comparar as métricas obtidas durante os experimentos de forma organizada. Essa separação entre a coleta, a extração e a análise dos dados também permite repetir o processo para diferentes arquivos de execução sem a necessidade de realizar manualmente os cálculos para cada experimento.

Os dois *scripts* são executados para os arquivos de todas as BVHs avaliadas neste trabalho. A partir dos arquivos **_report_analise.xlsx* gerados para cada estrutura, é possível reunir os dados e gerar gráficos que facilitam a visualização e a comparação das métricas.

Para isso, foi desenvolvido um terceiro *script Python*, responsável por reunir os arquivos de análise das diferentes BVHs e gerar imagens com os gráficos correspondentes. Esses gráficos auxiliam na identificação de diferenças entre as estruturas e facilitam a análise dos resultados obtidos durante os experimentos.

Ambos os *scripts* foram desenvolvidos com o auxílio de inteligência artificial generativa. Desse modo, embora a consistência de todas as métricas apresentadas tenha sido verificada, os *scripts* geram dados adicionais que não foram utilizados neste trabalho, sendo necessária a validação manual desses valores ao menos uma vez para trabalhos futuros.

5 RESULTADOS E DISCUSSÃO

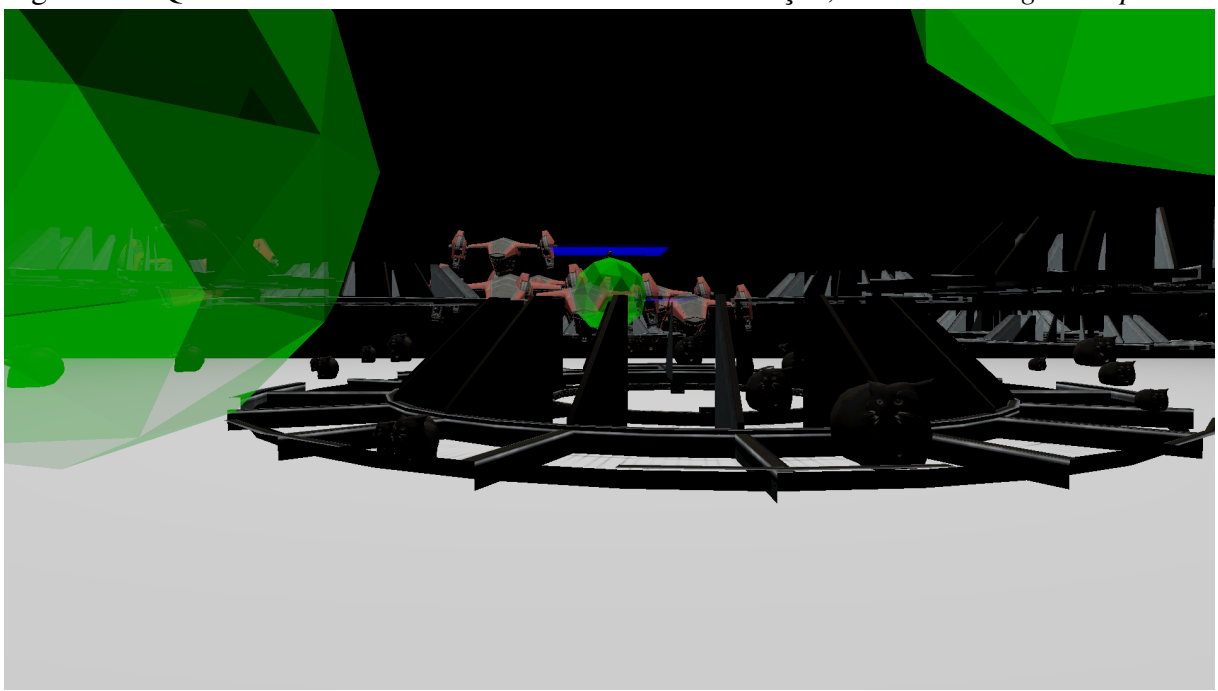
Neste capítulo são apresentados e discutidos os resultados obtidos durante os experimentos realizados com as diferentes estruturas BVHs. A análise considera as métricas coletadas durante a execução da aplicação, buscando identificar as diferenças de desempenho entre as abordagens avaliadas e relacioná-las às características dos algoritmos de construção e das estruturas geradas.

5.1 CENA DESENVOLVIDA E AMOSTRAS DE RENDERIZAÇÃO

Conforme apresentado anteriormente, o ambiente virtual adotado permaneceu idêntico em todas as simulações executadas. A partir das capturas realizadas durante o perfilamento no *Nsight Graphics*, é possível reconstruir os quadros renderizados pela aplicação e observar o resultado visual obtido.

A Figura 14 ilustra o quadro gerado no instante de 20 segundos de execução, destacando a composição final da cena e a aplicação do cálculo de sombras via *ray tracing*.

Figura 14 – Quadro renderizado no instante de 20s de simulação, obtido via *Nsight Graphics*.

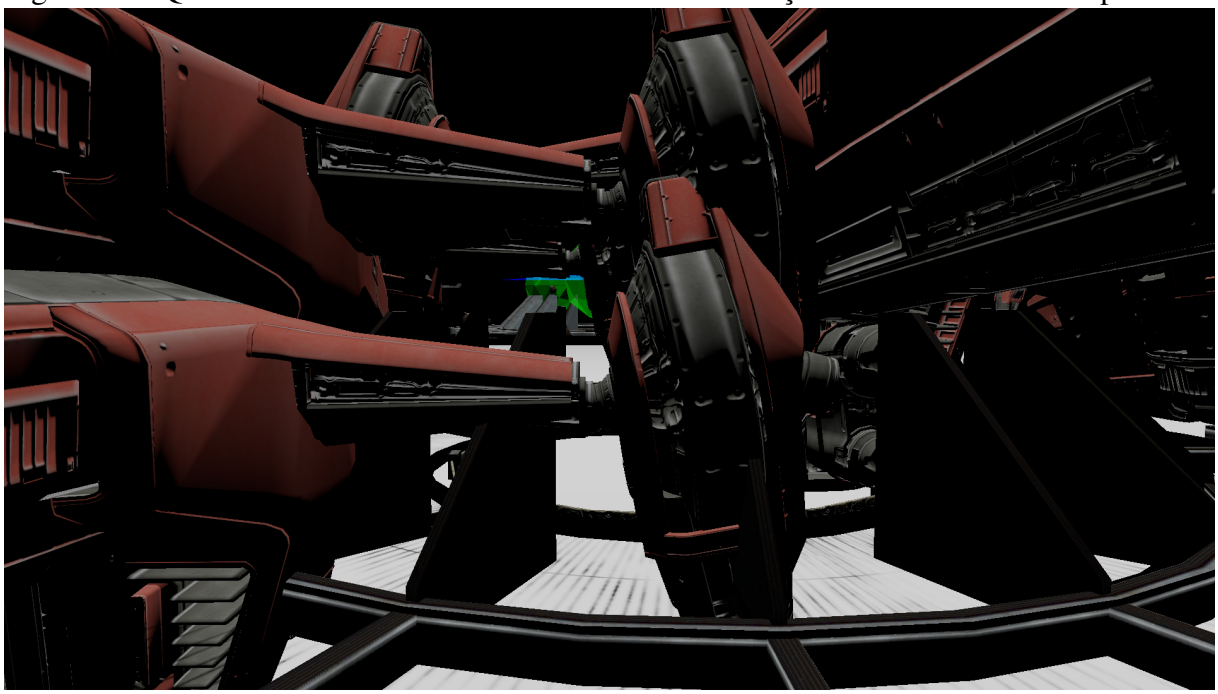


Fonte: Elaborado pelo autor, utilizando *Nsight Graphics*.

A Figura 15 apresenta um momento da simulação caracterizado pela sobreposição de múltiplas instâncias geométricas. A inclusão desses cenários com maior densidade espacial

é fundamental para avaliar o comportamento e a eficiência das BVHs sob diferentes cargas de trabalho e complexidades de travessia.

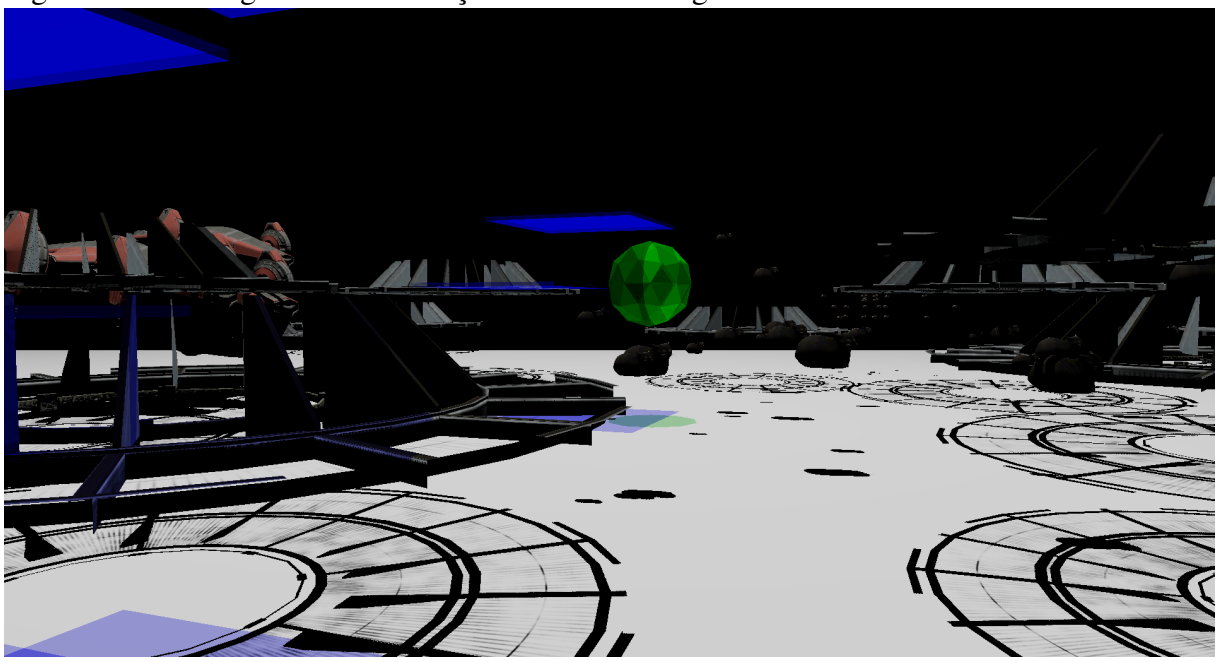
Figura 15 – Quadro renderizado no instante de 10s de simulação com instâncias sobrepostas.



Fonte: Elaborado pelo autor, utilizando *Nsight Graphics*.

Por fim, a Figura 16 expõe uma perspectiva alternativa da cena, permitindo visualizar de forma mais ampla a distribuição espacial dos modelos 3D e a movimentação realizada pela câmera ao longo do experimento.

Figura 16 – Visão geral da distribuição das instâncias geométricas no ambiente virtual.



Fonte: Elaborado pelo autor.

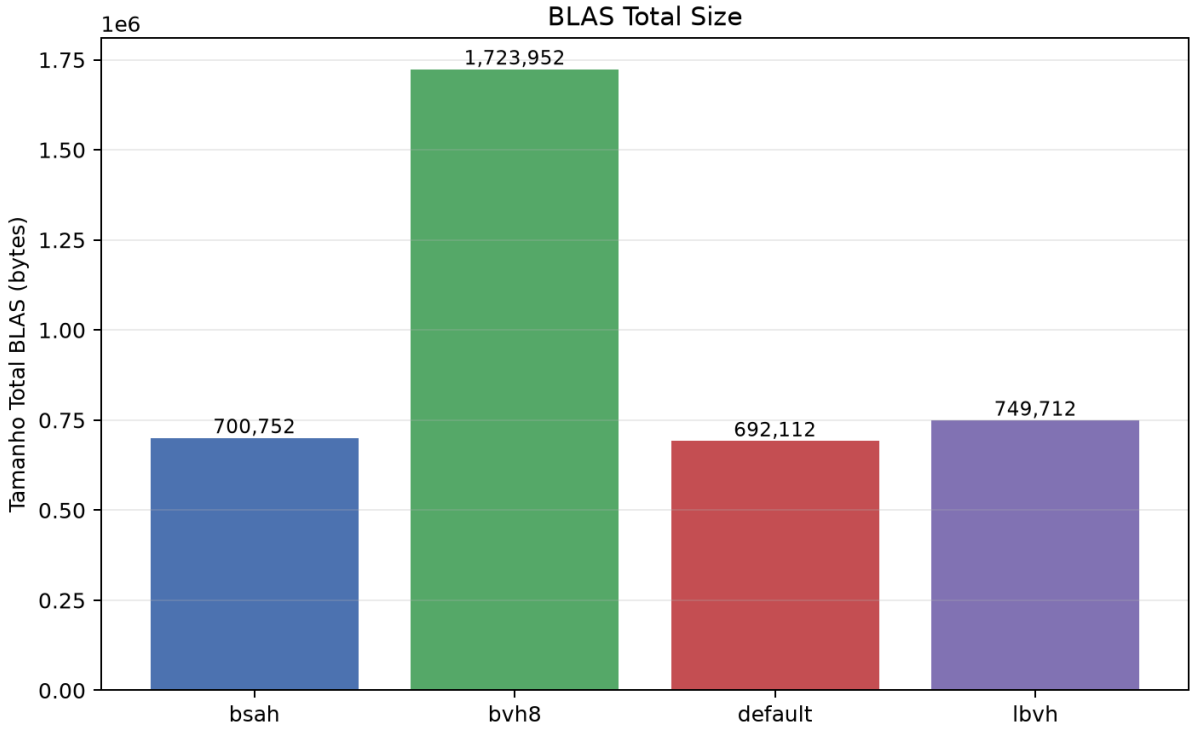
5.2 APRESENTAÇÃO DAS MÉTRICAS

5.2.1 Consumo de memória

Para iniciar a análise do consumo de memória, é necessário observar o tamanho das estruturas BVHs geradas durante os experimentos. Embora a BVH8 seja a única abordagem avaliada que utiliza um tipo de nó diferente, a quantidade de memória utilizada pelas estruturas também depende do número de nós gerados durante a construção da árvore. Dessa forma, mesmo entre as estruturas que utilizam o mesmo tipo de nó, podem existir diferenças no tamanho total, de acordo com a organização das primitivas e com a quantidade de nós necessária para representar a hierarquia.

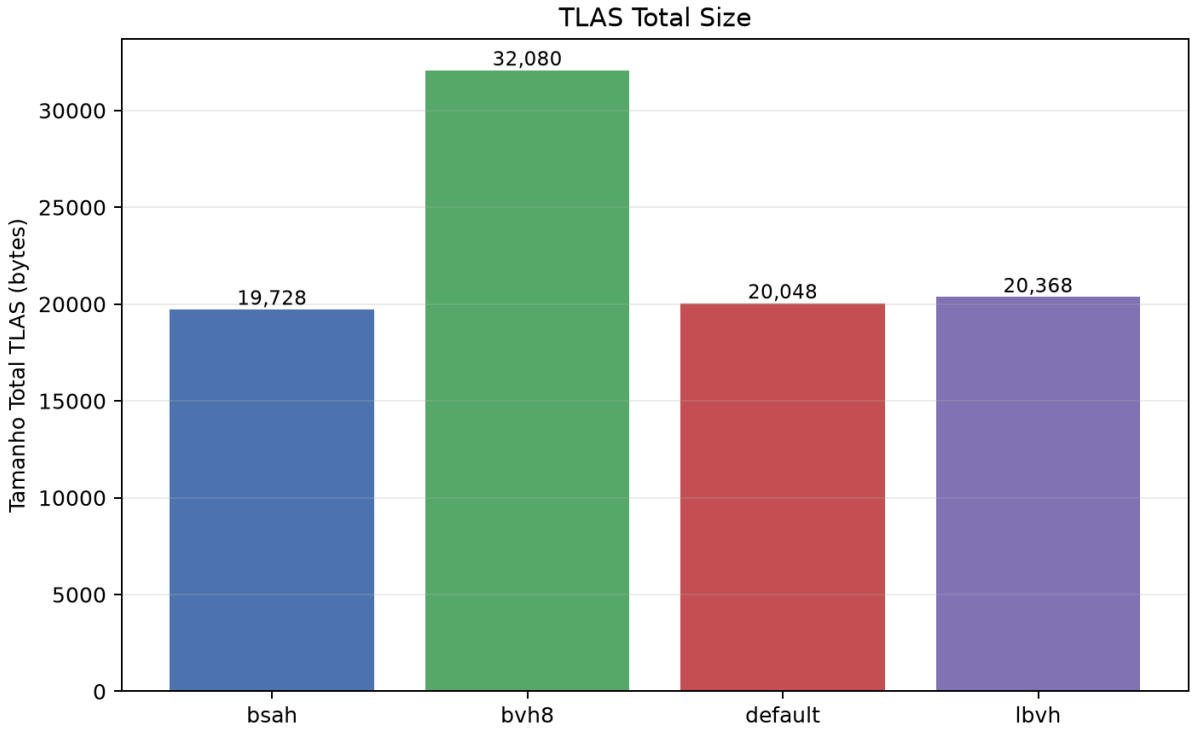
Ao analisar os resultados apresentados nas Figuras 17 e 18, observa-se que a BVH8 apresenta estruturas consideravelmente maiores que as demais abordagens. Esse comportamento está diretamente relacionado ao tamanho dos nós utilizados nessa estrutura. Enquanto as demais abordagens utilizam nós binários, a BVH8 possui espaço para armazenar uma quantidade maior de filhos em cada nó. Entretanto, nem todos os nós necessariamente utilizam todas essas posições, fazendo com que parte do espaço reservado possa permanecer sem utilização efetiva. Dessa forma, a maior capacidade de cada nó também resulta em um maior consumo de memória para representar a hierarquia.

Figura 17 – Tamanho total das BLASs de cada implementação



Fonte: Elaborado pelo autor, utilizando Python matplotlib

Figura 18 – Tamanho total da TLAS de cada implementação



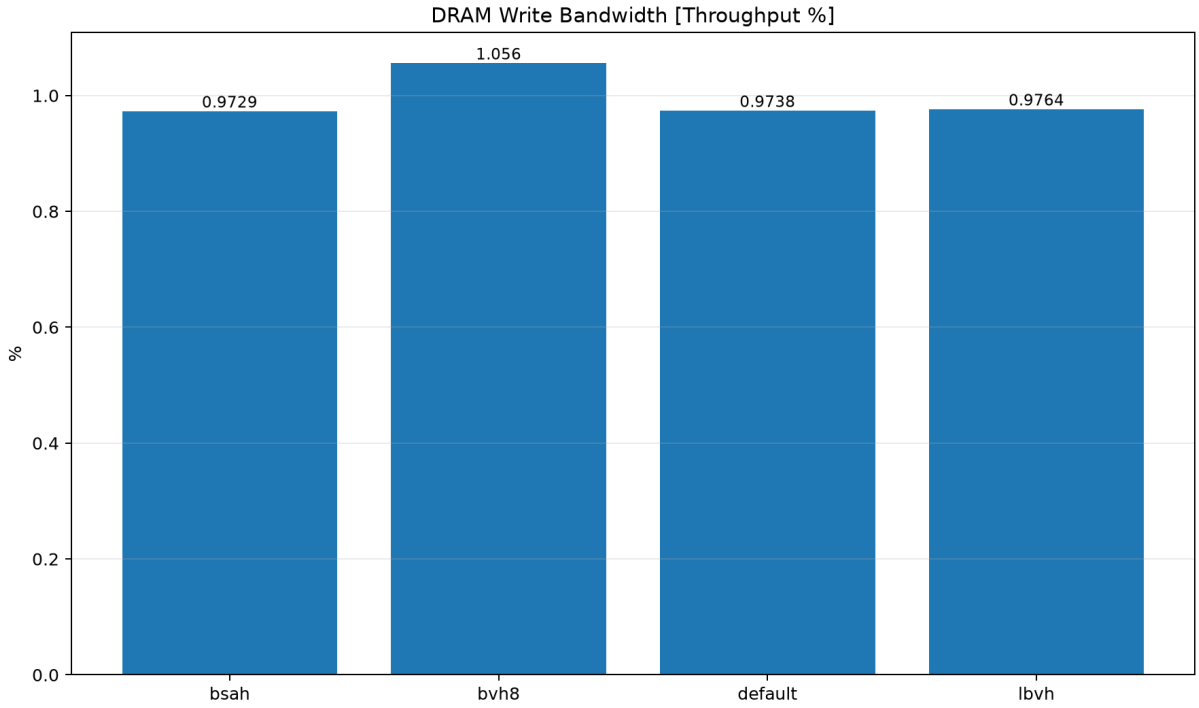
Fonte: Elaborado pelo autor, utilizando Python matplotlib

Também é possível observar diferenças entre as estruturas que utilizam nós binários. A quantidade de memória necessária não depende somente do tamanho individual de cada nó, mas também da quantidade de nós gerados durante a construção. Nesse caso, a forma como cada algoritmo organiza as primitivas pode resultar em árvores com diferentes quantidades de nós e, consequentemente, em diferentes tamanhos finais.

A BSAH apresentou, em alguns casos, estruturas ligeiramente menores que as demais abordagens binárias. Esse comportamento indica que a estratégia utilizada durante a construção pode influenciar não apenas a qualidade da hierarquia para a travessia, mas também a quantidade de memória necessária para armazená-la. Embora as diferenças sejam menores quando comparadas ao impacto observado na BVH8, elas demonstram que a organização da árvore também possui influência sobre o tamanho final da estrutura.

Além do tamanho das estruturas, também foi analisado o comportamento da memória RAM durante a execução. A utilização da banda de leitura não apresentou diferenças relevantes que permitissem associar diretamente o comportamento observado às diferentes estruturas BVHs. Por outro lado, a banda de escrita apresentou diferenças mais evidentes entre as abordagens, principalmente quando considerada em conjunto com a quantidade de memória necessária para armazenar as estruturas, conforme apresentado na Figura 19.

Figura 19 – Uso da banda de escrita da RAM

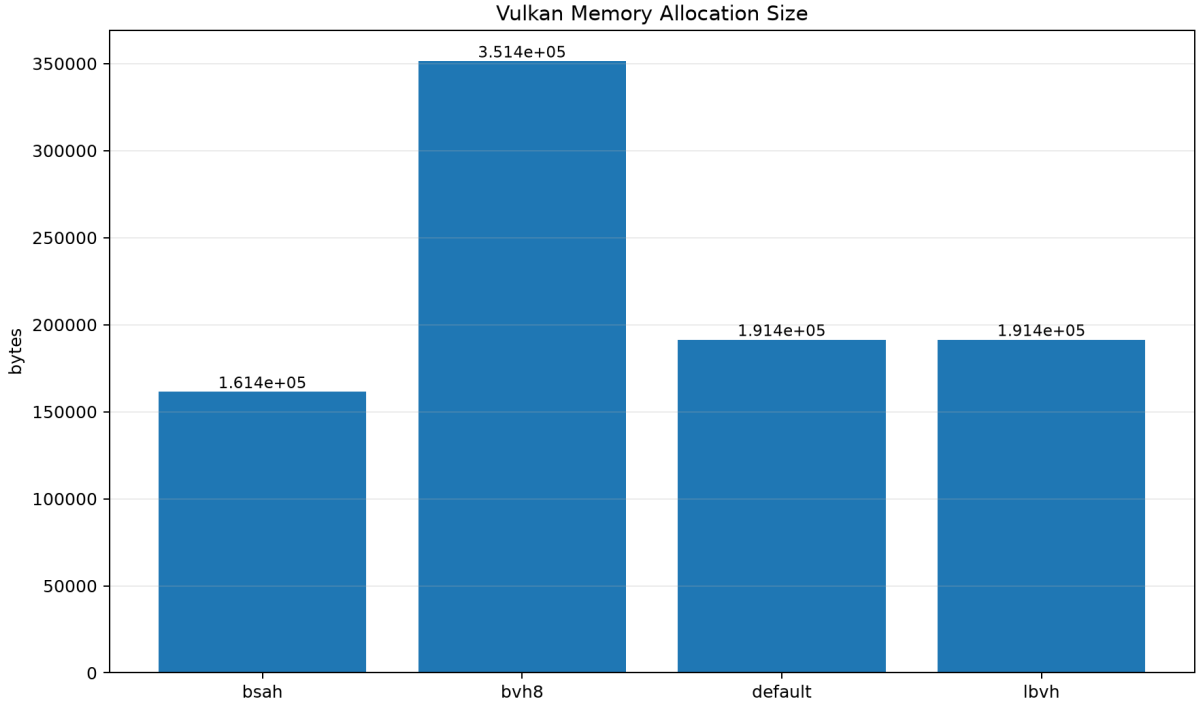


Fonte: Elaborado pelo autor, utilizando Python matplotlib

A BVH8 apresentou uma utilização da banda de escrita consideravelmente superior à das demais abordagens. Esse comportamento é compatível com o maior volume de dados necessário para armazenar e construir seus nós. Como a estrutura possui nós maiores, a construção e a transferência desses dados podem resultar em uma quantidade maior de operações de escrita na memória.

A diferença também pode ser observada na quantidade de memória alocada durante a execução da aplicação. Nesse caso, a BVH8 apresentou novamente o maior consumo, enquanto a BSAH apresentou um comportamento mais favorável em relação às demais estruturas avaliadas, conforme ilustrado na Figura 20.

Figura 20 – Tamanho das alocações de memória realizadas pelo Vulkan



Fonte: Elaborado pelo autor, utilizando Python matplotlib

Os resultados indicam, portanto, que existe um compromisso entre a organização da estrutura e o consumo de memória. A BVH8 apresentou vantagens em relação ao tempo de construção e, conforme observado anteriormente, também apresentou bom desempenho na duração dos quadros. Entretanto, essas vantagens são acompanhadas por um aumento significativo no espaço necessário para armazenar a estrutura e por uma maior atividade de escrita na memória.

Por outro lado, a BSAH apresentou um comportamento mais favorável em relação ao consumo de memória, além de apresentar bons resultados de desempenho durante a renderi-

zação. Esse comportamento reforça que a avaliação de uma estrutura BVH não deve considerar apenas o tempo de construção ou de travessia, mas também os recursos necessários para armazenar e manipular a estrutura durante sua utilização.

5.2.2 CPU

5.2.2.1 BLAS

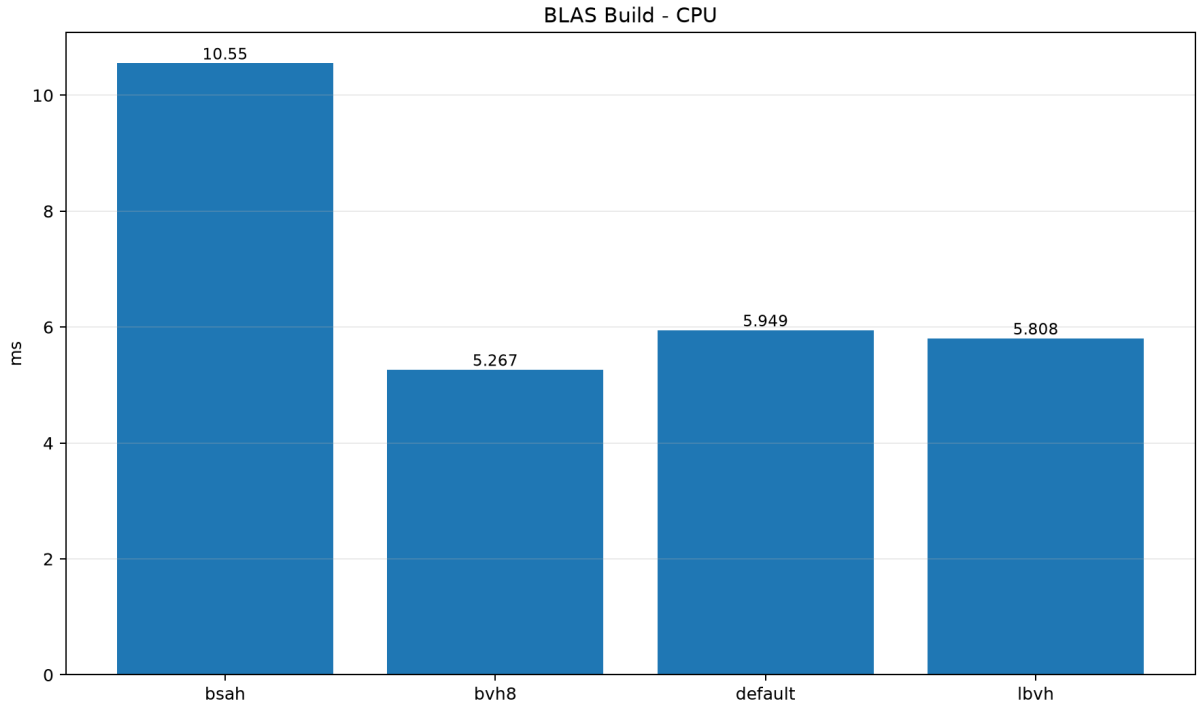
Inicialmente, são analisadas as métricas relacionadas ao tempo de construção das BLASs. Diferentemente da TLAS, cuja quantidade de instâncias permanece relativamente constante durante os experimentos, as BLASs podem representar geometrias com quantidades bastante diferentes de primitivas. Nesse caso, são consideradas estruturas que representam desde geometrias com poucos vértices até modelos com mais de mil vértices. Essa variação na complexidade das geometrias contribui para uma maior variação nos tempos de construção e, consequentemente, influencia o valor médio observado para cada abordagem.

A Figura 21 apresenta os tempos de construção das BLASs para as diferentes estruturas avaliadas. Observa-se que a BSAH apresentou o maior tempo de construção, com um valor superior ao dobro do observado para as demais abordagens. Esse comportamento está relacionado à maior complexidade do processo de construção utilizado pela BSAH, que realiza uma análise das possíveis divisões para buscar uma organização mais adequada da hierarquia. Dessa forma, o maior tempo de construção representa o custo adicional associado à busca por uma estrutura potencialmente mais eficiente durante a travessia.

A BVH8, por outro lado, apresentou o menor tempo de construção, sendo aproximadamente 40% mais rápida que as demais abordagens avaliadas. Uma possível explicação para esse comportamento está na quantidade de filhos presentes em cada nó da estrutura. Como a BVH8 possui uma maior quantidade de filhos por nó, a árvore tende a apresentar menor profundidade em comparação com uma estrutura binária equivalente. Essa característica pode reduzir a quantidade de níveis que precisam ser processados durante a construção da hierarquia.

A LBVH e a BVH binária apresentaram tempos de construção semelhantes. No caso da LBVH, parte do custo está associada à geração dos códigos de Morton e à ordenação das primitivas antes da construção da hierarquia. Embora essa abordagem utilize operações paralelas durante diferentes etapas da construção, essas operações também possuem um custo associado à sua execução. Dessa forma, o ganho obtido pelo paralelismo não resultou em uma diferença significativa em relação à BVH binária nos cenários avaliados.

Figura 21 – Tempo de construção das BLASs



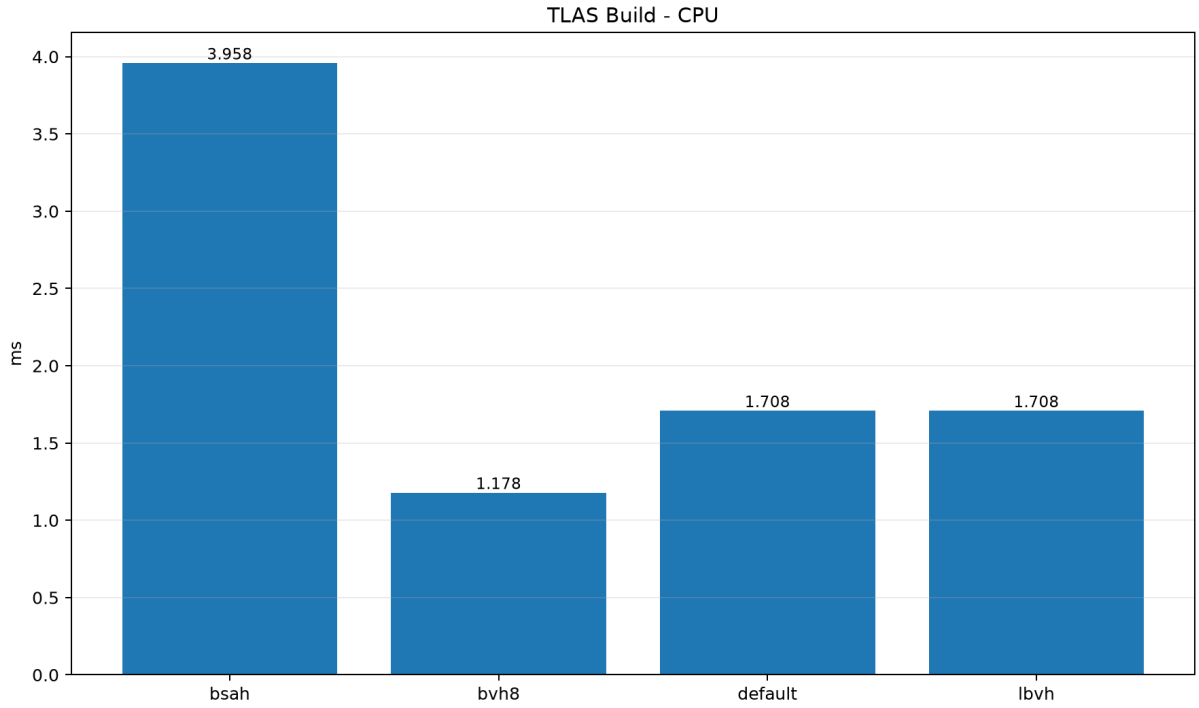
Fonte: Elaborado pelo autor, utilizando Python matplotlib

5.2.2.2 TLAS

Na construção das TLASs, foi observado um comportamento semelhante entre as diferentes abordagens. A principal diferença está nos valores médios dos tempos obtidos para cada estrutura. Como a quantidade de instâncias utilizada durante os experimentos permanece constante, a comparação dos tempos de construção da TLAS apresenta uma variação menor em relação à observada na construção das BLASs.

Essa característica torna a média dos tempos de construção da TLAS uma representação mais direta do comportamento da operação durante os experimentos, uma vez que a quantidade de elementos processados permanece aproximadamente constante entre as diferentes execuções. Dessa forma, as diferenças observadas entre as abordagens estão mais relacionadas ao próprio algoritmo utilizado para construir a estrutura do que à variação na quantidade de instâncias processadas. Os resultados dessa comparação são apresentados na Figura 22.

Figura 22 – Tempo de construção das TLASs



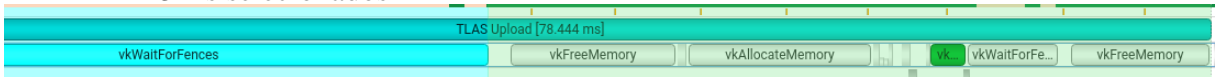
Fonte: Elaborado pelo autor, utilizando Python matplotlib

5.2.2.3 Tempo de *upload*

O gráfico referente ao tempo de *upload* das estruturas não será utilizado na comparação direta entre as abordagens. Isso ocorre porque o tempo registrado nessa operação não representa exclusivamente o tempo necessário para transferir os dados da CPU para a GPU. Parte significativa do intervalo registrado está relacionada a operações de sincronização entre a CPU e a GPU.

Em uma execução normal, por exemplo, a CPU pode finalizar a construção da TLAS e, posteriormente, aguardar a conclusão de operações da GPU. Nesse intervalo, a aplicação pode permanecer bloqueada em uma chamada *vkWaitForFences*. Como consequência, o tempo registrado para a operação de *upload* pode variar aproximadamente entre 20 ms e 90 ms, mesmo quando a parcela efetivamente relacionada ao processamento da estrutura permanece inferior a aproximadamente 3 ms.

Figura 23 – *Nsight Systems*: tempo registrado para o *upload* da TLAS, com aproximadamente 3 ms selecionados



Fonte: Elaborado pelo autor, utilizando *Nsight Systems*

É possível identificar e filtrar algumas dessas operações de espera para tentar obter

uma estimativa mais próxima do tempo efetivamente utilizado pelo *upload*, conforme exemplificado na Figura 23. Entretanto, durante a execução da aplicação existem outras chamadas *vkWaitForFences* que não estão diretamente relacionadas à transferência das estruturas BVHs. Dessa forma, seria necessário identificar com precisão quais comandos de sincronização estão associados a cada operação de *upload* e separar esses intervalos dos demais eventos presentes na execução.

Esse procedimento aumentaria significativamente a complexidade do tratamento dos dados e, devido às limitações de tempo para a realização dos experimentos, não foi possível estabelecer uma separação suficientemente precisa para utilizar essa métrica na comparação. Por esse motivo, o tempo de *upload* é apresentado apenas como uma observação sobre o comportamento da aplicação, não sendo utilizado como uma métrica principal para determinar o desempenho das diferentes estruturas.

5.2.3 Duração do quadro

Observando as diferenças entre as estruturas BVHs, é possível analisar como cada abordagem influenciou o tempo necessário para a geração dos quadros. Como a aplicação foi executada sem uma limitação fixa de *frames*, a duração de cada quadro é determinada principalmente pelo tempo necessário para que a GPU conclua as operações de renderização e esteja disponível para iniciar o próximo quadro.

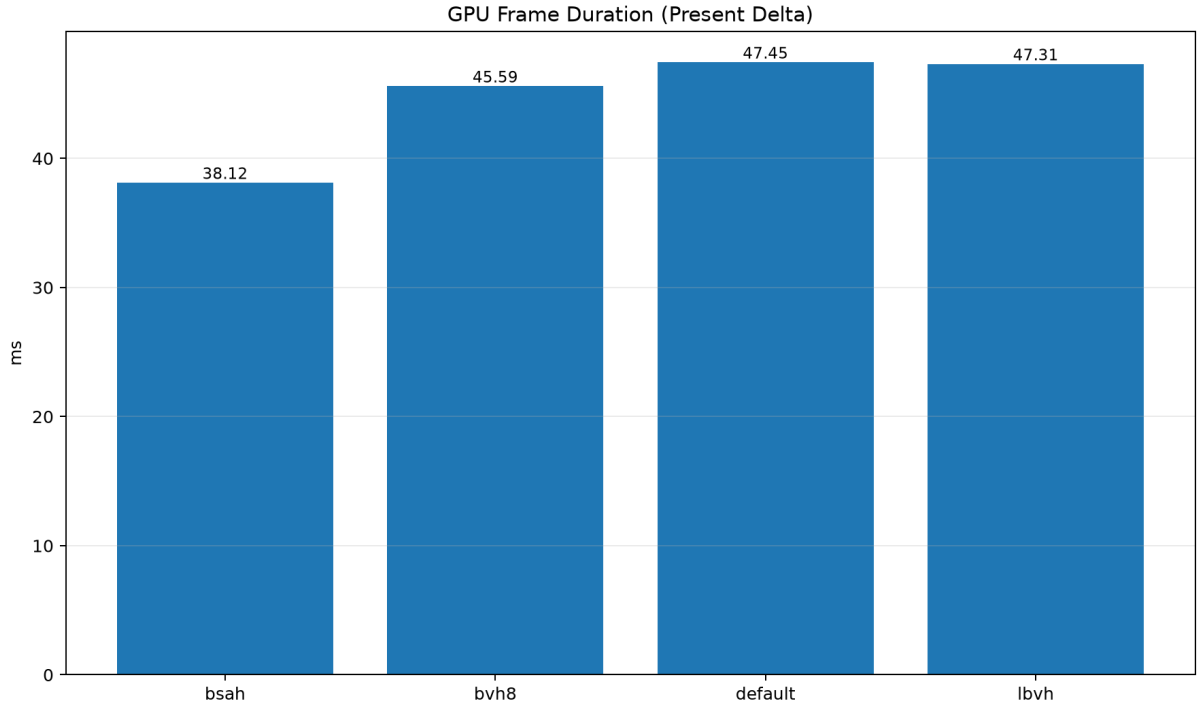
A partir dos resultados apresentados na Figura 24, observa-se uma vantagem da BSAH em relação às demais estruturas. Esse resultado indica que o maior custo associado à construção da estrutura pode ser compensado durante a etapa de renderização, quando a qualidade da hierarquia contribui para reduzir o custo da travessia da BVH. Dessa forma, investir mais recursos da CPU durante a construção da estrutura pode resultar em uma redução do tempo de execução na GPU.

Entretanto, esse comportamento depende da relação entre o custo adicional da construção e o ganho obtido durante a renderização. O investimento realizado durante a construção somente é vantajoso quando a redução no tempo de processamento da GPU é suficiente para compensar esse custo adicional. Além disso, para aplicações em tempo real, a estrutura precisa estar disponível no momento em que a GPU necessita utilizá-la. Caso o tempo necessário para sua construção seja muito elevado, o ganho obtido durante a renderização pode não compensar o atraso introduzido pela construção da estrutura.

A BVH8 também apresentou um desempenho ligeiramente superior ao das demais abordagens em relação à duração dos quadros. Esse comportamento indica que a utilização de uma estrutura com maior quantidade de filhos por nó pode apresentar vantagens durante a travessia, dependendo das características da cena e da distribuição das primitivas. A partir desse resultado, seria interessante avaliar outras configurações de largura da árvore, como BVH4 e BVH16, permitindo verificar se existe uma configuração intermediária ou superior que apresente um melhor equilíbrio entre o custo de construção e o custo de travessia.

Outra possibilidade seria utilizar uma heurística capaz de selecionar a quantidade de filhos por nó de acordo com as características da cena. Dessa forma, diferentes geometrias ou cenários poderiam utilizar estruturas com diferentes larguras, buscando adaptar a organização da árvore ao tipo de carga de trabalho.

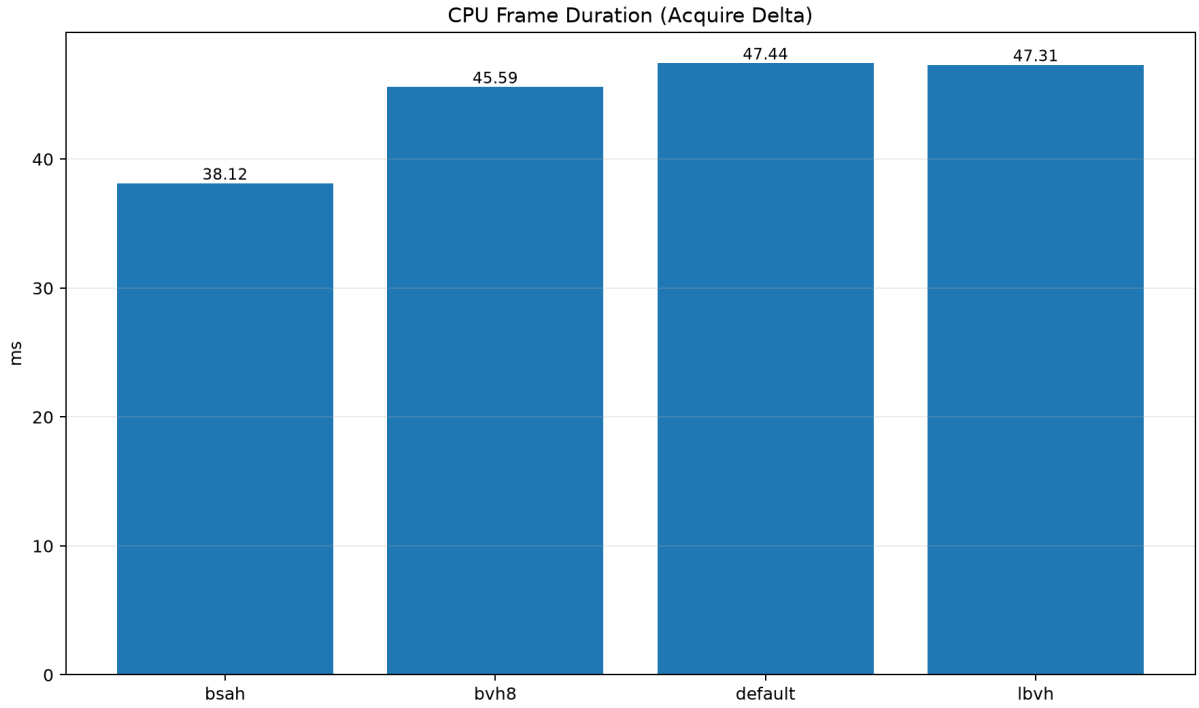
Figura 24 – Tempo médio de produção do quadro na GPU



Fonte: Elaborado pelo autor, utilizando Python matplotlib

Em relação à duração do quadro na CPU, a aplicação não permite que a CPU avance mais de um quadro em relação à GPU. Dessa forma, após iniciar as operações necessárias para um quadro, a CPU pode permanecer aguardando a conclusão das operações correspondentes na GPU antes de iniciar o processamento do próximo quadro. Essa característica da execução faz com que o comportamento observado na CPU seja semelhante ao apresentado na GPU, conforme ilustrado na Figura 25.

Figura 25 – Tempo médio de produção do quadro na CPU



Fonte: Elaborado pelo autor, utilizando Python matplotlib

Durante testes realizados anteriormente, também foi observado que a quantidade de instâncias presentes na cena possui influência significativa sobre a duração dos quadros. Em cenários com menos de 10 instâncias, foi possível obter tempos inferiores a 5 ms por quadro. Esse resultado indica que, além da estrutura utilizada, a quantidade e a complexidade das instâncias presentes na cena possuem influência direta sobre o tempo necessário para a geração de cada quadro.

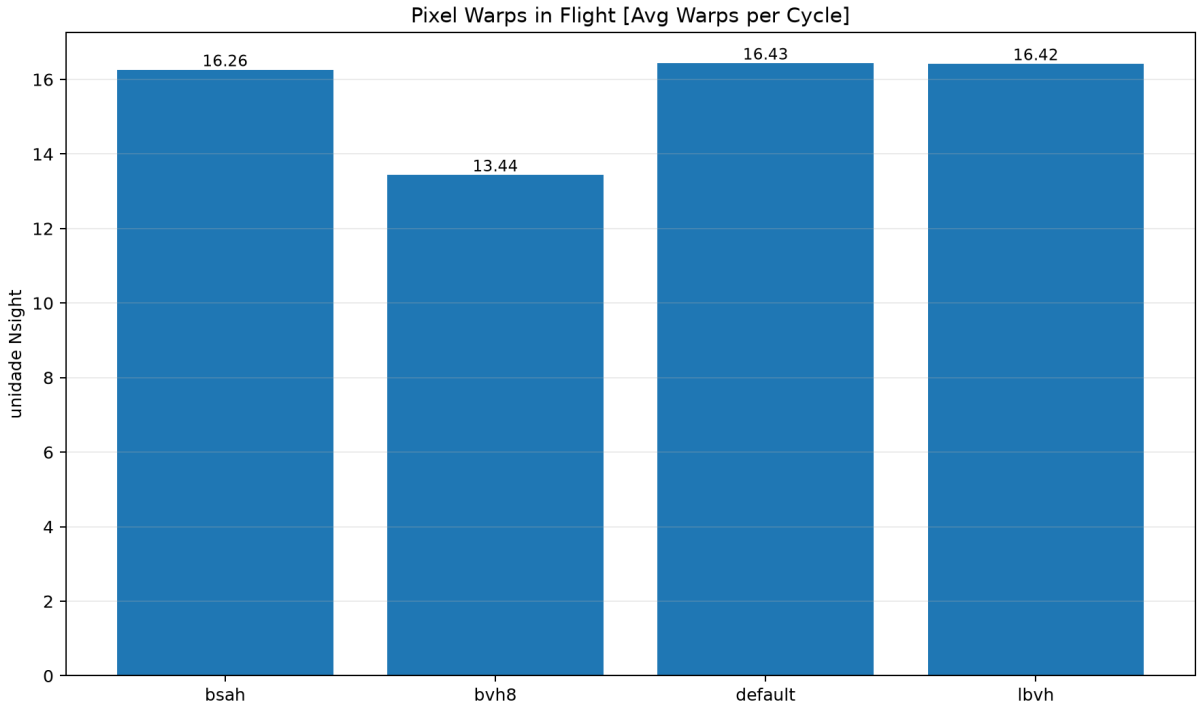
5.2.4 Ocupação dos SMs e dos *warps*

5.2.4.1 *Warps*

Uma forma de analisar o comportamento dos SMs durante a execução é observar a quantidade de *warps* de pixels. Nesse contexto, pode ser analisado o número médio de *warps* por ciclo. Essa informação permite relacionar a quantidade de *warps* ativos aos ciclos de execução da GPU. Também é possível analisar essa atividade como um percentual de *throughput*, permitindo observar a utilização desse recurso em relação à capacidade considerada pela métrica (NVIDIA, 2026b).

A BVH8 apresentou um nível de atividade consideravelmente menor quando comparada às demais abordagens, conforme observado nas Figuras 26 e 27. Entretanto, apesar dessa

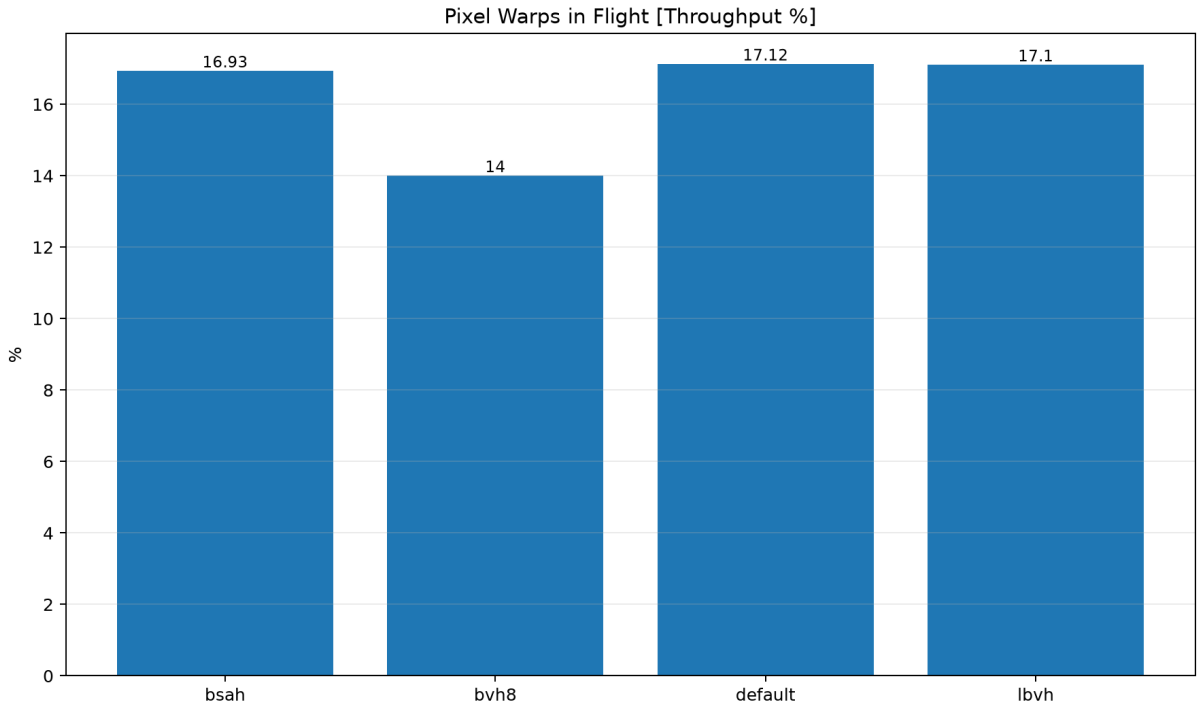
Figura 26 – Quantidade média de *warps* de pixels simultaneamente ativos por ciclo



Fonte: Elaborado pelo autor, utilizando Python matplotlib

menor atividade dos *warps*, a BVH8 ainda apresentou uma duração de quadro inferior à observada para a BVH binária tradicional e para a LBVH. Esse resultado indica que uma menor quantidade de *warps* ativos, isoladamente, não implica necessariamente um pior desempenho. Outros fatores relacionados à organização da estrutura e ao trabalho realizado durante a travessia também podem influenciar o tempo total de execução.

Figura 27 – Quantidade média de *warps* de pixels simultaneamente ativos por ciclo, expressa em percentual



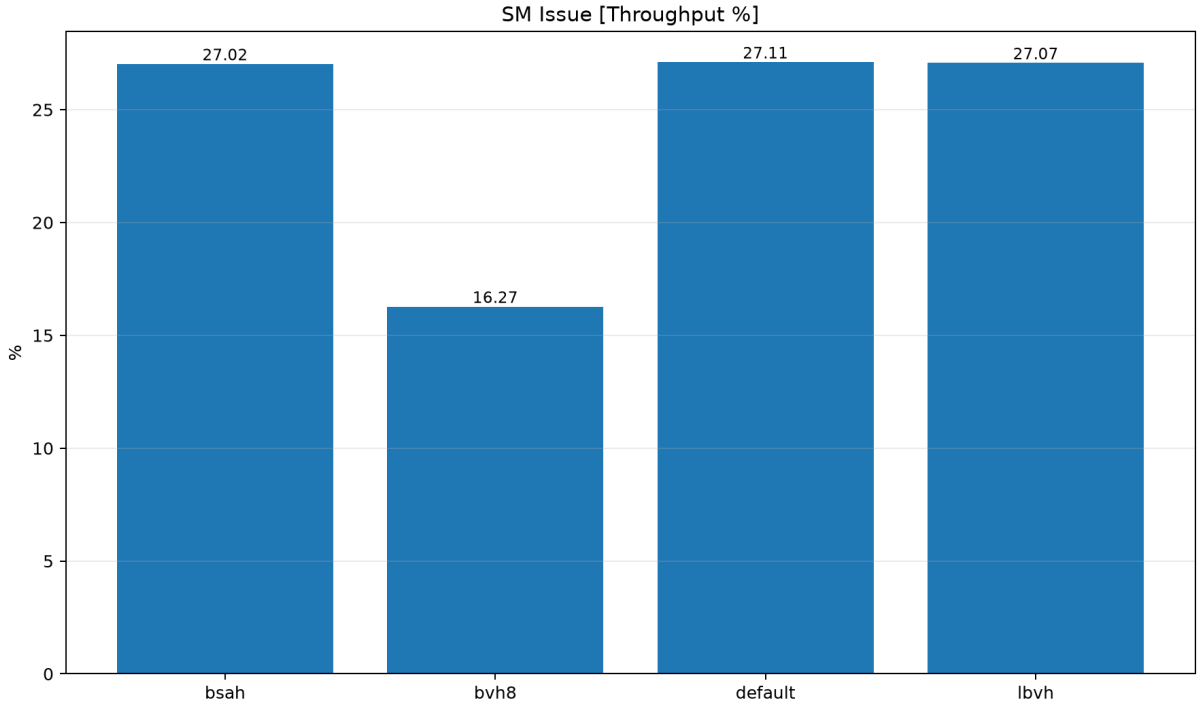
Fonte: Elaborado pelo autor, utilizando Python matplotlib

5.2.4.2 SMs

Para compreender melhor a ocupação da GPU, também podem ser analisadas métricas relacionadas ao agendamento e à emissão de operações pelos SMs.

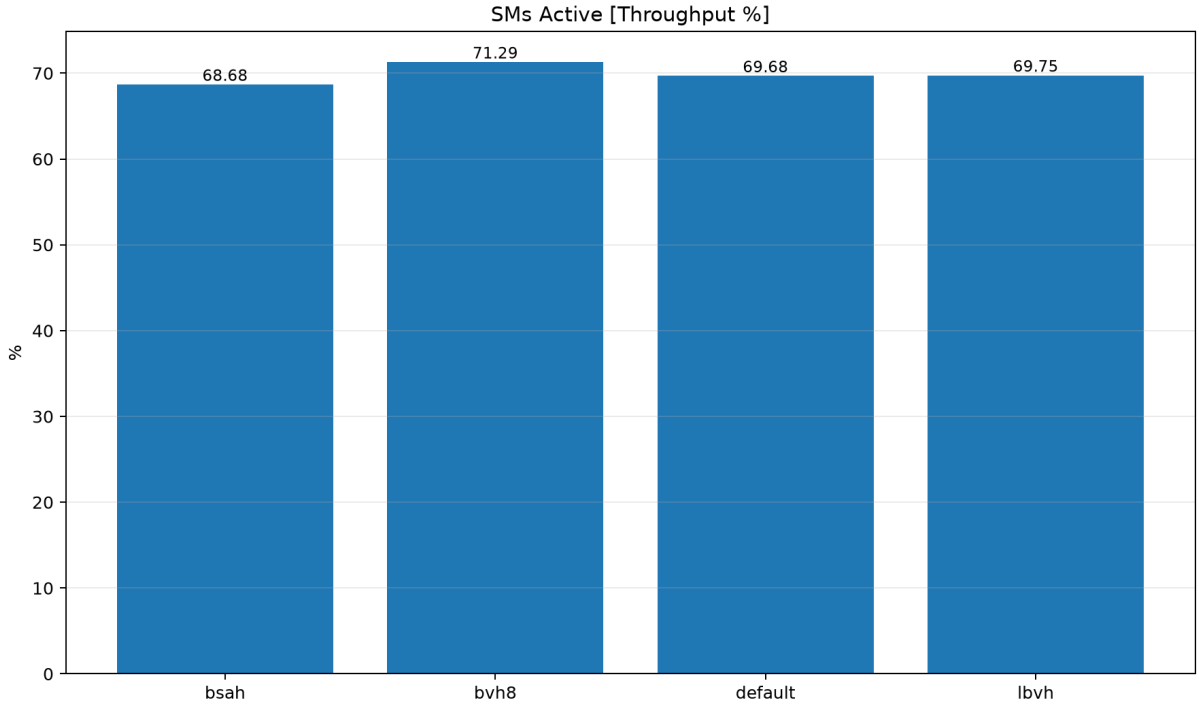
Os resultados apresentados nas Figuras 28 e 29 indicam que a carga de trabalho dos SMs permaneceu relativamente semelhante entre as diferentes abordagens. A BVH8 apresentou o comportamento menos favorável entre as estruturas avaliadas, com uma utilização ligeiramente inferior dos recursos analisados. Entretanto, essa diferença deve ser interpretada em conjunto com as demais métricas de desempenho, uma vez que a utilização isolada dos SMs não determina o tempo total necessário para a produção de um quadro.

Figura 28 – Percentual médio de emissão de instruções dos SMs utilizado durante a execução



Fonte: Elaborado pelo autor, utilizando Python matplotlib

Figura 29 – Percentual de ocupação efetiva dos SMs da GPU



Fonte: Elaborado pelo autor, utilizando Python matplotlib

5.2.5 Consumo de energia e temperatura

A temperatura da GPU permaneceu próxima de 60 °C durante os experimentos. Entretanto, como os testes não foram realizados em um ambiente com temperatura e condições térmicas controladas, foram observadas variações mesmo quando a mesma estrutura BVH era utilizada. Em alguns casos, a temperatura média apresentou uma diferença próxima de 2 °C entre execuções da mesma estrutura. Dessa forma, a precisão obtida não é suficiente para utilizar a temperatura como um critério confiável para diferenciar as estruturas avaliadas.

Ainda assim, é possível observar uma tendência esperada: quanto menor for a utilização dos recursos da GPU para produzir um quadro, menor tende a ser a quantidade de calor gerada durante a execução. Entretanto, essa relação também depende de fatores externos à aplicação, como as condições térmicas do ambiente e o comportamento do sistema de refrigeração. Por esse motivo, os valores de temperatura são apresentados apenas como uma observação complementar dos experimentos, não sendo utilizados para estabelecer diferenças de desempenho entre as estruturas.

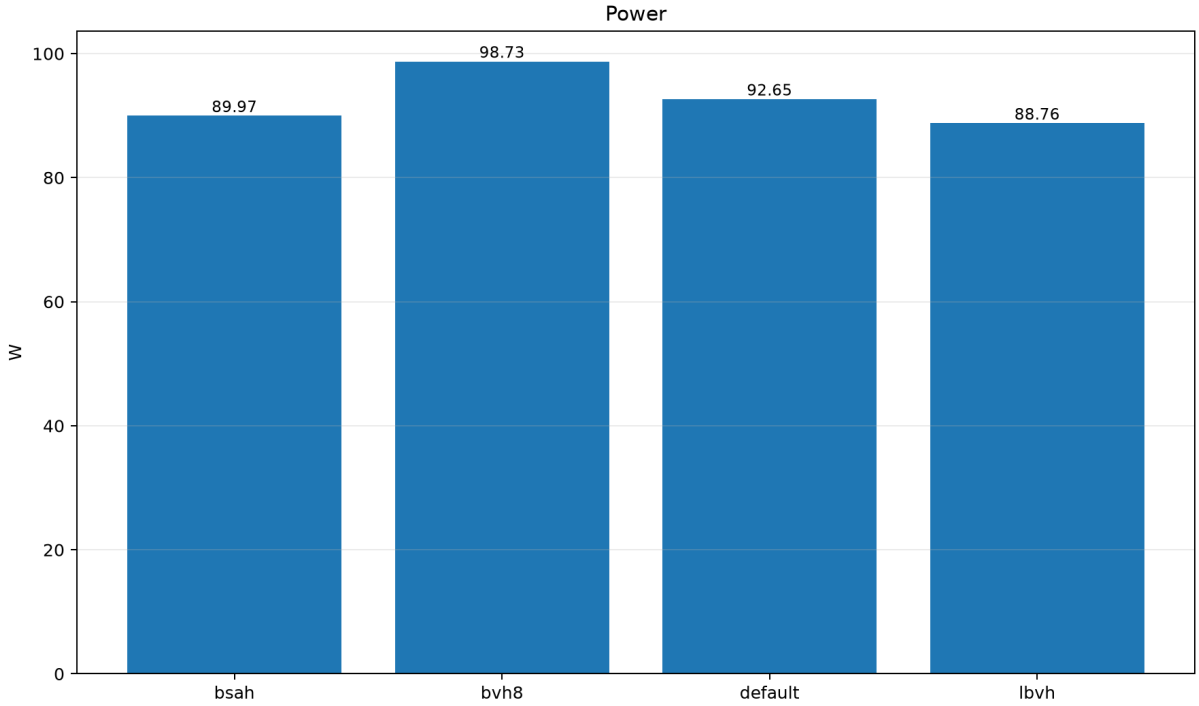
A análise do consumo energético apresenta uma limitação semelhante, mas com uma diferença importante. A comparação da potência média não considera diretamente a quantidade de trabalho realizada durante o intervalo analisado. Dessa forma, uma estrutura que consome uma potência maior, mas consegue produzir os quadros mais rapidamente, pode ser mais eficiente energeticamente do que outra que consome menos potência, mas permanece executando por mais tempo.

No presente experimento, a comparação da potência é realizada de forma independente da quantidade de quadros produzidos ou do tempo necessário para produzi-los. Em outras palavras, nesse critério, é atribuído melhor desempenho à estrutura que apresenta menor consumo médio, mesmo que ela possa necessitar de mais tempo para produzir o mesmo número de quadros. Portanto, essa métrica deve ser interpretada como uma comparação do consumo de potência durante a execução, e não como uma medida direta da eficiência energética da aplicação.

Observando os resultados apresentados na Figura 30, a BVH8 apresentou um consumo médio consideravelmente superior ao das demais abordagens. Esse comportamento está de acordo com os resultados observados anteriormente, principalmente em relação ao maior tamanho de suas estruturas e à maior utilização dos recursos de memória.

Entre as demais estruturas, as diferenças de consumo foram menores, com uma

Figura 30 – Consumo médio de potência da GPU



Fonte: Elaborado pelo autor, utilizando Python matplotlib

variação inferior a 4 W entre o maior e o menor valor observado. Como a metodologia utilizada não considera o tempo necessário para produzir os quadros durante a análise do consumo de potência, essa diferença não é suficiente para estabelecer uma conclusão significativa sobre qual dessas estruturas apresenta melhor eficiência energética.

Dessa forma, os resultados de consumo de energia devem ser interpretados com cautela. A BVH8 apresentou uma diferença suficientemente evidente para indicar um maior consumo de potência durante os experimentos, enquanto as diferenças entre a BVH, a LBVH e a BSAH foram pequenas em relação às limitações do método de medição. Para uma análise mais precisa da eficiência energética, seria necessário considerar conjuntamente a potência consumida, o tempo de execução e a quantidade de quadros produzidos, permitindo avaliar o consumo de energia necessário para realizar uma quantidade equivalente de trabalho.

5.2.6 *Shaders* e ocupação dos *warps*

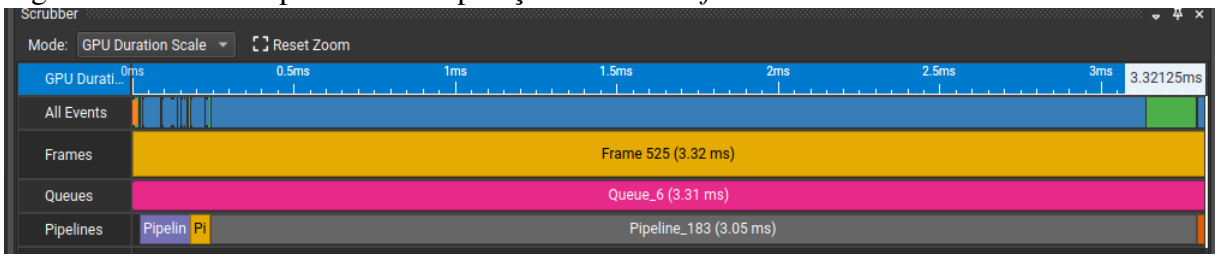
Deixando de lado as métricas obtidas pelo *Nsight Systems*, é possível analisar informações mais detalhadas sobre a execução dos *shaders* por meio dos dados coletados pelo *Nsight Graphics*. A ferramenta disponibiliza uma quantidade elevada de métricas e, em muitos casos, algumas delas apresentam informações semelhantes ou complementares. Por esse mo-

tivo, foram selecionadas apenas as métricas consideradas mais relevantes para a análise deste trabalho. Os valores apresentados nesta seção correspondem ao quadro analisado no instante de 20 segundos de execução.

Inicialmente, pode-se observar a distribuição das operações realizadas durante a execução de um *frame*. Na linha correspondente aos *pipelines*, são apresentados quatro *pipelines* diferentes. Os dois primeiros são responsáveis pela geração dos dois *G-buffers* utilizados pela aplicação. O terceiro *pipeline* corresponde à etapa em que são realizados os testes de interseção utilizando as estruturas BVHs. Essa etapa representa a maior parte do processamento do quadro, ocupando mais de 90% do tempo apresentado na visualização. O quarto *pipeline* corresponde a um algoritmo de processamento rápido responsável por realizar alguns ajustes na imagem.

É importante destacar que o tempo apresentado nessa visualização não corresponde diretamente ao tempo real de execução utilizado nos demais experimentos. O *Nsight Graphics* utiliza uma representação própria para a visualização temporal das operações durante a análise do *frame*. Dessa forma, a figura é utilizada principalmente para observar a distribuição relativa das operações e identificar quais etapas concentram a maior parte da execução, conforme apresentado na Figura 31.

Figura 31 – Linhas representando operações durante o *frame*



Fonte: Elaborado pelo autor, utilizando *Nsight Graphics*

A Tabela 1 apresenta algumas das métricas relacionadas à execução dos *shaders*, à atividade dos *threads* e à ocupação dos *warps*. Essas métricas permitem observar não apenas o tempo de execução, mas também como os *threads* que executam os *shaders* permanecem ativos durante o processamento.

A métrica *Active Threads (Th/Warp)* representa a quantidade média de *threads* ativos em cada *warp*. Como um *warp* é composto por 32 *threads*, valores próximos de 32 indicam que uma grande parte dos *threads* está ativa durante a execução. Na tabela, os valores variam entre 29,6 e 30,6 *threads* por *warp*, indicando uma alta proporção de *threads* ativos para todas as estruturas avaliadas.

A métrica *Active Threads (Coherence)* apresenta essa atividade na forma de um percentual de coerência. Nesse caso, valores maiores indicam que uma parcela maior dos *threads* do *warp* permanece ativa simultaneamente. A BSAH apresentou o maior valor, com 95,5%, enquanto a LBVH apresentou o menor, com 92,4%. Essa métrica possui relação com a *warp divergence*, uma vez que uma maior divergência pode resultar em uma menor quantidade de *threads* ativos simultaneamente.

As métricas *Predicated-On (Th/Warp)* e *Predicated-On (Coherence)* estão relacionadas aos *threads* que permanecem habilitados pela predicação durante a execução. A primeira apresenta a quantidade média de *threads* que estão com a predicação habilitada em cada *warp*, enquanto a segunda apresenta essa informação como um percentual. Essas métricas permitem observar quanto do trabalho dos *warps* permanece efetivamente habilitado após a aplicação das condições de predicação.

Os valores apresentados mostram um comportamento semelhante ao observado nas métricas de *threads* ativos. A BSAH apresenta novamente o maior valor, com 28,6 *threads* por *warp* e 89,3% de coerência, enquanto a BVH8 apresenta 27,6 *threads* por *warp* e 86,1%. Essa diferença indica uma maior quantidade de *threads* habilitados pela predicação no cenário da BSAH.

A métrica *Unallocated Warps (Avg Warps)* representa a quantidade média de *warps* que poderiam estar residentes nos SMs, mas que permanecem não alocados durante a execução. A métrica *Unallocated Warps (Occupancy)* apresenta essa mesma informação como um percentual em relação à capacidade de ocupação considerada pela GPU. A NVIDIA disponibiliza essa métrica como parte das informações de ocupação dos SMs, juntamente com métricas como *SM Active*, *SM Issue* e *Total SM Occupancy* (NVIDIA, 2026b).

Um valor elevado de *Unallocated Warps* indica que existe capacidade de execução nos SMs que não está sendo ocupada por *warps* residentes naquele momento. Entretanto, essa informação não significa, isoladamente, que exista um problema específico de memória ou que a aplicação esteja deixando de utilizar corretamente a memória compartilhada. A quantidade de *warps* que pode permanecer residente depende de diferentes recursos da GPU, incluindo recursos utilizados pelo *shader*, como registradores e memória compartilhada, além da quantidade de trabalho disponível para execução. Portanto, essa métrica deve ser utilizada como um indicador da capacidade de ocupação não utilizada.

Os resultados apresentam valores relativamente próximos entre as abordagens. A

BVH8 apresentou a maior quantidade média de *warps* não alocados, com 23,4 *warps*, correspondendo a 48,7% de ocupação, enquanto a LBVH apresentou os menores valores, com 22,4 *warps* e 46,8%. Isso indica que, no *frame* analisado, a BVH8 deixou uma quantidade ligeiramente maior da capacidade de ocupação dos SMs sem alocação.

Por fim, as métricas *Pixel Shader (Avg Warps)* e *Pixel Shader (Occupancy)* estão relacionadas especificamente aos *warps* utilizados na execução dos *pixel shaders*. A primeira representa a quantidade média de *warps* de pixels em execução, enquanto a segunda apresenta essa utilização como um percentual de ocupação.

Nesse caso, a BVH8 apresenta um comportamento diferente das demais estruturas. Enquanto a BVH, a LBVH e a BSAH apresentam valores próximos, entre 13,0 e 13,5 *warps* em média, a BVH8 apresenta apenas 8,7 *warps*. Consequentemente, sua ocupação de *pixel shaders* também é menor, correspondendo a 18,0%, enquanto as demais estruturas apresentam valores próximos de 27% a 28%, conforme sintetizado na Tabela 1.

Tabela 1 – Comparação de desempenho e ocupação da GPU entre as abordagens

Métrica	**default**	**lbvh**	**bvh8**	**bsah**
Frame	441	440	455	526
Active Threads (Th/Warp)	29.7	29.6	29.9	30.6
Active Threads (Coherence)	92.9%	92.4%	93.5%	95.5%
Predicated-On (Th/Warp)	27.8	27.7	27.6	28.6
Predicated-On (Coherence)	86.9%	86.4%	86.1%	89.3%
Unallocated Warps (Avg Warps)	22.7	22.4	23.4	23.0
Unallocated Warps (Occupancy)	47.3%	46.8%	48.7%	47.8%
Pixel Shader (Avg Warps)	13.4	13.0	8.7	13.5
Pixel Shader (Occupancy)	27.8%	27.1%	18.0%	28.2%

Fonte: Elaborado pelo autor

5.3 CONCLUSÕES

Este trabalho apresentou uma comparação entre diferentes estratégias de construção e organização de estruturas BVHs para utilização em uma aplicação de *ray tracing* em tempo real. Foram avaliadas as estruturas BVH, LBVH, BSAH e BVH8, considerando métricas relacionadas ao tempo de construção, à duração dos quadros, à utilização dos recursos da GPU, ao consumo de memória e ao consumo de energia. Os resultados obtidos permitem observar como diferentes estratégias de construção e organização podem influenciar o desempenho da aplicação, evidenciando que o custo ou qualidade nem sempre representa o principal fator na escolha de uma estrutura.

5.3.1 Desempenho das estruturas

Dentre as estruturas avaliadas, a BSAH apresentou o melhor desempenho durante a renderização. A redução de aproximadamente 10 ms na duração dos quadros representa uma diferença significativa, principalmente considerando que, para atingir uma taxa de 60 quadros por segundo, cada quadro precisa ser produzido em aproximadamente 16,6 ms. Dessa forma, o maior custo de construção da BSAH foi compensado pela melhoria obtida durante a travessia da estrutura e pela consequente redução do tempo necessário para produzir os quadros.

No cenário avaliado, a aplicação possui tempo suficiente entre as etapas de construção e utilização da estrutura para que o custo adicional da BSAH não represente uma limitação significativa. Dessa forma, a utilização de uma estratégia de construção mais custosa mostrou-se vantajosa quando o objetivo principal é reduzir o tempo de renderização. Esse resultado reforça a importância de considerar conjuntamente o custo de construção e o custo de travessia ao avaliar uma estrutura BVH.

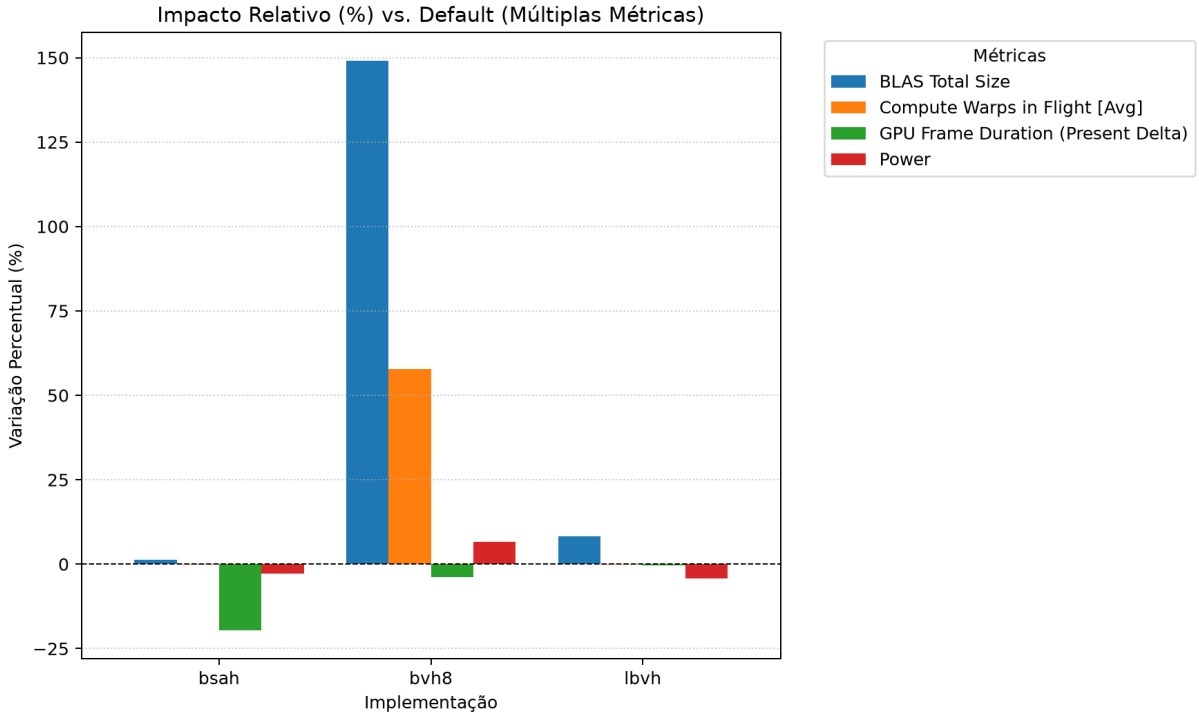
Como o tempo de construção apresentou menor impacto na aplicação quando comparado ao tempo de renderização, a vantagem esperada de uma abordagem com construção mais rápida não foi suficiente para compensar seu desempenho durante a geração dos quadros. Nesse contexto, a LBVH não apresentou uma vantagem significativa em relação às demais estruturas avaliadas. Embora a utilização de códigos de Morton e de operações paralelas permita construir a hierarquia utilizando uma abordagem adequada ao paralelismo, o resultado obtido neste trabalho não demonstrou um ganho suficiente durante a renderização para justificar seu custo de implementação em relação às outras abordagens avaliadas.

A BVH8 também apresentou um resultado relevante. Apesar de utilizar nós com espaço para uma quantidade maior de filhos, a estrutura apresentou uma duração de quadros ligeiramente menor que a BVH binária tradicional. Esse resultado indica que o aumento da largura da árvore pode apresentar benefícios durante a travessia, mesmo acompanhado por um aumento significativo no consumo de memória.

Além disso, a BVH8 apresentou um tempo de construção inferior ao das demais estruturas avaliadas. Esse resultado demonstra que uma estrutura com nós maiores não necessariamente apresenta um custo de construção maior. A redução da profundidade da hierarquia pode contribuir para diminuir a quantidade de níveis processados durante a construção, embora esse comportamento dependa da implementação e das características da cena.

De maneira geral, os resultados mostram que não existe uma única característica

Figura 32 – Comparação das principais métricas em relação à BVH padrão



Fonte: Elaborado pelo autor, utilizando Python matplotlib

capaz de determinar a melhor estrutura para todos os cenários. A BSAH apresentou o melhor resultado em desempenho de renderização, enquanto a BVH8 apresentou uma combinação interessante entre tempo de construção e tempo de renderização, porém com maior consumo de memória. A LBVH, por sua vez, apresentou uma construção baseada em operações paralelas e ordenação espacial, mas não obteve vantagem suficiente no cenário avaliado. Essa comparação conjunta das principais métricas está sintetizada na Figura 32.

Os resultados da LBVH também indicam que a utilização de uma construção paralela não garante, por si só, uma melhoria no desempenho final da aplicação. O custo associado à geração dos códigos de Morton, à ordenação das primitivas e às demais etapas da construção precisa ser considerado em conjunto com o benefício obtido pela estrutura durante a travessia. Outras estratégias de construção paralela de BVHs poderiam apresentar resultados diferentes e constituem uma possibilidade para trabalhos futuros.

Diante dos resultados obtidos, a BSAH apresentou o melhor desempenho entre as estruturas avaliadas nos cenários deste trabalho. Embora tenha apresentado um custo de construção superior, a melhoria obtida durante a travessia compensou esse custo, principalmente considerando que o tempo adicional de construção ainda foi relativamente baixo. Portanto, considerando as condições experimentais utilizadas, a BSAH mostrou-se a alternativa mais ade-

quada quando o principal objetivo é reduzir o tempo de geração dos quadros.

5.3.2 Avaliação da aplicação desenvolvida

A aplicação desenvolvida foi capaz de realizar a renderização com uma representação física adequada das sombras e manter uma média próxima de 30 quadros por segundo nos cenários avaliados. Entretanto, quando comparada a outras técnicas utilizadas para geração de sombras em aplicações de grande relevância, principalmente em jogos eletrônicos lançados anteriormente a 2018, observa-se que outras abordagens já eram capazes de atingir taxas de quadros mais elevadas e estáveis, embora com diferentes níveis de aproximação em relação ao comportamento físico da iluminação.

É importante considerar que a implementação apresentada neste trabalho realiza algumas operações adicionais em relação a uma implementação básica de sombras. A utilização das estruturas BVHs, a construção das hierarquias e a realização da travessia para determinar as interseções dos raios introduzem custos adicionais durante a execução. Dessa forma, apesar de o método permitir uma representação mais próxima do comportamento físico das sombras, o custo computacional necessário pode ser elevado quando comparado a técnicas tradicionais utilizadas em aplicações em tempo real, principalmente em sistemas que não possuem unidades de hardware dedicadas ao processamento de *ray tracing*.

Os resultados obtidos também mostram que a qualidade da estrutura de aceleração possui influência sobre esse custo. As diferenças observadas entre as estruturas avaliadas demonstram que a escolha da BVH pode alterar tanto o tempo necessário para sua construção quanto o desempenho durante a renderização. Portanto, a utilização de *ray tracing* em tempo real não depende apenas da capacidade de realizar as interseções entre os raios e a geometria, mas também da escolha e da organização da estrutura utilizada para acelerar esse processo.

Apesar das limitações observadas na aplicação desenvolvida, o código produzido não se restringe ao cenário utilizado nos experimentos. As estruturas BVHs implementadas podem ser adaptadas para outras aplicações que necessitem de consultas espaciais ou de interseções entre raios e geometria. Também é possível realizar a poda da estrutura, removendo determinados nós ou primitivas para reduzir a quantidade de elementos que precisam ser percorridos. Essa abordagem reduz a precisão da estrutura, pois algumas interseções podem deixar de ser consideradas, mas pode proporcionar uma redução significativa no custo da travessia. Dessa forma, uma BVH podada pode ser utilizada em aplicações de tempo real que priorizem

o desempenho em relação à precisão, como testes de colisão executados na CPU, quando não é necessário obter uma detecção completamente precisa em todos os casos. Da mesma forma, parte das implementações pode ser utilizada em aplicações executadas na CPU, desde que sejam realizadas as adaptações necessárias para o ambiente de execução.

Além disso, a aplicação desenvolvida fornece uma base para a realização de novos experimentos com estruturas de aceleração. A possibilidade de configurar diferentes estruturas e comparar seu comportamento permite utilizar a aplicação como ponto de partida para novas avaliações. Os dados obtidos durante este trabalho também fornecem uma referência inicial para comparar novas implementações e identificar como diferentes estratégias de construção podem influenciar o desempenho, o consumo de memória e a utilização dos recursos computacionais.

Em relação aos trabalhos correlatos apresentados, os resultados deste trabalho complementam as avaliações realizadas por Chung et al. (Chung *et al.*, 2018) e Meister e Bittner (Meister; Bittner, 2022), ao analisar diferentes estruturas BVH em uma aplicação de *ray tracing* em tempo real. Enquanto os trabalhos anteriores investigam o comportamento das estruturas em dispositivos móveis e diferentes configurações de execução em GPU, este trabalho apresenta uma implementação própria das estruturas BVH, LBVH, BSAH e BVH8, permitindo comparar seu comportamento em uma mesma aplicação. Além do desempenho durante a renderização e do tempo de construção, foram consideradas métricas relacionadas ao consumo de memória, à utilização dos recursos da GPU e ao consumo de energia. Dessa forma, o experimento amplia a análise dessas estruturas ao reunir diferentes aspectos de seu comportamento em um cenário de renderização em tempo real.

5.3.3 Trabalhos futuros

Como trabalhos futuros, podem ser avaliadas outras larguras de BVHs, como BVH4 e BVH16, além de estratégias capazes de selecionar a largura da árvore de acordo com as características da cena. Também podem ser investigadas outras técnicas de construção paralela, diferentes heurísticas de divisão e métodos capazes de adaptar a estrutura ao comportamento da aplicação. Essas extensões permitiriam verificar se os resultados observados neste trabalho permanecem consistentes em diferentes cenários e se é possível obter um equilíbrio mais adequado entre custo de construção, custo de travessia e consumo de memória.

Também podem ser investigadas abordagens híbridas, combinando características de diferentes estruturas de aceleração ou utilizando estratégias distintas de construção e traves-

sia de acordo com as características da cena. A utilização de diferentes métodos de travessia também pode ser avaliada, principalmente considerando os resultados observados nas métricas de ocupação dos *warps*. Nesse sentido, uma possível linha de investigação seria buscar formas de aumentar a quantidade de *warps* residentes e efetivamente utilizados pelos SMs, verificando se uma melhor ocupação dos recursos da GPU pode resultar em uma redução adicional no tempo de geração dos quadros.

REFERÊNCIAS

Advanced Micro Devices, Inc. **HIP Documentation**, 2026. Disponível em: https://rocm.docs.amd.com/projects/HIP/en/latest/index.html. Acesso em: 23 abr. 2026.

AKENINE-MÖLLER, Tomas *et al.* **Real-Time Rendering**. 4th. ed. United Kingdom: Taylor & Francis, 2018. 1199 p.

Alexander Overvoorde. **Vulkan Tutorial**, 2023. Disponível em: https://vulkan-tutorial.com/Overview. Acesso em: 23 abr. 2026.

APETREI, Ciprian. Fast and Simple Agglomerative LBVH Construction. *In*: BORGO, Rita; TANG, Wen (Ed.). **Computer Graphics and Visual Computing (CGVC)**. [S.l.]: The Eurographics Association, 2014. DOI: 10.2312/cgvc.20141206.

BARRÉ-BRISEBOIS, Colin *et al.* Hybrid rendering for real-time ray tracing. *In*: HAINES, Eric; AKENINE-MÖLLER, Tomas (Ed.). **Ray Tracing Gems: High-Quality and Real-Time Rendering with DXR and Other APIs**. Berkeley, CA: Apress, 2019. p. 437–473. DOI: 10.1007/978-1-4842-4427-2_25. Disponível em: https://doi.org/10.1007/978-1-4842-4427-2_25. Acesso em: 29 out. 2026.

BENTHIN, Carsten *et al.* Compressed-leaf bounding volume hierarchies. *In*: **Proceedings of the Conference on High-Performance Graphics**. New York, NY, USA: Association for Computing Machinery, 2018. (HPG '18), p. 1–4. DOI: 10.1145/3231578.3231581. Disponível em: https://doi.org/10.1145/3231578.3231581. Acesso em: 29 out. 2026.

CHUNG, SeungWoo *et al.* Comparison of BVH and KD-tree for the GPGPU acceleration on real mobile devices. *In*: HUNG, Jason C.; YEN, Neil Y.; HUI, Lin (Ed.). **International Conference on Frontier Computing**. Singapore: Springer Singapore, 2018. p. 535–540. DOI: 10.1007/978-981-13-3648-5_62.

CPPREFERENCE. **Templates**, 2026. Disponível em: https://en.cppreference.com/cpp/language/templates. Acesso em: 18 outubro 2026.

DAVIDOVIČ, Tomáš *et al.* 3d rasterization: a bridge between rasterization and ray casting. *In*: **Proceedings of Graphics Interface 2012**. CAN: Canadian Information Processing Society, 2012. (GI '12), p. 201–208. DOI: 10.5555/2305276.2305310.

DUPUY, Jonathan; BENYOUB, Anis. Sampling visible ggx normals with spherical caps. *In*: WILEY ONLINE LIBRARY. **Computer Graphics Forum**. 2023. Disponível em: https://arxiv.org/abs/2306.05044. Acesso em: 30 out. 2026.

FRANCHITTI, Jean-Claude. **Introduction to Computer Science**. Houston, Texas: OpenStax, 2024. Disponível em: https://openstax.org/books/introduction-computer-science. Acesso em: 3 nov. 2026.

FREUD, N. *et al.* Fast and robust ray casting algorithms for virtual x-ray imaging. **Nuclear Instruments and Methods in Physics Research Section B: Beam Interactions with Materials and Atoms**, Elsevier, v. 248, n. 1, p. 175–180, 2006. DOI: 10.1016/j.nimb.2006.03.009. Disponível em: https://www.sciencedirect.com/science/article/pii/S0168583X0600245X. Acesso em: 29 out. 2026.

GARANZHA, Kirill *et al.* Grid-based sah bvh construction on a gpu. **The Visual Computer**, v. 27, p. 697–706, 06 2011. DOI: 10.1007/s00371-011-0593-8. Disponível em: https://doi.org/10.1007/s00371-011-0593-8. Acesso em: 29 out. 2026.

GLFW. **GLFW Documentation**, 2026. Disponível em: https://www.glfw.org/documentation.html. Acesso em: 16 outubro 2026.

GONçALVES, Ademar. **Ray Tracing Interactivo: Estrutura de Aceleração**. Braga, 2009. Disponível em: https://www.di.uminho.pt/~prh/uce15-0809/g22.pdf. Acesso em: 3 nov. 2026.

GREENE, Ned; KASS, Michael; MILLER, Gavin. Hierarchical z-buffer visibility. *In*: **Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques**. New York, NY, USA: Association for Computing Machinery, 1993. (SIGGRAPH '93), p. 231–238. DOI: 10.1145/166117.166147. Disponível em: https://doi.org/10.1145/166117.166147. Acesso em: 29 out. 2026.

JAKOB, J.; GUTHE, M. Optimizing lbvh-construction and hierarchy-traversal to accelerate knn queries on point clouds using the gpu. **Computer Graphics Forum**, v. 40, n. 1, p. 124–137, 2021. DOI: 10.1111/cgf.14177. Disponível em: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.14177. Acesso em: 30 out. 2026.

Khronos Group. **Vulkan Documentation**, 2026. Disponível em: https://docs.vulkan.org/spec/latest/index.html. Acesso em: 23 abr. 2026.

LAINE, Samuli; KARRAS, Tero. High-performance software rasterization on gpus. *In*: **Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics**. New York, NY, USA: Association for Computing Machinery, 2011. (HPG '11), p. 79–88. ISBN 9781450308960. DOI: 10.1145/2018323.2018337. Disponível em: https://doi.org/10.1145/2018323.2018337. Acesso em: 29 out. 2026.

LARSSON, Thomas; AKENINE-MÖLLER, Tomas. A dynamic bounding volume hierarchy for generalized collision detection. **Computers & Graphics**, Elsevier, USA, v. 30, n. 3, p. 450–459, 2006. DOI: 10.1016/j.cag.2006.02.011. Disponível em: https://doi.org/10.1016/j.cag.2006.02.011. Acesso em: 30 out. 2026.

MARSCHNER, Steve; SHIRLEY, Peter. **Fundamentals of Computer Graphics**. 5th. ed. New York: A K Peters/CRC Press, 2021. 716 p.

MEISTER, Daniel; BITTNER, Jiří. Performance comparison of bounding volume hierarchies for GPU ray tracing. **Journal of Computer Graphics Techniques (JCGT)**, v. 11, n. 4, p. 1–19, October 2022. Disponível em: http://jcgt.org/published/0011/04/01/. Acesso em: 30 out. 2026.

MEISTER, Daniel *et al.* A survey on bounding volume hierarchies for ray tracing. **Computer Graphics Forum**, Wiley, v. 40, n. 2, p. 683–712, 2021. DOI: 10.1111/cgf.142662. Disponível em: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.142662. Acesso em: 29 out. 2026.

NVIDIA. **Nsight Graphics — GPU Trace UI**, 2026. Disponível em: https://docs.nvidia.com/nsight-graphics/UserGuide/gpu-trace-ui.html. Acesso em: 09 maio 2026.

____. **NVIDIA Nsight Systems User Guide**, 2026. Disponível em: https://docs.nvidia.com/nsight-systems/UserGuide/index.html. Acesso em: 15 outubro 2026.

____. **NVTX**, 2026. Disponível em: https://github.com/NVIDIA/NVTX. Acesso em: 21 outubro 2026.

____. **Post-Collection Analysis Guide**, 2026. Disponível em: https://docs.nvidia.com/nsight-systems/AnalysisGuide/index.html. Acesso em: 22 outubro 2026.

NVIDIA Corporation. **CUDA C++ Programming Guide**. 11.8. ed. [S.l.], 2022. Disponível em: https://docs.nvidia.com/cuda/archive/11.8.0/pdf/CUDA_C_Programming_Guide.pdf. Acesso em: 29 out. 2026.

PANTALEONI, Jacopo; LUEBKE, David. Hlbvh: Hierarchical lbvh construction for real-time ray tracing of dynamic geometry. *In*: **Proceedings of the Conference on High Performance Graphics**. Goslar, DEU: Eurographics Association, 2010. p. 87–95.

PINEDA, Juan. A parallel algorithm for polygon rasterization. *In*: **Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques**. New York, NY, USA: Association for Computing Machinery, 1988. (SIGGRAPH '88), p. 17–20. DOI: 10.1145/54852.378457. Disponível em: https://doi.org/10.1145/54852.378457. Acesso em: 29 out. 2026.

RUBIN, Steven M; WHITTED, Turner. A 3-dimensional representation for fast rendering of complex scenes. *In*: **Proceedings of the 7th annual conference on Computer graphics and interactive techniques**. New York, NY, USA: Association for Computing Machinery, 1980. (SIGGRAPH '80), p. 110–116. DOI: 10.1145/800250.807479. Disponível em: https://doi.org/10.1145/800250.807479. Acesso em: 29 out. 2026.

SCHUSTER, Kersten *et al.* Compression and rendering of textured point clouds via sparse coding. *In*: **Proceedings of the Conference on High-Performance Graphics**. Goslar, DEU: Eurographics Association, 2022. (HPG '21), p. 61–73. DOI: 10.2312/hpg.20211284. Disponível em: https://doi.org/10.2312/hpg.20211284. Acesso em: 29 out. 2026.

SCHWÄRZLER, Michael *et al.* Fast Accurate Soft Shadows with Adaptive Light Source Sampling. *In*: **Proceedings of the 17th International Workshop on Vision, Modeling, and Visualization (VMV 2012)**. Magdeburg, Germany: Eurographics Association, 2012. p. 39–46. DOI: 10.2312/PE/VMV/VMV12/039-046. Disponível em: https://doi.org/10.2312/PE/VMV/VMV12/039-046. Acesso em: 30 out. 2026.

SONG, Yupu *et al.* A parallel canny edge detection algorithm based on opencl acceleration. **PLOS ONE**, Public Library of Science, v. 19, n. 1, p. 1–31, 01 2024. DOI: 10.1371/journal.pone.0292345. Disponível em: https://doi.org/10.1371/journal.pone.0292345.

SrPatsu21. **Apotheosis**, 2026. Disponível em: https://github.com/SrPatsu21/Apotheosis/tree/c2a8f06305483b138f14c58bbcccbfe16577f394. Acesso em: 16 outubro 2026.

STIBORA, Radek. **Building of SBVH on Graphical Hardware**. Tese (Doutorado) — Master's thesis, Faculty of Informatics, Masaryk University, 2016. Disponível em: https://api.semanticscholar.org/CorpusID:131761871. Acesso em: 29 out. 2026.

STICH, Martin; FRIEDRICH, Heiko; DIETRICH, Andreas. Spatial splits in bounding volume hierarchies. *In*: **Proceedings of the Conference on High Performance Graphics 2009**. New York, NY, USA: Association for Computing Machinery, 2009. (HPG '09), p. 7–13. DOI: 10.1145/1572769.1572771. Disponível em: https://doi.org/10.1145/1572769.1572771. Acesso em: 29 out. 2026.

TSAKOK, John A. Faster incoherent rays: Multi-bvh ray stream tracing. *In*: **Proceedings of the Conference on High Performance Graphics 2009**. New York, NY, USA: Association for Computing Machinery, 2009. (HPG '09), p. 151–158. DOI: 10.1145/1572769.1572793. Disponível em: https://doi.org/10.1145/1572769.1572793. Acesso em: 29 out. 2026.

VAIDYANATHAN, K.; WOOP, S.; BENTHIN, C. Wide bvh traversal with a short stack. *In*: **Proceedings of the Conference on High-Performance Graphics**. Goslar, DEU: Eurographics Association, 2022. (HPG '19), p. 15–19. DOI: 10.2312/hpg.20191190. Disponível em: https://doi.org/10.2312/hpg.20191190. Acesso em: 29 out. 2026.

Vulkan Documentation Project. **Acceleration Structures (BLAS/TLAS)**, 2025. Disponível em: https://docs.vulkan.org/tutorial/latest/courses/18_Ray_tracing/02_Acceleration_structures.html. Acesso em: 15 outubro 2026.

____. **TLAS Animation**, 2025. Disponível em: https://docs.vulkan.org/tutorial/latest/courses/18_Ray_tracing/04_TLAS_animation.html. Acesso em: 15 outubro 2026.

Vulkan.org. **Buffer Device Address Alignment**, 2026. Disponível em: https://docs.vulkan.org/guide/latest/buffer_device_address_alignment.html. Acesso em: 20 outubro 2026.

____. **Index buffer)**, 2026. Disponível em: https://docs.vulkan.org/tutorial/latest/04_Vertex_buffers/03_Index_buffer.html. Acesso em: 20 outubro 2026.

____. **Shader Memory Layout**, 2026. Disponível em: https://docs.vulkan.org/guide/latest/shader_memory_layout.html. Acesso em: 20 outubro 2026.

____. **vkCmdDrawIndexed(3)**, 2026. Disponível em: https://docs.vulkan.org/refpages/latest/refpages/source/vkCmdDrawIndexed.html. Acesso em: 18 outubro 2026.

WANG, Zijin. The Development of Ray Tracing and Its Future. *In*: XING, Ying; RATNARAJAH, Tharmalingam (Ed.). **Proceedings of The 2nd International Conference on Signal Processing and Machine Learning, Virtual Event, Oxford, United Kingdom, May 12–18, 2022**. CEUR-WS.org, 2022. (CEUR Workshop Proceedings, v. 3150), p. 19–28. Disponível em: https://ceur-ws.org/Vol-3150/short3.pdf. Acesso em: 29 out. 2026.

WYVILL, Geoff. **Practical Ray Tracing**, 1995. Course notes, Computer Graphics International 1995. Disponível em: https://www.researchgate.net/profile/Geoff-Wyvill/publication/266178977_Practical_Ray_Tracing/links/561a12f008ae78721f9de7d8/Practical-Ray-Tracing.pdf. Acesso em: 29 out. 2026.